Bootstrap_YUC

January 20, 2025

[1]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
import math
import random
import seaborn as sns
import matplotlib.pyplot as plt
# Sirve para medir la velocidad de una función
import time
%load_ext line_profiler
```

0.1 Funciones auxiliares

[2]:
```
def inter_prob_l(arr):
    return np.quantile(arr, 0.025)

def inter_prob_u(arr):
    return np.quantile(arr, 0.975)

# Función para crear vectores Dirichlet
def f(x,y):
    return np.random.dirichlet(np.ones(x), y)

# Función para crear vectores Dirichlet con distintos valores (no solo unos)
def f_a(x,y):
    # Vemos si tiene valores con cero
    if np.where(x==0)[0].size==0:
        z_c=np.random.dirichlet(x, y)
    else:
        x_sc=np.delete(x,np.where(x==0)[0])
        # El vector Dirichlet con los valores que no son cero
        z=np.random.dirichlet(x_sc, y)
        # Insertar los valores ceros al vector
        indices=np.where(x==0)[0]
        i = indices - np.arange(len(indices))
        z_c=np.insert(arr=z, obj=i, values=0, axis=1)
    return z_c
```

```python
# Funciones para agregar unos
def aggregar_unos(array_ceros, indices):

    # Hacemos el conteo de los índices
    unique, counts = np.unique(indices, return_counts=True)

    #array_ceros[indices]=1
    array_ceros[unique]=counts

    return array_ceros

# Función para hacer el conteo del array
def bincount(a):
    y = np.bincount(a)
    ii = np.nonzero(y)[0]
    return np.vstack((ii, y[ii])).T
```

[3]:
```python
# Función para obtener totales que sumen un número dado dado las proporciones y␣
↪el número dado
def obtener_totales(array_proporciones, numero_dado):

    '''
    array_proporciones: Las proporciones por cada estrato (es un np array)
    numero_dado: Número dado que es el tamaño de la muestra
    '''

    # Calculamos los totales de las proporciones preliminares
    totales_proporciones=np.round(array_proporciones*numero_dado)
    totales_proporciones_sr=array_proporciones*numero_dado

    # Si supera el valor necesitado
    array_sup_valor=np.
↪where((totales_proporciones-totales_proporciones_sr)<=0,0,totales_proporciones-totales_prop

    # Agregamos para el caso que sea la suma cero
    array_sup_valor=np.where((array_sup_valor == 0).all(),1,array_sup_valor)

    # Agregamos (o quitamos) valores si faltan (o sobran)
    totales_proporciones = np.where(totales_proporciones.sum()==numero_dado,␣
↪totales_proporciones,
                                np.where(totales_proporciones.sum()<numero_dado,
                                        totales_proporciones + aggregar_unos(np.
↪zeros(totales_proporciones.shape[0]), np.random.choice(np.
↪arange(totales_proporciones.shape[0]), size =␣
↪abs(int(numero_dado-totales_proporciones.sum())), p=array_proporciones)),
```

```
                                        totales_proporciones - aggregar_unos(np.
↪zeros(totales_proporciones.shape[0]), np.random.choice(np.
↪arange(totales_proporciones.shape[0]), size = abs(int(totales_proporciones.
↪sum()-numero_dado)), p=array_sup_valor/array_sup_valor.sum()))))

    # Regresa los totales de las proporciones
    return totales_proporciones.astype(int)
```

[4]:
```
# Recibe las proporciones
def metricas_bootstrap(prop_reales, inter_prob, est_punt, porcenta_tama):
    '''
    prop_reales: Los datos reales de la elección.
    inter_prob: Los intervalos calculados por el bootstrap (bayesiano).
    est_punt: Las estimaciones puntuales del bootstrap (bayesiano).

    Regresa:

    df_metr_boot: Base con todas las métricas del bootstrap (bayesiano).
    df_error_max: Base con los errores máximos
    df_can_cob: El porcentaje de cobertura de cada candidato

    '''
    # Guardamos los resultados reales en un data frame
    resultados_reales=prop_reales.reset_index()
    resultados_reales=resultados_reales.rename(columns={'index':'Candidato', 0:
↪'Porcentaje_real'})
    resultados_reales

    # Lista auxiliar
    df_lista_esta=[]
    for i in range(int(inter_prob.shape[1]/2)):
        df_lista_inter=pd.DataFrame(inter_prob[:,[i,i+5]],␣
↪columns=["cuantil_025", "cuantil_975"])
        ␣
↪df_lista_inter["Longitud_intervalo"]=df_lista_inter["cuantil_975"]-df_lista_inter["cuantil_
        df_lista_inter["Estamacion_puntual"]=est_punt[:,i]
        df_lista_inter["Porcentaje_real"]=resultados_reales.
↪loc[i,"Porcentaje_real"]
        df_lista_inter["Candidato"]=resultados_reales.loc[i,"Candidato"]
        df_lista_inter=df_lista_inter.reset_index().rename(columns={'index':
↪'Num_bootstrap'})
        df_lista_inter["Num_bootstrap"]=df_lista_inter["Num_bootstrap"]+1
        ␣
↪df_lista_inter["Error"]=abs(df_lista_inter["Estamacion_puntual"]-df_lista_inter["Porcentaje
        df_lista_inter["Cobertura"]=np.where(np.
↪logical_and(df_lista_inter["Porcentaje_real"]>=df_lista_inter["cuantil_025"],df_lista_inter
```

```
        df_lista_inter["Porcenta_tama"]=porcenta_tama
        df_lista_esta.append(df_lista_inter)

    # Base con todos los datos
    df_metr_boot=pd.concat(df_lista_esta).reset_index(drop=True)

    # Calulo de error máximo por bootstrap
    df_error_max=df_metr_boot.groupby(["Num_bootstrap"]).agg({'Error':'max'}).
↪reset_index().rename(columns={'Error':'Error_max'})
    df_error_max["Porcenta_tama"]=porcenta_tama
    # Calculamos la cobertura por candidato
    df_can_cob=df_metr_boot.groupby(["Candidato"]).agg({'Cobertura':'sum'}).
↪reset_index().rename(columns={'Cobertura':'Numero_total_cob'})
    df_can_cob["Cobertura"]=df_can_cob["Numero_total_cob"]/
↪df_error_max["Num_bootstrap"].max()
    df_can_cob["Porcenta_tama"]=porcenta_tama
    df_can_cob

    return df_metr_boot, df_error_max, df_can_cob
```

0.2 Funciones muestreo estratificado (con número de datos específico)

[10]:
```
def __smpl_size(population, size):
    '''
    A function to compute the sample size. If not informed, a sampling
    size will be calculated using Cochran adjusted sampling formula:
        cochran_n = (Z**2 * p * q) /e**2
        where:
            - Z is the z-value. In this case we use 1.96 representing 95%
            - p is the estimated proportion of the population which has an
              attribute. In this case we use 0.5
            - q is 1-p
            - e is the margin of error
        This formula is adjusted as follows:
        adjusted_cochran = cochran_n / 1+((cochran_n -1)/N)
        where:
            - cochran_n = result of the previous formula
            - N is the population size
    Parameters
    ----------
        :population: population size
        :size: sample size (default = None)
    Returns
    -------
    Calculated sample size to be used in the functions:
        - stratified_sample
        - stratified_sample_report
```

```
    '''
    if size is None:
        cochran_n = round(((1.96)**2 * 0.5 * 0.5)/ 0.02**2)
        n = round(cochran_n/(1+((cochran_n -1) /population)))
    elif size >= 0 and size < 1:
        n = round(population * size)
    elif size < 0:
        raise ValueError('Parameter "size" must be an integer or a proportion␣
↪between 0 and 0.99.')
    elif size >= 1:
        n = size
    return n
```

[11]:
```
# the functions:
def stratified_sample(df, strata, size=None, seed=None, keep_index= True):
    '''
    It samples data from a pandas dataframe using strata. These functions use
    proportionate stratification:
    n1 = (N1/N) * n
    where:
        - n1 is the sample size of stratum 1
        - N1 is the population size of stratum 1
        - N is the total population size
        - n is the sampling size
    Parameters
    ----------
    :df: pandas dataframe from which data will be sampled.
    :strata: list containing columns that will be used in the stratified␣
↪sampling.
    :size: sampling size. If not informed, a sampling size will be calculated
        using Cochran adjusted sampling formula:
        cochran_n = (Z**2 * p * q) /e**2
        where:
            - Z is the z-value. In this case we use 1.96 representing 95%
            - p is the estimated proportion of the population which has an
                attribute. In this case we use 0.5
            - q is 1-p
            - e is the margin of error
        This formula is adjusted as follows:
        adjusted_cochran = cochran_n / 1+((cochran_n -1)/N)
        where:
            - cochran_n = result of the previous formula
            - N is the population size
    :seed: sampling seed
    :keep_index: if True, it keeps a column with the original population index␣
↪indicator
```

```
    Returns
    -------
    A sampled pandas dataframe based in a set of strata.
    Examples
    --------
    >> df.head()
            id  sex age city
    0        123 M   20  XYZ
    1        456 M   25  XYZ
    2        789 M   21  YZX
    3        987 F   40  ZXY
    4        654 M   45  ZXY
    ...
    # This returns a sample stratified by sex and city containing 30% of the␣
↪size of
    # the original data
    >> stratified = stratified_sample(df=df, strata=['sex', 'city'], size=0.3)
    Requirements
    ------------
    - pandas
    - numpy
    '''
    population = len(df)
    size = __smpl_size(population, size)
    tmp = df[strata]
    tmp['size'] = 1
    tmp_grpd = tmp.groupby(strata).count().reset_index()
    tmp_grpd['samp_size'] = round(size/population * tmp_grpd['size']).
↪astype(int)
    print(size)
    list_aux=[]

    # controlling variable to create the dataframe or append to it
    first = True
    for i in range(len(tmp_grpd)):
        # query generator for each iteration
        qry=''
        for s in range(len(strata)):
            stratum = strata[s]
            value = tmp_grpd.iloc[i][stratum]
            n = tmp_grpd.iloc[i]['samp_size']

            if type(value) == str:
                value = "'" + str(value) + "'"

            if s != len(strata)-1:
                qry = qry + stratum + ' == ' + str(value) +' & '
```

```
            else:
                qry = qry + stratum + ' == ' + str(value)

        # final dataframe
        if first:
            stratified_df = df.query(qry).sample(n=n, random_state=seed)
            first = False
        else:
            tmp_df = df.query(qry).sample(n=n, random_state=seed)
            list_aux.append(tmp_df)

    # Guardamos todo en una base
    df_aux=pd.concat(list_aux)
    stratified_df=pd.concat([stratified_df,df_aux], axis=0)

    return stratified_df
```

[12]:
```
def muestreo_estratificado(df, strata, sample_size=250, seed=None):

    population = len(df)
    population


    # Estrata valores
    if len(strata)==1:
        strata_val=strata[0]
    else:
        strata_val=strata
    # Tamaño de la muestra
    sample_size=sample_size
    # size = __smpl_size(population, size)
    tmp = df[strata]
    tmp['size'] = 1
    # Conteos de los estratos
    tmp_grpd = tmp.groupby(strata).count().reset_index()

    # Obtenemos los índices de los estratos
    indices_estrato=df.reset_index().groupby(strata_val)['index'].apply(list).
↪reset_index(drop=True)
    pru=df.reset_index()[strata]
    # Los pasamos a la lista
    indices_dist=[np.array(indices_estrato[xi]) for xi in range(indices_estrato.
↪shape[0])]

    # Proporciones de los estratos
    tmp_grpd_prop=tmp_grpd.copy()
    tmp_grpd_prop['size']=tmp_grpd_prop['size']/tmp_grpd_prop['size'].sum()
```

```
    #tmp_grpd = tmp.groupby(['DISTRITO']).count().reset_index()/
    tmp_grpd['samp_size'] = round(sample_size/population * tmp_grpd['size']).
↪astype(int)
    tmp_grpd_array=np.array(tmp_grpd["samp_size"])
    if tmp_grpd["samp_size"].sum()==sample_size:
        pass
    # Agregamos un valor extra (o valores extra)
    elif tmp_grpd["samp_size"].sum()<sample_size:
        # Creamos un vector para agregar el (o los) valores extra
        vector_extra=np.zeros(tmp_grpd[strata].shape[0])
        # Seleccionamos de manera aleatoria
        valor_extra=np.random.choice(np.arange(tmp_grpd[strata_val].
↪shape[0])+1, size = sample_size-tmp_grpd["samp_size"].sum(),␣
↪p=tmp_grpd_prop['size'])
        vector_extra[valor_extra-1]=1
        # Sumamos el valor extra
        tmp_grpd["samp_size"]=(tmp_grpd["samp_size"]+vector_extra).astype(int)
    # Quitamos un valor extra (o valores extra)
    else:
        # Creamos un vector para agregar el (o los) valores extra
        vector_extra=np.zeros(tmp_grpd[strata].shape[0])
        # Seleccionamos de manera aleatoria
        valor_extra=np.random.choice(np.arange(tmp_grpd[strata_val].
↪shape[0])+1, size = tmp_grpd["samp_size"].sum()-sample_size,␣
↪p=tmp_grpd_prop['size'])
        vector_extra[valor_extra-1]=-1
        # Sumamos el valor extra
        tmp_grpd["samp_size"]=(tmp_grpd["samp_size"]+vector_extra).astype(int)

    tmp_grpd_array=np.array(tmp_grpd["samp_size"])
    # Lista auxiliar
    lista_aux=[]
    for i in range(tmp_grpd.shape[0]):
        tmp_df = np.random.
↪choice(indices_dist[i],tmp_grpd_array[i],replace=False)
        lista_aux.append(tmp_df)

    # Indices de la muestr
    indices_muestra=np.concatenate(lista_aux)
    # Seleccionamos todos los indices
    df_mue_est=df.loc[indices_muestra]

    return df_mue_est
```

0.2.1 Mejor de todas las funciones para estratificar (con una variable)

[13]:
```
def muestreo_estratificado_mej(df, strata, sample_size=250, seed=None):

    # seed=None
    # sample_size=400
    # df=df_act_yuc_i.copy()
    population = len(df)

    strata=["DISTRITO"]

    # Estrata valores
    # if len(strata)==1:
    #     strata_val=strata[0]
    # else:
    #     strata_val=strata

    # Tamaño de la muestra
    sample_size=sample_size
    # Los estratos como numpy arra
    tmp = df[strata]

    # Hacemos los conteos de los estratos
    unique, counts = np.unique(np.array(tmp), return_counts=True)
    # Creamos el vector para guardar las proporciones
    tmp_grpd=np.zeros((unique.shape[0], 4))
    tmp_grpd[:,[0,1]]=np.asarray((unique, counts)).T

    # Los índices de cada estrato en array
    array_index_estrato=np.array(tmp.reset_index()[['index']+ strata])

    # Los pasamos a la lista
    indices_dist=[array_index_estrato[array_index_estrato[:,1]==i][:,0] for i␣
↪in range(1,unique.shape[0]+1)]

    # Proporciones de los estratos
    tmp_grpd[:,2] = tmp_grpd[:,1]/tmp_grpd[:,1].sum()
    # Obtenemos los valores de las muestras por cada estrato
    tmp_grpd[:,3] = np.round((sample_size/population)*tmp_grpd[:,1],0)

    # Si supera el valor necesitado
    array_sup_valor=np.where(np.round((sample_size/population)*tmp_grpd[:
↪,1],0)-(sample_size/population)*tmp_grpd[:,1]<=0,0,np.round((sample_size/
↪population)*tmp_grpd[:,1],0)-(sample_size/population)*tmp_grpd[:,1])

    # Agregamos para el caso que sea la suma cero
    array_sup_valor=np.where((array_sup_valor == 0).all(),1,array_sup_valor)
```

```
    # Agregamos (o quitamos) valores si faltan (o sobran)
    tmp_grpd[:,3] = np.where(tmp_grpd[:,3].sum()==sample_size, tmp_grpd[:,3],
                             np.where(tmp_grpd[:,3].sum()<sample_size,
                                      tmp_grpd[:,3]+ aggregar_unos(np.
↪zeros(tmp_grpd.shape[0]), np.random.choice(np.arange(tmp_grpd.shape[0]),␣
↪size = abs(int(sample_size-tmp_grpd[:,3].sum())), p=tmp_grpd[:,2])),
                                      tmp_grpd[:,3]- aggregar_unos(np.
↪zeros(tmp_grpd.shape[0]), np.random.choice(np.arange(tmp_grpd.shape[0]),␣
↪size = abs(int(tmp_grpd[:,3].sum()-sample_size)), p=array_sup_valor/
↪array_sup_valor.sum()))))

    # Lista auxiliar
    lista_aux=[]
    for i in range(tmp_grpd.shape[0]):
        tmp_df = np.random.
↪choice(indices_dist[i],int(tmp_grpd[i,3]),replace=False)
        lista_aux.append(tmp_df)

    # Indices de la muestr
    indices_muestra=np.concatenate(lista_aux)
    # Seleccionamos todos los indices
    df_mue_est_j=df.loc[indices_muestra]

    return df_mue_est_j
```

0.3 Gubernatura Yucatán

[15]:
```
# Leemos los datos
df_act_yuc=pd.read_excel(r"/Users/damian/Documents/Tesis maestria/Bootstraps/
↪Bases/Casilla_por_casilla_gubernatura_280627_YUC.xlsx")
df_act_yuc
```

[15]:
```
      ID ESTADO NOMBRE ESTADO CABECERA DISTRITAL LOCAL  ID MUNICIPIO  \
0            31       YUCATÁN                     UMAN             1
1            31       YUCATÁN                     UMAN             1
2            31       YUCATÁN                     UMAN             1
3            31       YUCATÁN                     UMAN             1
4            31       YUCATÁN                     UMAN             1
…          …          …                       …            …
2974         31       YUCATÁN                    MOTUL            54
2975         31       YUCATÁN                    MOTUL            93
2976         31       YUCATÁN                  TIZIMIN            96
2977         31       YUCATÁN                  TIZIMIN            96
2978         31       YUCATÁN                  TEMOZON            19

      DISTRITO   MUNICIPIO  SECCION  CASILLA TIPO DE CASILLA  ID CASILLA  …  \
```

```
0           12       ABALA        1  '0001B1               B           1  …
1           12       ABALA        1  '0001C1               C           2  …
2           12       ABALA        1  '0001C2               C           3  …
3           12       ABALA        2  '0002B1               B           4  …
4           12       ABALA        3   0003B1               B           5  …
…        …         …      …      …             …         …  …
2974        15     MUXUPIP     1203   1203V1              VA        2975  …
2975        15    TIXKOKOB     1204   1204V1              VA        2976  …
2976        17  SAN FELIPE     1205   1205V1              VA        2977  …
2977        17     TIZIMIN     1206   1206V1              VA        2978  …
2978        18      CHEMAX     1207  '1207V1              VA        2979  …

      PVEM-PT-MORENA PVEM-PT PVEM-MORENA  PT-MORENA  \
0                2.0     1.0         0.0        0.0
1                1.0     0.0         2.0        0.0
2                5.0     0.0         2.0        0.0
3                0.0     0.0         0.0        1.0
4                NaN     NaN         NaN        NaN
…              …     …         …         …
2974             NaN     NaN         NaN        NaN
2975             NaN     NaN         NaN        NaN
2976             NaN     NaN         NaN        NaN
2977             NaN     NaN         NaN        NaN
2978             NaN     NaN         NaN        NaN

      CANDIDATOS NO REGISTRADOS  VOTOS NULOS  LISTA NOMINAL  TOTAL  \
0                           0.0         12.0          596.0  532.0
1                           0.0         11.0          596.0  547.0
2                           0.0         13.0          595.0  542.0
3                           0.0          3.0          174.0  162.0
4                           NaN          NaN          205.0    NaN
…                         …          …            …    …
2974                        NaN          NaN            NaN    NaN
2975                        NaN          NaN            NaN    NaN
2976                        NaN          NaN            NaN    NaN
2977                        NaN          NaN            NaN    NaN
2978                        NaN          NaN            NaN    1.0

      PARTICIPACIÓN  Unnamed: 34
0          0.892617          NaN
1          0.917785          NaN
2          0.910924          NaN
3          0.931034          NaN
4          0.000000          NaN
…             …          …
2974            NaN          NaN
2975            NaN          NaN
```

```
2976            NaN          NaN
2977            NaN          NaN
2978            NaN          NaN

[2979 rows x 35 columns]
```

0.4 Limpieza de datos

[16]:
```
# Hay que revisar si se quitan los de SIN RESULTADO, por ahora solo quitaré los␣
↪PENDIENTE DE CAPTURA
#df_act_yuc_r=df_act_yuc.copy()
df_act_yuc_r=df_act_yuc[(~(df_act_yuc["ESTATUS ACTA"].isin(["PENDIENTE DE␣
↪CAPTURA","SIN RESULTADO"]))) & (df_act_yuc["TOTAL"]>0)].
↪reset_index(drop=True)
# Quitamos la última columna
df_act_yuc_r=df_act_yuc_r.drop(df_act_yuc_r.columns[len(df_act_yuc_r.
↪columns)-1], axis=1)
# Llenamos los missing con cero de los votos y totales
df_act_yuc_r.iloc[:,13:]=df_act_yuc_r.iloc[:,13:].fillna(0)
df_act_yuc_r
```

[16]:
```
      ID ESTADO NOMBRE ESTADO CABECERA DISTRITAL LOCAL  ID MUNICIPIO  \
0            31       YUCATÁN                     UMAN             1
1            31       YUCATÁN                     UMAN             1
2            31       YUCATÁN                     UMAN             1
3            31       YUCATÁN                     UMAN             1
4            31       YUCATÁN                     UMAN             1
…          …          …                       …            …
2836         31       YUCATÁN                   MERIDA            50
2837         31       YUCATÁN                   MERIDA            50
2838         31       YUCATÁN                   MERIDA            50
2839         31       YUCATÁN                   MERIDA            50
2840         31       YUCATÁN                  TEMOZON            19

      DISTRITO MUNICIPIO  SECCION  CASILLA TIPO DE CASILLA  ID CASILLA  …  \
0           12     ABALA        1  '0001B1               B           1  …
1           12     ABALA        1  '0001C1               C           2  …
2           12     ABALA        1  '0001C2               C           3  …
3           12     ABALA        2  '0002B1               B           4  …
4           12     ABALA        4  '0004B1               B           6  …
…         …        …       …       …              …          …  …
2836         3    MERIDA     1195  '1195V1              VA        2967  …
2837         5    MERIDA     1197  '1197V1              VA        2969  …
2838         8    MERIDA     1199  '1199V1              VA        2971  …
2839         9    MERIDA     1200  '1200V1              VA        2972  …
2840        18    CHEMAX     1207  '1207V1              VA        2979  …
```

```
      PRI-NAY PVEM-PT-MORENA PVEM-PT  PVEM-MORENA  PT-MORENA  \
0         0.0            2.0     1.0          0.0        0.0
1         0.0            1.0     0.0          2.0        0.0
2         0.0            5.0     0.0          2.0        0.0
3         0.0            0.0     0.0          0.0        1.0
4         0.0            4.0     0.0          0.0        0.0
…       …             …     …          …         …
2836      0.0            0.0     0.0          0.0        0.0
2837      0.0            0.0     0.0          0.0        0.0
2838      0.0            0.0     0.0          0.0        0.0
2839      0.0            0.0     0.0          0.0        0.0
2840      0.0            0.0     0.0          0.0        0.0

      CANDIDATOS NO REGISTRADOS  VOTOS NULOS  LISTA NOMINAL  TOTAL  \
0                           0.0         12.0          596.0  532.0
1                           0.0         11.0          596.0  547.0
2                           0.0         13.0          595.0  542.0
3                           0.0          3.0          174.0  162.0
4                           0.0         14.0          550.0  513.0
…                         …           …             …    …
2836                        0.0          0.0            0.0    2.0
2837                        0.0          0.0            0.0    5.0
2838                        0.0          0.0            0.0    3.0
2839                        0.0          0.0            0.0    1.0
2840                        0.0          0.0            0.0    1.0

      PARTICIPACIÓN
0          0.892617
1          0.917785
2          0.910924
3          0.931034
4          0.932727
…             …
2836       0.000000
2837       0.000000
2838       0.000000
2839       0.000000
2840       0.000000

[2841 rows x 34 columns]
```

[17]:
```
import warnings
warnings.filterwarnings("ignore")
# Nos quedamos con las columnas necesarias
df_act_yuc_i=df_act_yuc_r[["ID ESTADO","NOMBRE␣
↪ESTADO","DISTRITO","SECCION","CASILLA","ID CASILLA","ESTATUS ACTA",'PAN',␣
↪'PRI', 'PRD', 'PVEM', 'PT', 'MC', 'MORENA',
```

```
        'NAY', 'PAN-PRI-NAY', 'PAN-PRI', 'PAN-NAY', 'PRI-NAY', 'PVEM-PT-MORENA',
       'PVEM-PT', 'PVEM-MORENA', 'PT-MORENA', 'CANDIDATOS NO REGISTRADOS',
       'VOTOS NULOS', 'LISTA NOMINAL', 'TOTAL', 'PARTICIPACIÓN']]
# Creamos los votos por coalición
# Candidato sigamos haciendo historia
df_act_yuc_i["JOAQUIN_DIAZ_MENA"]=df_act_yuc_i.loc[:,"MORENA"]+df_act_yuc_i.
 ↪loc[:,"PT"]+df_act_yuc_i.loc[:,"PVEM"]+df_act_yuc_i.loc[:
 ↪,"PVEM-MORENA"]+df_act_yuc_i.loc[:,"PVEM-PT"]+df_act_yuc_i.loc[:
 ↪,"PT-MORENA"]+df_act_yuc_i.loc[:,"PVEM-PT-MORENA"]
# Candidato Unidos gana Yucatán
df_act_yuc_i["RENAN_BARRERA_CONCHA"]=df_act_yuc_i.loc[:,"PAN"]+df_act_yuc_i.
 ↪loc[:,"PRI"]+df_act_yuc_i.loc[:,"NAY"]+df_act_yuc_i.loc[:
 ↪,"PAN-PRI"]+df_act_yuc_i.loc[:,"PAN-NAY"]+df_act_yuc_i.loc[:
 ↪,"PRI-NAY"]+df_act_yuc_i.loc[:,"PAN-PRI-NAY"]
# Candidato MC
df_act_yuc_i["VIDA_ARAVARI_GOMEZ_HERRERA"]=df_act_yuc_i.loc[:,"MC"]
# Canidato PRD
df_act_yuc_i["YAMIL_JASMIN_LOPEZ_MANRIQUE"]=df_act_yuc_i.loc[:,"PRD"]
# Votos nulos y candidatos no registrados
df_act_yuc_i["VOTOS_NULOS_CAND_NO_REGIS"]=df_act_yuc_i.loc[:,"VOTOS␣
 ↪NULOS"]+df_act_yuc_i.loc[:,"CANDIDATOS NO REGISTRADOS"]
# Datos reales de la proporción
df_act_yuc_i
```

[17]:

```
      ID ESTADO NOMBRE ESTADO  DISTRITO  SECCION  CASILLA  ID CASILLA  \
0            31       YUCATÁN        12        1  '0001B1           1
1            31       YUCATÁN        12        1  '0001C1           2
2            31       YUCATÁN        12        1  '0001C2           3
3            31       YUCATÁN        12        2  '0002B1           4
4            31       YUCATÁN        12        4  '0004B1           6
…          …            …       …       …      …         …
2836         31       YUCATÁN         3     1195  '1195V1        2967
2837         31       YUCATÁN         5     1197  '1197V1        2969
2838         31       YUCATÁN         8     1199  '1199V1        2971
2839         31       YUCATÁN         9     1200  '1200V1        2972
2840         31       YUCATÁN        18     1207  '1207V1        2979

     ESTATUS ACTA    PAN    PRI  PRD  …  CANDIDATOS NO REGISTRADOS  \
0       COMPUTADA  171.0  107.0  3.0  …                        0.0
1       COMPUTADA  177.0  104.0  0.0  …                        0.0
2       COMPUTADA  176.0  100.0  1.0  …                        0.0
3       COMPUTADA   46.0   33.0  2.0  …                        0.0
4       COMPUTADA  110.0   96.0  7.0  …                        0.0
…           …    …     …   …   …                        …
2836    COMPUTADA    2.0    0.0  0.0  …                        0.0
2837    COMPUTADA    2.0    1.0  0.0  …                        0.0
2838    COMPUTADA    2.0    0.0  0.0  …                        0.0
```

```
2839    COMPUTADA    0.0    0.0  0.0  …                        0.0
2840    COMPUTADA    1.0    0.0  0.0  …                        0.0

      VOTOS NULOS  LISTA NOMINAL  TOTAL  PARTICIPACIÓN  JOAQUIN_DIAZ_MENA  \
0            12.0          596.0  532.0       0.892617              209.0
1            11.0          596.0  547.0       0.917785              208.0
2            13.0          595.0  542.0       0.910924              228.0
3             3.0          174.0  162.0       0.931034               71.0
4            14.0          550.0  513.0       0.932727              275.0
…             …             …     …            …                 …
2836          0.0            0.0    2.0       0.000000                0.0
2837          0.0            0.0    5.0       0.000000                2.0
2838          0.0            0.0    3.0       0.000000                1.0
2839          0.0            0.0    1.0       0.000000                1.0
2840          0.0            0.0    1.0       0.000000                0.0

      RENAN_BARRERA_CONCHA  VIDA_ARAVARI_GOMEZ_HERRERA  \
0                    290.0                        18.0
1                    299.0                        29.0
2                    289.0                        11.0
3                     79.0                         7.0
4                    207.0                        10.0
…                    …                          …
2836                   2.0                         0.0
2837                   3.0                         0.0
2838                   2.0                         0.0
2839                   0.0                         0.0
2840                   1.0                         0.0

      YAMIL_JASMIN_LOPEZ_MANRIQUE  VOTOS_NULOS_CAND_NO_REGIS
0                             3.0                       12.0
1                             0.0                       11.0
2                             1.0                       13.0
3                             2.0                        3.0
4                             7.0                       14.0
…                            …                         …
2836                          0.0                        0.0
2837                          0.0                        0.0
2838                          0.0                        0.0
2839                          0.0                        0.0
2840                          0.0                        0.0

[2841 rows x 33 columns]
```

[18]:
```
# Datos reales de la proporción
```

```
prop_reales=df_act_yuc_i[["JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRE
 ↪sum()/
 ↪df_act_yuc_i[["JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA","YAMI
 ↪sum().sum()
prop_reales
```

[18]:
```
JOAQUIN_DIAZ_MENA              0.515125
RENAN_BARRERA_CONCHA           0.421366
VIDA_ARAVARI_GOMEZ_HERRERA     0.036792
YAMIL_JASMIN_LOPEZ_MANRIQUE    0.005249
VOTOS_NULOS_CAND_NO_REGIS      0.021467
dtype: float64
```

[19]:
```
# Prueba del muestreo
pr=muestreo_estratificado_mej(df_act_yuc_i, ["DISTRITO"], sample_size=221,␣
 ↪seed=None)
pr
```

[19]:
```
      ID ESTADO NOMBRE ESTADO  DISTRITO  SECCION  CASILLA  ID CASILLA  \
1359         31       YUCATÁN         1      600  '0600B1        1404
1204         31       YUCATÁN         1      524  '0524B1        1243
1348         31       YUCATÁN         1      594  '0594B1        1393
830          31       YUCATÁN         1      345  '0345B1         859
978          31       YUCATÁN         1      420  '0420B1        1011
…          …            …        …       …       …         …
1668         31       YUCATÁN        21      699  '0699C1        1742
2047         31       YUCATÁN        21      885  '0885B1        2148
1970         31       YUCATÁN        21      847  '0847C2        2068
1661         31       YUCATÁN        21      696  '0696C3        1735
1682         31       YUCATÁN        21      705  '0705B1        1757

     ESTATUS ACTA    PAN    PRI  PRD  …  CANDIDATOS NO REGISTRADOS  \
1359    COMPUTADA  173.0   11.0  1.0  …                       0.0
1204    COMPUTADA  167.0   17.0  2.0  …                       0.0
1348    COMPUTADA  182.0   25.0  3.0  …                       0.0
830     COMPUTADA  115.0   19.0  4.0  …                       0.0
978     COMPUTADA  118.0   13.0  0.0  …                       0.0
…           …    …    …   …  …                         …
1668    COMPUTADA   56.0   96.0  2.0  …                       0.0
2047    COMPUTADA  150.0    5.0  3.0  …                       0.0
1970    COMPUTADA  236.0    1.0  0.0  …                       0.0
1661    COMPUTADA   23.0   83.0  1.0  …                       0.0
1682    COMPUTADA   14.0  205.0  1.0  …                       0.0

      VOTOS NULOS  LISTA NOMINAL  TOTAL  PARTICIPACIÓN  JOAQUIN_DIAZ_MENA  \
1359          5.0          691.0  425.0       0.615051              190.0
1204          8.0          702.0  433.0       0.616809              216.0
```

```
1348          8.0          716.0  467.0       0.652235              218.0
830           5.0          389.0  280.0       0.719794              116.0
978           8.0          504.0  307.0       0.609127              138.0
…           …            …    …           …                 …
1668          9.0          529.0  392.0       0.741021              211.0
2047         10.0          555.0  426.0       0.767568              244.0
1970          6.0          578.0  514.0       0.889273              267.0
1661         25.0          563.0  377.0       0.669627              216.0
1682         10.0          578.0  457.0       0.790657              221.0

      RENAN_BARRERA_CONCHA  VIDA_ARAVARI_GOMEZ_HERRERA  \
1359                 192.0                        37.0
1204                 192.0                        15.0
1348                 220.0                        18.0
830                  141.0                        14.0
978                  151.0                        10.0
…                    …                          …
1668                 165.0                         5.0
2047                 158.0                        11.0
1970                 238.0                         3.0
1661                 131.0                         4.0
1682                 221.0                         4.0

      YAMIL_JASMIN_LOPEZ_MANRIQUE  VOTOS_NULOS_CAND_NO_REGIS
1359                          1.0                        5.0
1204                          2.0                        8.0
1348                          3.0                        8.0
830                           4.0                        5.0
978                           0.0                        8.0
…                           …                        …
1668                          2.0                        9.0
2047                          3.0                       10.0
1970                          0.0                        6.0
1661                          1.0                       25.0
1682                          1.0                       10.0

[221 rows x 33 columns]
```

[20]:
```
#%lprun -f muestreo_estratificado_mej muestreo_estratificado_mej(df_act_yuc_i,␣
↪["DISTRITO"], sample_size=250, seed=None)
```

1 Bootstrap por casilla

[15]:
```
# Número de bootstraps
num_boot=1000
```

```
# Semilla para las muestras de la Dirichlet
rng = np.random.default_rng(2)

# Número total de casillas
n_cas=df_act_yuc_i.shape[0]



# Hacemos el muestreo de 10% de las casilas estratificado por distrito
#df_stra_sam=df_act_yuc_i.groupby("DISTRITO", group_keys=False).apply(lambda x:␣
↪x.sample(frac=0.09, random_state=22))[["DISTRITO","ID␣
↪CASILLA","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA","YAMIL_JAS
X_train, df_stra_sam = train_test_split(df_act_yuc_i[["DISTRITO","ID␣
↪CASILLA","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA","YAMIL_JAS
↪test_size=0.1)
n_boot=1000

# Vector de pesos de la distribución Dirichlet
#pesos_bootstrap=np.random.dirichlet(np.ones(df_stra_sam.shape[0]), n_boot)

pesos_bootstrap=np.random.dirichlet(np.ones(df_stra_sam.shape[0]), n_boot)

df_stra_sam_2=np.
↪array(df_stra_sam[["JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA",
# Data frame con los resultados
df_prop_list=[]

for j in range(n_boot):
    #print(X_test.shape[0])
    #rng_2 = np.random.default_rng(j)

    # Lista de las casillas muestreadas por el bootstrap
    df_casillas_boot=[]
    df_combined=pd.DataFrame()

    #cas_sel_boot=np.random.choice(a=df_stra_sam.index,  size=n_cas-df_stra_sam.
↪shape[0],  p=pesos_bootstrap[0], replace=True)

    #cas_sel_boot=np.random.choice(a=df_stra_sam.index,  size=n_cas-df_stra_sam.
↪shape[0],  replace=True)

    #cas_sel_boot=random.choices(list(df_stra_sam.index), weights = np.
↪exp(pesos_bootstrap[0]), k = n_cas-df_stra_sam.shape[0])
    #cas_sel_boot=np.random.choice(list(df_stra_sam.index), size = n_cas,␣
↪p=pesos_bootstrap[j])
```

```
    #cas_sel_boot=np.random.multinomial(n=n_cas, )

    kl=np.random.multinomial(n=n_cas, pvals=1/len(df_stra_sam.index)*np.
 ↪ones(len(df_stra_sam.index)))


    # Guardamos todas las casillas del bootstrap en un dataframe
    df_combined=df_stra_sam.loc[np.repeat(np.array(df_stra_sam.index),kl)]

    # Juntamos todo en una base (incluyendo la muestra original)
    df_todas_cas=pd.
 ↪concat([df_stra_sam,df_combined])[["JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI

    # La estimación puntual
    df_prop_est_p=df_todas_cas.sum(0)/df_todas_cas.sum(0).sum()
    df_prop_list.append(np.array(df_prop_est_p))

array_res=np.array(df_prop_list)
array_res
```

[15]:
```
array([[0.49793414, 0.43957706, 0.03765183, 0.00404808, 0.02078889],
       [0.49585599, 0.44190042, 0.03779041, 0.00411519, 0.02033799],
       [0.5004151 , 0.43790351, 0.03696379, 0.00407299, 0.02064461],
       …,
       [0.4985747 , 0.4406642 , 0.03619527, 0.00400962, 0.02055621],
       [0.49305884, 0.44617172, 0.03647961, 0.00407551, 0.02021433],
       [0.49919182, 0.43898996, 0.03761727, 0.00397457, 0.02022638]])
```

1.1 Función de bootstrap simple

[21]:
```
def bootstrap_simple(df, porcentaje_tam=0.1, num_boot: int=1000, n_sample=None,␣
 ↪frac=None):

    est_punt_list=[]

    inter_list=[]

    # Semilla para las muestras de la Dirichlet
    rng = np.random.default_rng(20)

    # Vector de pesos de la distribución Dirichlet
    #pesos_bootstrap=rng.dirichlet(np.ones(math.ceil(df.shape[0]*porcentaje)),␣
 ↪num_boot)
    #pesos_bootstrap=np.random.dirichlet(np.ones(math.ceil(df.
 ↪shape[0]*porcentaje_tam)), num_boot)

    # Número total de casillas
```

```
n_cas=df.shape[0]

rng_2 = np.random.default_rng(10)

for i in range(num_boot):

    df_stra_sam = df.sample(n=n_sample, frac=frac)

    pesos_bootstrap=np.random.dirichlet(np.ones(df_stra_sam.shape[0]), 1000)

    df_stra_sam=np.
↪array(df_stra_sam[["JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA",

    # Data frame con los resultados
    df_prop_list=[]

    for j in range(1000):
        #print(X_test.shape[0])
        #rng_2 = np.random.default_rng(j)

        # La casilla seleccionada por el bootstrap

        # Para completar toda la población
        #cas_sel_boot=np.random.choice(a=df_stra_sam.shape[0],  size=n_cas,␣
↪ p=pesos_bootstrap[j])
        #array_datos=df_stra_sam[cas_sel_boot]

        # Calculamos una multinomial para completar la población
        # Utilizando los pesos bootstrap para cada casilla
        # Como resultado nos da las veces que se repitió cada casilla en la␣
↪remuestra
        cas_sel_boot=np.random.multinomial(n=n_cas,␣
↪pvals=pesos_bootstrap[j])

        # Guardamos todas las casillas del bootstrap en un dataframe␣
↪repitiendo el valor dependiendo
        # de su valor en la multinomial
        # No incluimos a la muestra inicial (por lo que comentó la␣
↪profesora)
        array_datos=df_stra_sam[np.repeat(np.arange(df_stra_sam.
↪shape[0]),cas_sel_boot)]

        # Para completar toda la población (en caso de que si se incluya la␣
↪muestra)
        # array_casos=np.concatenate((df_stra_sam,array_datos))
```

```
            # La estimación puntual
            array_prop_i=array_datos.sum(0)/array_datos.sum()
            #df_prop_est_p=df_todas_cas.sum(0)/df_todas_cas.sum(0).sum()
            df_prop_list.append(array_prop_i)

        array_res=np.array(df_prop_list)

        # Los valores de los intervalos de probabilidad lower y upper
        inter_prob=np.concatenate((np.apply_along_axis(inter_prob_l, axis=0,␣
↪arr=array_res),np.apply_along_axis(inter_prob_u, axis=0, arr=array_res)))

        # Estimaciones puntuales
        est_puntual=array_res.mean(0)

        # Guardamos las estimaciones puntuales
        est_punt_list.append(est_puntual)

        # Guardamos los intervalos
        inter_list.append(inter_prob)
        #print(i)

    print(df_stra_sam.shape[0])

    # Todas las estimaciones puntuales
    array_est_punt=np.array(est_punt_list)

    array_inter_prob=np.array(inter_list)

    return array_inter_prob,array_est_punt
```

[17]:
```
# Los resultados de la función
porcenta_tama=250
inter_prob, est_punt=bootstrap_simple(df_act_yuc_i, num_boot=1000,␣
↪n_sample=porcenta_tama)
```

250

[18]:
```
# %lprun -f bootstrap_simple bootstrap_simple(df_act_yuc_i, num_boot=50,␣
↪n_sample=porcenta_tama)
```

1.2 Cálculo de métricas referentes al bootstrap

[19]:
```
# Guardamos las bases
df_metr_boot, df_error_max, df_can_cob= metricas_bootstrap(prop_reales,␣
↪inter_prob, est_punt, porcenta_tama=porcenta_tama)
```

[20]:
```
df_can_cob
```

[20]:

	Candidato	Numero_total_cob	Cobertura	Porcenta_tama
0	JOAQUIN_DIAZ_MENA	964	0.964	250
1	RENAN_BARRERA_CONCHA	967	0.967	250
2	VIDA_ARAVARI_GOMEZ_HERRERA	948	0.948	250
3	VOTOS_NULOS_CAND_NO_REGIS	938	0.938	250
4	YAMIL_JASMIN_LOPEZ_MANRIQUE	880	0.880	250

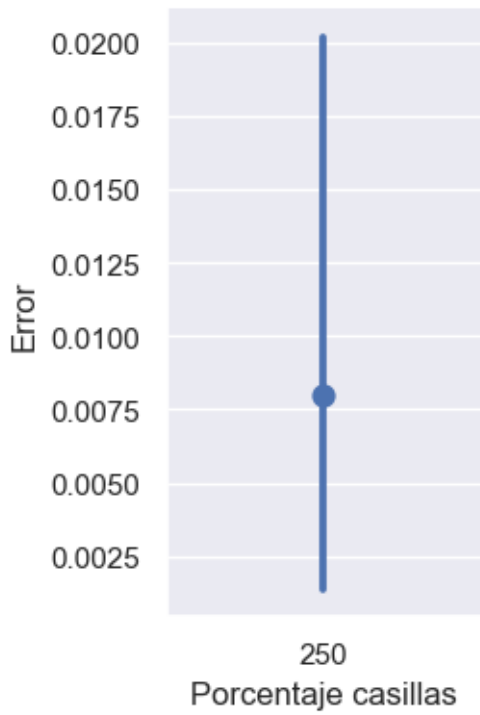
[21]:
```
df_error_max["Error_max"].describe(percentiles=[0.025, 0.25, 0.75, 0.975])
```

[21]:
```
count    1000.000000
mean        0.007987
std         0.004989
min         0.000694
2.5%        0.001379
25%         0.004126
50%         0.006975
75%         0.010842
97.5%       0.020220
max         0.027577
Name: Error_max, dtype: float64
```

[22]:
```
# Candidato con intervalo más largo
df_inter_lar=df_metr_boot.groupby(["Candidato"]).agg({'Longitud_intervalo':
 ↪'mean'}).reset_index()
df_inter_lar["Porcenta_tama"]=porcenta_tama
candidato_mas_largo=df_inter_lar[df_inter_lar["Longitud_intervalo"]==df_inter_lar["Longitud_in
 ↪max()]["Candidato"].values[0]
```

[23]:
```
# Candidato con menor cobertura
candidato_cober_men=df_can_cob[df_can_cob["Cobertura"]==df_can_cob["Cobertura"].
 ↪min()]["Candidato"].values[0]
```

[24]:
```
sns.set_theme();
ax = sns.catplot(
    data=df_error_max, x="Porcenta_tama", y="Error_max",
    kind="point", height=4, aspect=.7, errorbar='pi'
);
ax.set(xlabel='Porcentaje casillas', ylabel='Error');
```

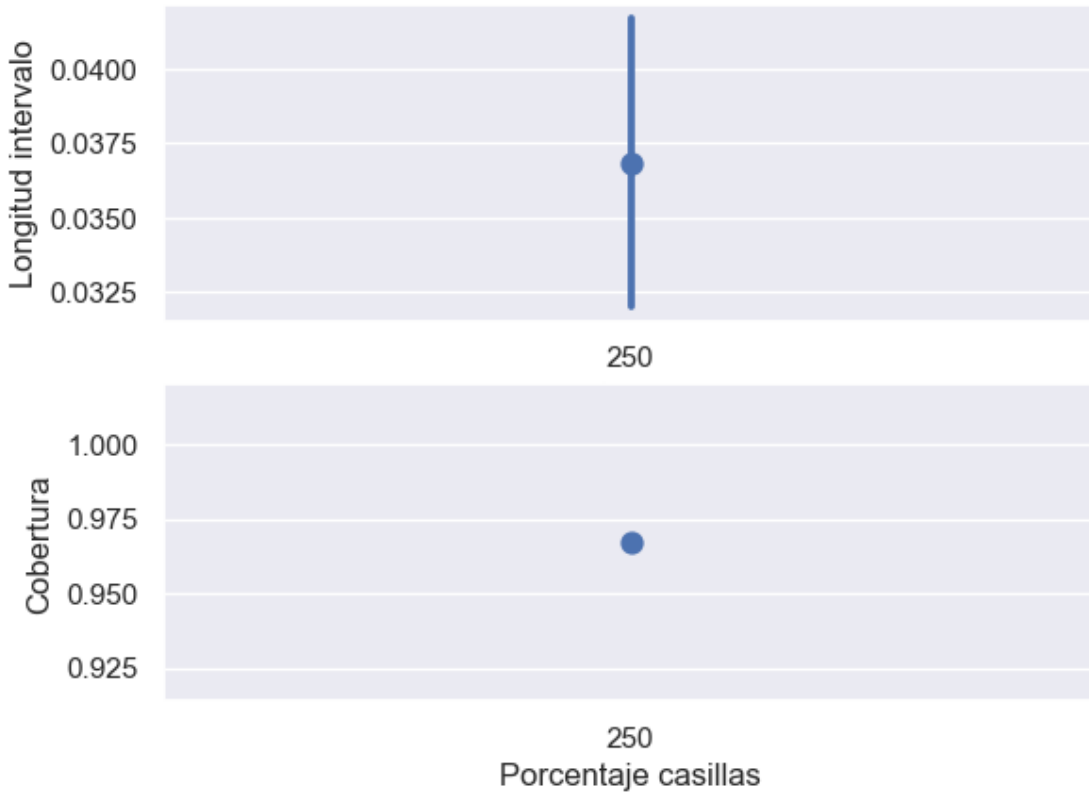


[25]:
```
# Base para poder hacer las gráficas de mínima cobertura e intervalos más largos
df_metr_boot_c=df_metr_boot.drop(columns="Cobertura").
 ↪merge(df_can_cob[["Candidato","Cobertura"]], on=["Candidato"], how='left')
df_metr_boot_c
```

[25]:
```
      Num_bootstrap  cuantil_025  cuantil_975  Longitud_intervalo  \
0                 1     0.495650     0.531646            0.035996
1                 2     0.506798     0.538037            0.031239
2                 3     0.492760     0.530162            0.037402
3                 4     0.521144     0.552082            0.030938
4                 5     0.485649     0.518750            0.033101
…               …           …            …                   …
4995            996     0.019814     0.023120            0.003307
4996            997     0.019832     0.022983            0.003151
4997            998     0.019770     0.022832            0.003063
4998            999     0.020687     0.024206            0.003520
4999           1000     0.020073     0.023007            0.002934

      Estamacion_puntual  Porcentaje_real                  Candidato  \
0               0.513870         0.515125          JOAQUIN_DIAZ_MENA
1               0.521877         0.515125          JOAQUIN_DIAZ_MENA
2               0.512820         0.515125          JOAQUIN_DIAZ_MENA
3               0.536836         0.515125          JOAQUIN_DIAZ_MENA
```

```
4                0.502751         0.515125          JOAQUIN_DIAZ_MENA
…                   …               …                          …
4995             0.021409         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4996             0.021436         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4997             0.021330         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4998             0.022309         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4999             0.021456         0.021467  VOTOS_NULOS_CAND_NO_REGIS

         Error  Porcenta_tama  Cobertura
0     0.001255            250      0.964
1     0.006752            250      0.964
2     0.002305            250      0.964
3     0.021711            250      0.964
4     0.012374            250      0.964
…        …             …         …
4995  0.000058            250      0.938
4996  0.000031            250      0.938
4997  0.000137            250      0.938
4998  0.000842            250      0.938
4999  0.000011            250      0.938

[5000 rows x 10 columns]
```

[26]:
```
facet_kws={'sharey': False, 'sharex': True}
fig, axx =plt.subplots(2,1)
ax_1=sns.pointplot(
    data=df_metr_boot_c[df_metr_boot_c["Candidato"]==candidato_mas_largo],␣
 ↪x="Porcenta_tama", y="Longitud_intervalo", errorbar='pi',ax=axx[0]
);
ax_2=sns.pointplot(
    data=df_metr_boot_c[df_metr_boot_c["Candidato"]==candidato_mas_largo].
 ↪head(1), x="Porcenta_tama", y="Cobertura", ax=axx[1]
)
ax_1.set(xlabel='Porcentaje casillas', ylabel='Longitud intervalo');
ax_2.set(xlabel='Porcentaje casillas', ylabel='Cobertura');
```

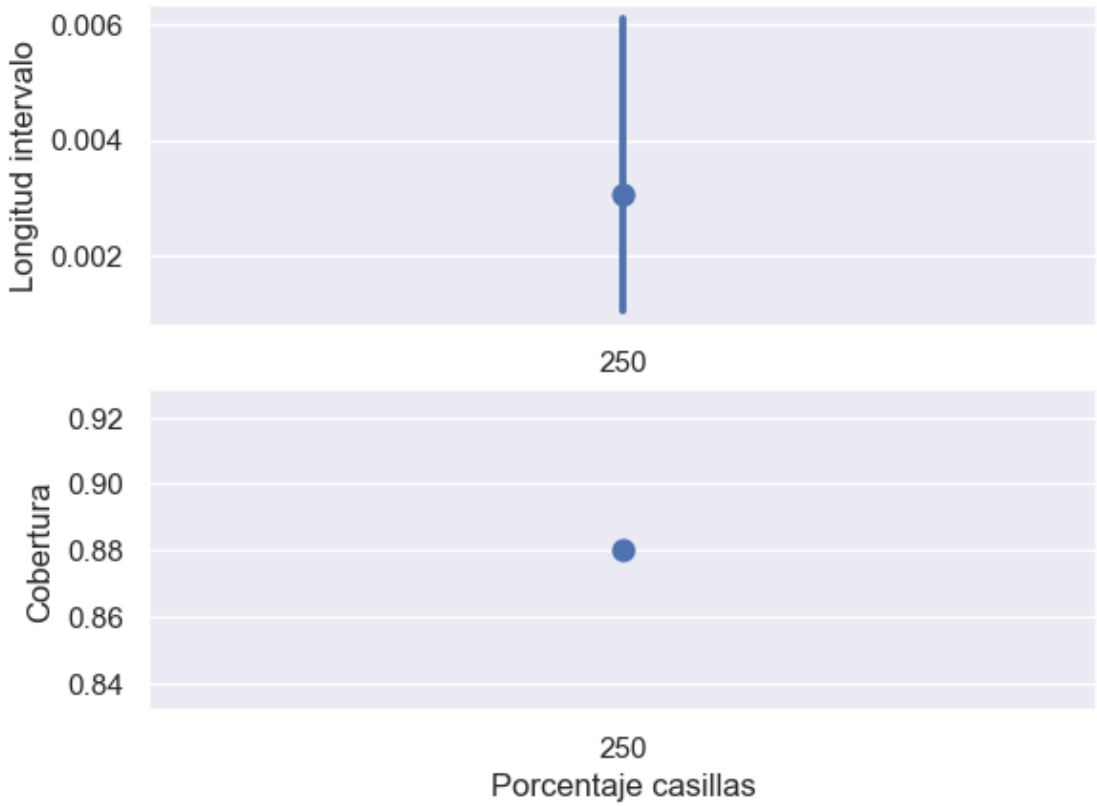


[27]:
```
df_metr_boot_c[df_metr_boot_c["Candidato"]==candidato_mas_largo]
```

[27]:
```
      Num_bootstrap  cuantil_025  cuantil_975  Longitud_intervalo  \
1000              1     0.397408     0.434789            0.037380
1001              2     0.394513     0.426825            0.032312
1002              3     0.408099     0.447328            0.039229
1003              4     0.386466     0.417827            0.031361
1004              5     0.418466     0.453885            0.035419
…               …            …            …                   …
1995            996     0.391066     0.427806            0.036740
1996            997     0.398483     0.440168            0.041684
1997            998     0.404607     0.440035            0.035428
1998            999     0.405233     0.435191            0.029958
1999           1000     0.401791     0.441192            0.039402

      Estamacion_puntual  Porcentaje_real             Candidato     Error  \
1000            0.415813         0.421366  RENAN_BARRERA_CONCHA  0.005553
1001            0.411023         0.421366  RENAN_BARRERA_CONCHA  0.010343
1002            0.425990         0.421366  RENAN_BARRERA_CONCHA  0.004623
1003            0.402002         0.421366  RENAN_BARRERA_CONCHA  0.019364
1004            0.435345         0.421366  RENAN_BARRERA_CONCHA  0.013979
```

```
…                   …               …                    …        …
1995            0.409077         0.421366  RENAN_BARRERA_CONCHA  0.012289
1996            0.418442         0.421366  RENAN_BARRERA_CONCHA  0.002924
1997            0.421735         0.421366  RENAN_BARRERA_CONCHA  0.000369
1998            0.419227         0.421366  RENAN_BARRERA_CONCHA  0.002140
1999            0.421004         0.421366  RENAN_BARRERA_CONCHA  0.000362

      Porcenta_tama  Cobertura
1000            250      0.967
1001            250      0.967
1002            250      0.967
1003            250      0.967
1004            250      0.967
…               …        …
1995            250      0.967
1996            250      0.967
1997            250      0.967
1998            250      0.967
1999            250      0.967

[1000 rows x 10 columns]
```

[28]:
```
facet_kws={'sharey': False, 'sharex': True}
fig, axx =plt.subplots(2,1)
ax_1=sns.pointplot(
    data=df_metr_boot_c[df_metr_boot_c["Candidato"]==candidato_cober_men],␣
 ↪x="Porcenta_tama", y="Longitud_intervalo", errorbar='pi', ax=axx[0]
);
ax_2=sns.pointplot(
    data=df_metr_boot_c[df_metr_boot_c["Candidato"]==candidato_cober_men].
 ↪head(1), x="Porcenta_tama", y="Cobertura", ax=axx[1]
)
ax_1.set(xlabel='Porcentaje casillas', ylabel='Longitud intervalo');
ax_2.set(xlabel='Porcentaje casillas', ylabel='Cobertura');
```



1.3 Bootstrap estratificado (borrador)

[760]:
```
# Número de bootstraps
num_boot=1000

# Semilla para las muestras de la Dirichlet
rng = np.random.default_rng(2)

# Número total de casillas
n_cas=df_act_yuc_i.shape[0]

# Hacemos el muestreo de 10% de las casilas estratificado por distrito
#df_stra_sam=df_act_yuc_i.groupby("DISTRITO", group_keys=False).apply(lambda x:␣
↪x.sample(frac=0.09, random_state=22))[["DISTRITO","ID␣
↪CASILLA","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA","YAMIL_JAS
X_train, df_stra_sam = train_test_split(df_act_yuc_i[["DISTRITO","ID␣
↪CASILLA","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA","YAMIL_JAS
↪stratify=df_act_yuc_i['DISTRITO'], test_size=0.1)
n_boot=1000
```

```
# Guardamos en un numpy array el dataframe de la muestra
array_muest=np.
 ↪array(df_stra_sam[["DISTRITO","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOME

# Los valores distintos de los distritos
val_dist=df_stra_sam["DISTRITO"].sort_values().unique()

# Separamos todos los distritos en distintos arrays
valores_sep_est=[array_muest[array_muest[:,0]==val] for val in val_dist]

# Los porcentajes de cada distrito en la muestra
array_pesos_normalizados_dist=np.array(df_stra_sam["DISTRITO"].
 ↪value_counts(normalize=True).sort_index())

# Cuantas casillas hay por distrito en la muestra
array_frec_est_mue=np.array(df_stra_sam["DISTRITO"].value_counts().sort_index())

# Vector de pesos de la distribución Dirichlet
vectores_dir=[f(xi,n_boot) for xi in array_frec_est_mue]
# Cuantas casillas hay por distrito en toda la base
array_frec_est=np.array(df_act_yuc_i["DISTRITO"].value_counts().sort_index())
# Índices de la base por distritos
array_index_est=np.array(df_stra_sam.reset_index().groupby('DISTRITO')['index'].
 ↪apply(list))

#pesos_bootstrap=np.random.dirichlet(np.ones(df_stra_sam.shape[0],␣
 ↪len(df_stra_sam["DISTRITO"].sort_values().unique())), n_boot)
# Data frame con los resultados
df_prop_list=[]

for j in range(n_boot):
    #print(X_test.shape[0])
    #rng_2 = np.random.default_rng(j)

    # Lista de las casillas muestreadas por el bootstrap
    df_casillas_boot=[]

    #cas_sel_boot=np.random.choice(a=df_stra_sam.index,  size=n_cas-df_stra_sam.
 ↪shape[0],  p=pesos_bootstrap[0], replace=True)

    #cas_sel_boot=np.random.choice(a=df_stra_sam.index,  size=n_cas-df_stra_sam.
 ↪shape[0],  replace=True)

    for k in val_dist:

        #estrato_tam=df_stra_sam[df_stra_sam["DISTRITO"]==k].shape[0]
```

```
        #pesos_bootstrap_est=np.random.dirichlet(np.ones(estrato_tam), n_boot)

        cas_sel_boot=np.random.choice(array_frec_est_mue[k-1], size =␣
↪array_frec_est[k-1]-array_frec_est_mue[k-1], p=vectores_dir[k-1][j])

        df_casillas_boot.append(valores_sep_est[k-1][cas_sel_boot])

    # Juntamos todas las simulaciones

    df_combined_m=np.concatenate((array_muest, np.vstack(df_casillas_boot)))

    # Para poder agrupar en numpy
    # Ordenamos los datos por la primera columna (que es el distrito)
    df_combined_m=np.take(df_combined_m, df_combined_m[:, 0].argsort(), axis=0)
    # Vemos en que valor cambia el indice de (estrato)
    indices = np.nonzero(np.diff(df_combined_m[:, 0]))[0] + 1
    indices = np.concatenate(([0],indices))

    # Porcentaje de los candidatos por distritos
    array_por_est=np.add.reduceat(df_combined_m[:, 1:], indices)/np.array(np.
↪add.reduceat(df_combined_m[:, 1:], indices).sum(1)).reshape(val_dist[-1],1)
    array_tot_agg=np.c_[df_combined_m[:,0][indices],array_por_est]

    # Obtenemos la estimacion puntual
    array_prop_i=(array_tot_agg[:,1:]*array_pesos_normalizados_dist.
↪reshape(val_dist[-1],1)).sum(0)
    df_prop_list.append(array_prop_i)


# Guardamos todo en un array con las estimaciones
array_res=np.array(df_prop_list)

# Los valores de los intervalos de probabilidad lower y upper
inter_prob=np.concatenate((np.apply_along_axis(inter_prob_l, axis=0,␣
↪arr=array_res),np.apply_along_axis(inter_prob_u, axis=0, arr=array_res)))

# Estimaciones puntuales
est_puntual=array_res.mean(0)

# # Guardamos las estimaciones puntuales
# est_punt_list.append(est_puntual)

# # Guardamos los intervalos
# inter_list.append(inter_prob)
```

1.4 Función de bootstrap estratificado

[29]:
```
# Versión función
def boot_estratificado(df_act_yuc_i, num_boot: int=1000, n_sample=None,␣
↪frac=None):

    # Listas auxiliares para guardar los datos
    est_punt_list=[]

    inter_list=[]

    # Cuantas casillas hay por distrito en toda la base
    array_frec_est=np.array(df_act_yuc_i["DISTRITO"].value_counts().
↪sort_index())

    rng_2 = np.random.default_rng()

    for i in range(num_boot):

        # Número de poblaciones
        n_boot=1000

        # Semilla para las muestras de la Dirichlet
        rng = np.random.default_rng(2)

        # Hacemos el muestreo estratificado
        if n_sample==None:
            # Hacemos el muestreo de frac de las casilas estratificado por␣
↪distrito
            X_train, df_stra_sam =␣
↪train_test_split(df_act_yuc_i[["DISTRITO","ID␣
↪CASILLA","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA","YAMIL_JAS
↪stratify=df_act_yuc_i['DISTRITO'], test_size=frac)
        else:
            # Hacemos el muestreo de tamaño n_sample de las casilas␣
↪estratificado por distrito
            df_stra_sam=muestreo_estratificado_mej(df_act_yuc_i, ["DISTRITO"],␣
↪sample_size=n_sample, seed=None)


        # Guardamos en un numpy array el dataframe de la muestra
        array_muest=np.
↪array(df_stra_sam[["DISTRITO","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOME

        # Los valores distintos de los distritos
        val_dist=df_stra_sam["DISTRITO"].sort_values().unique()
```

```
        # Separamos todos los distritos en distintos arrays
        valores_sep_est=[array_muest[array_muest[:,0]==val] for val in val_dist]

        # Los porcentajes de cada distrito en la muestra
        array_pesos_normalizados_dist=np.array(df_stra_sam["DISTRITO"].
↪value_counts(normalize=True).sort_index())

        # Cuantas casillas hay por distrito en la muestra
        array_frec_est_mue=np.array(df_stra_sam["DISTRITO"].value_counts().
↪sort_index())

        # Vector de pesos de la distribución Dirichlet
        vectores_dir=[f(xi,n_boot) for xi in array_frec_est_mue]
        # Cuantas casillas hay por distrito en toda la base
        # array_frec_est=np.array(df_act_yuc_i["DISTRITO"].value_counts().
↪sort_index())

        # Índices de la base por distritos
        #array_index_est=np.array(df_stra_sam.reset_index().
↪groupby('DISTRITO')['index'].apply(list))

        #pesos_bootstrap=np.random.dirichlet(np.ones(df_stra_sam.shape[0],␣
↪len(df_stra_sam["DISTRITO"].sort_values().unique())), n_boot)
        # Data frame con los resultados
        df_prop_list=[]

        for j in range(n_boot):


            # Lista de las casillas muestreadas por el bootstrap
            df_casillas_boot=[]

            # Completamos la muestra mediante el bootstrap bayesiano en cada␣
↪estrato
            for k in val_dist:

                # cas_sel_boot=np.random.choice(array_frec_est_mue[k-1], size =␣
↪array_frec_est[k-1], p=vectores_dir[k-1][j])
                # df_casillas_boot.append(valores_sep_est[k-1][cas_sel_boot])

                # Calculamos una multinomial para completar la población␣
↪(estratificada)
                # Utilizando los pesos bootstrap para cada casilla␣
↪(estratificada)
                # Como resultado nos da las veces que se repitió cada casilla␣
↪en la remuestra
```

```
                cas_sel_boot=rng_2.multinomial(n=array_frec_est[k-1],␣
↪pvals=vectores_dir[k-1][j])

                # Guardamos todas las casillas (estratificadas) del bootstrap␣
↪en un dataframe repitiendo el valor dependiendo
                # de su valor en la multinomial
                df_casillas_boot.append(valores_sep_est[k-1][np.repeat(np.
↪arange(array_frec_est_mue[k-1]),cas_sel_boot)])

            # Juntamos todas las simulaciones
            # Solo utlizamos los datos remuestrados
            df_combined_m=np.vstack(df_casillas_boot)

            # df_combined_m=np.concatenate((array_muest, np.
↪vstack(df_casillas_boot)))

            # Para poder agrupar en numpy
            # Ordenamos los datos por la primera columna (que es el distrito)
            df_combined_m=np.take(df_combined_m, df_combined_m[:, 0].
↪argsort(kind='stable'), axis=0)

            # Vemos en que valor cambia el indice de (estrato)
            indices = np.nonzero(np.diff(df_combined_m[:, 0]))[0] + 1
            indices = np.concatenate(([0],indices))

            # Porcentaje de los candidatos por distritos
            array_por_est=np.add.reduceat(df_combined_m[:, 1:], indices)/np.
↪array(np.add.reduceat(df_combined_m[:, 1:], indices).sum(1)).
↪reshape(val_dist[-1],1)
            array_tot_agg=np.column_stack((df_combined_m[:
↪,0][indices],array_por_est))

            # Obtenemos la estimacion puntual
            array_prop_i=(array_tot_agg[:,1:]*array_pesos_normalizados_dist.
↪reshape(val_dist[-1],1)).sum(0)
            df_prop_list.append(array_prop_i)

        # Guardamos todo en un array con las estimaciones
        array_res=np.array(df_prop_list)

        # Los valores de los intervalos de probabilidad lower y upper
        inter_prob=np.concatenate((np.apply_along_axis(inter_prob_l, axis=0,␣
↪arr=array_res),np.apply_along_axis(inter_prob_u, axis=0, arr=array_res)))

        # Estimaciones puntuales
        est_puntual=array_res.mean(0)
```

```
        # Guardamos las estimaciones puntuales
        est_punt_list.append(est_puntual)

        # Guardamos los intervalos
        inter_list.append(inter_prob)

    print(df_stra_sam.shape[0])

    # Todas las estimaciones puntuales
    array_est_punt=np.array(est_punt_list)

    array_inter_prob=np.array(inter_list)


    return array_inter_prob,array_est_punt
```

[30]:
```
# Tamaño de la muestra
tama_muestra=350
inter_prob_est_e, est_punt_est_e=boot_estratificado(df_act_yuc_i,␣
↪num_boot=1000, n_sample=tama_muestra)
```

350

[31]:
```
# Guardamos las bases
df_metr_boot_est, df_error_max_est, df_can_cob_est=␣
↪metricas_bootstrap(prop_reales, inter_prob_est_e, est_punt_est_e,␣
↪porcenta_tama=350)
```

[32]:
```
df_can_cob_est
```

[32]:

	Candidato	Numero_total_cob	Cobertura	Porcenta_tama
0	JOAQUIN_DIAZ_MENA	958	0.958	350
1	RENAN_BARRERA_CONCHA	940	0.940	350
2	VIDA_ARAVARI_GOMEZ_HERRERA	930	0.930	350
3	VOTOS_NULOS_CAND_NO_REGIS	955	0.955	350
4	YAMIL_JASMIN_LOPEZ_MANRIQUE	916	0.916	350

[33]:
```
# Candidato con intervalo más largo
df_inter_lar_est=df_metr_boot_est.groupby(["Candidato"]).
↪agg({'Longitud_intervalo':'mean'}).reset_index()
df_inter_lar_est["Porcenta_tama"]=tama_muestra
candidato_mas_largo_est=df_inter_lar_est[df_inter_lar_est["Longitud_intervalo"]==df_inter_lar_
↪max()]["Candidato"].values[0]
```

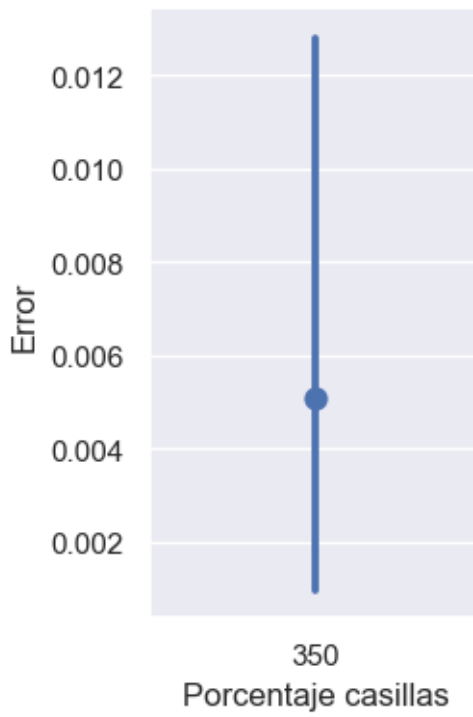
[34]:
```
# Candidato con menor cobertura
candidato_cober_men_est=df_can_cob_est[df_can_cob_est["Cobertura"]==df_can_cob_est["Cobertura"
↪min()]["Candidato"].values[0]
```

[35]:
```
df_error_max_est.describe(percentiles=[0.025,0.975])
```

[35]:

	Num_bootstrap	Error_max	Porcenta_tama
count	1000.000000	1000.000000	1000.0
mean	500.500000	0.005077	350.0
std	288.819436	0.003149	0.0
min	1.000000	0.000459	350.0
2.5%	25.975000	0.000965	350.0
50%	500.500000	0.004366	350.0
97.5%	975.025000	0.012822	350.0
max	1000.000000	0.017620	350.0

[36]:
```
sns.set_theme();
ax = sns.catplot(
    data=df_error_max_est, x="Porcenta_tama", y="Error_max",
    kind="point", height=4, aspect=.7, errorbar='pi'
);
ax.set(xlabel='Porcentaje casillas', ylabel='Error');
```



[37]:
```
# Base para poder hacer las gráficas de mínima cobertura e intervalos más largos
df_metr_boot_c_est=df_metr_boot_est.drop(columns="Cobertura").
 ↪merge(df_can_cob_est[["Candidato","Cobertura"]], on=["Candidato"],␣
 ↪how='left')
```

```
df_metr_boot_c_est
```

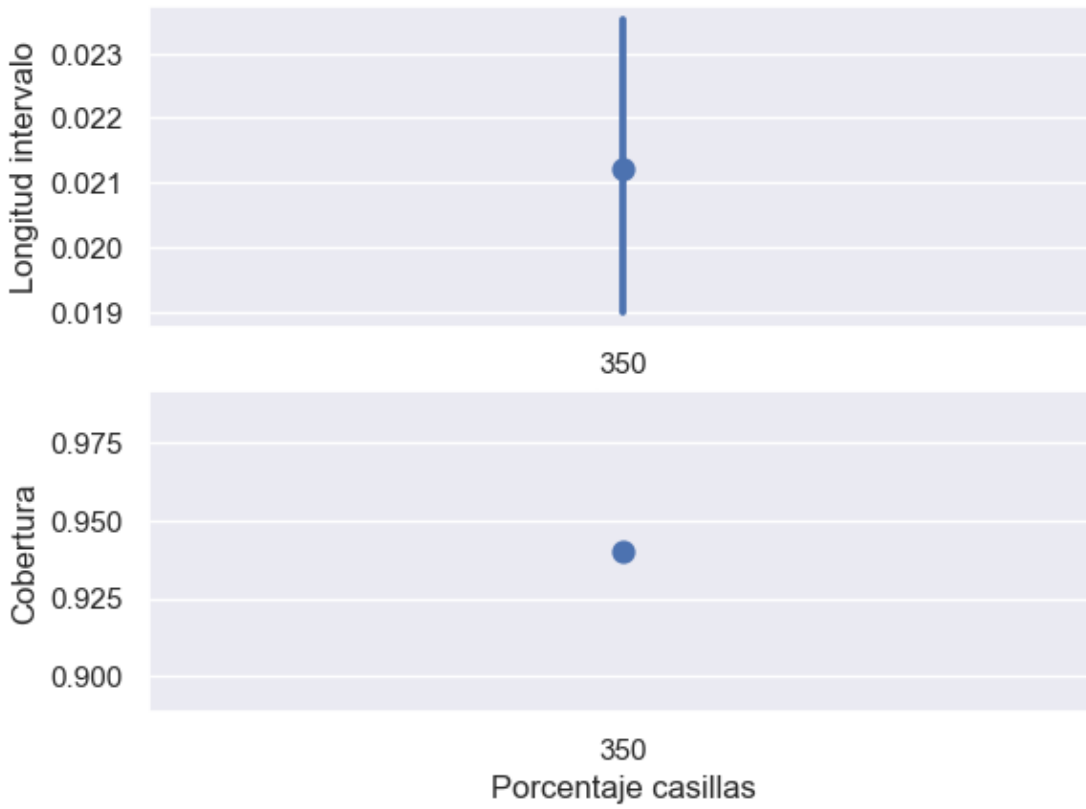
[37]:

```
      Num_bootstrap  cuantil_025  cuantil_975  Longitud_intervalo  \
0                 1     0.504538     0.524355            0.019818
1                 2     0.504789     0.526216            0.021426
2                 3     0.507495     0.529000            0.021504
3                 4     0.518357     0.537707            0.019350
4                 5     0.508045     0.529303            0.021257
…               …           …           …                   …
4995            996     0.020737     0.023032            0.002296
4996            997     0.020174     0.022603            0.002428
4997            998     0.020034     0.022386            0.002352
4998            999     0.020708     0.022917            0.002209
4999           1000     0.020001     0.022103            0.002101

      Estamacion_puntual  Porcentaje_real                  Candidato  \
0               0.514397         0.515125          JOAQUIN_DIAZ_MENA
1               0.514759         0.515125          JOAQUIN_DIAZ_MENA
2               0.518720         0.515125          JOAQUIN_DIAZ_MENA
3               0.528246         0.515125          JOAQUIN_DIAZ_MENA
4               0.518284         0.515125          JOAQUIN_DIAZ_MENA
…                  …              …                          …
4995            0.021887         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4996            0.021357         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4997            0.021203         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4998            0.021824         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4999            0.021034         0.021467  VOTOS_NULOS_CAND_NO_REGIS

         Error  Porcenta_tama  Cobertura
0     0.000729            350      0.958
1     0.000367            350      0.958
2     0.003594            350      0.958
3     0.013121            350      0.958
4     0.003159            350      0.958
…        …             …         …
4995  0.000420            350      0.955
4996  0.000110            350      0.955
4997  0.000264            350      0.955
4998  0.000357            350      0.955
4999  0.000433            350      0.955

[5000 rows x 10 columns]
```

[38]:

```
facet_kws={'sharey': False, 'sharex': True}
fig, axx =plt.subplots(2,1)
ax_1=sns.pointplot(
```

```
    ␣
 ↪data=df_metr_boot_c_est[df_metr_boot_c_est["Candidato"]==candidato_mas_largo_est],␣
 ↪x="Porcenta_tama", y="Longitud_intervalo", errorbar='pi',ax=axx[0]
);
ax_2=sns.pointplot(
    ␣
 ↪data=df_metr_boot_c_est[df_metr_boot_c_est["Candidato"]==candidato_mas_largo_est].
 ↪head(1), x="Porcenta_tama", y="Cobertura", ax=axx[1]
)
ax_1.set(xlabel='Porcentaje casillas', ylabel='Longitud intervalo');
ax_2.set(xlabel='Porcentaje casillas', ylabel='Cobertura');
```



[39]:
```
df_metr_boot_c_est[df_metr_boot_c_est["Candidato"]==candidato_mas_largo_est]
```

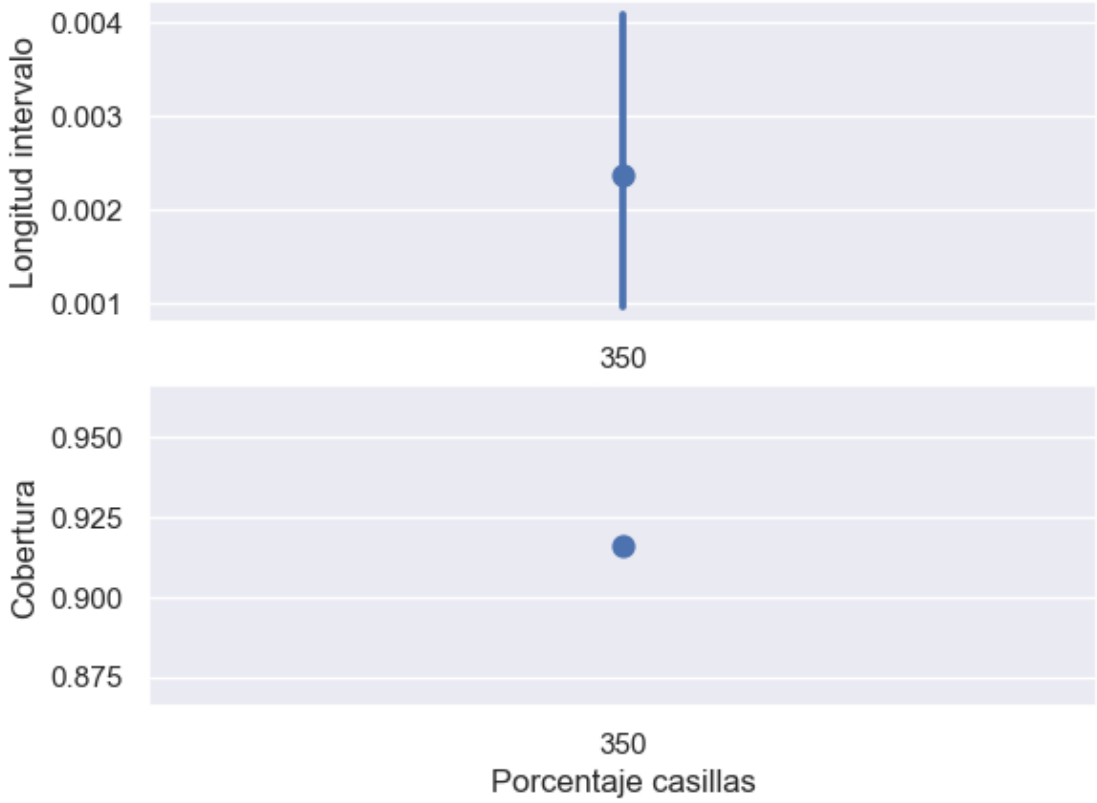
[39]:

	Num_bootstrap	cuantil_025	cuantil_975	Longitud_intervalo	\
1000	1	0.409688	0.431248	0.021560	
1001	2	0.409650	0.432850	0.023200	
1002	3	0.406891	0.428042	0.021152	
1003	4	0.396800	0.417819	0.021018	
1004	5	0.405752	0.427160	0.021408	

```
…              …          …          …                  …
1995           996     0.401370     0.420755           0.019385
1996           997     0.400643     0.421399           0.020756
1997           998     0.397779     0.417225           0.019446
1998           999     0.399986     0.420447           0.020461
1999          1000     0.407500     0.428008           0.020508

      Estamacion_puntual  Porcentaje_real             Candidato     Error  \
1000            0.420482         0.421366  RENAN_BARRERA_CONCHA  0.000884
1001            0.421699         0.421366  RENAN_BARRERA_CONCHA  0.000333
1002            0.417110         0.421366  RENAN_BARRERA_CONCHA  0.004257
1003            0.406870         0.421366  RENAN_BARRERA_CONCHA  0.014496
1004            0.416625         0.421366  RENAN_BARRERA_CONCHA  0.004741
…                  …               …                    …       …
1995            0.410962         0.421366  RENAN_BARRERA_CONCHA  0.010404
1996            0.410664         0.421366  RENAN_BARRERA_CONCHA  0.010702
1997            0.407329         0.421366  RENAN_BARRERA_CONCHA  0.014037
1998            0.410275         0.421366  RENAN_BARRERA_CONCHA  0.011092
1999            0.417876         0.421366  RENAN_BARRERA_CONCHA  0.003490

      Porcenta_tama  Cobertura
1000            350       0.94
1001            350       0.94
1002            350       0.94
1003            350       0.94
1004            350       0.94
…             …         …
1995            350       0.94
1996            350       0.94
1997            350       0.94
1998            350       0.94
1999            350       0.94

[1000 rows x 10 columns]
```

[40]:
```python
facet_kws={'sharey': False, 'sharex': True}
fig, axx =plt.subplots(2,1)
ax_1=sns.pointplot(
    ␣
 ↪data=df_metr_boot_c_est[df_metr_boot_c_est["Candidato"]==candidato_cober_men_est],␣
 ↪x="Porcenta_tama", y="Longitud_intervalo", errorbar='pi', ax=axx[0]
);
ax_2=sns.pointplot(
    ␣
 ↪data=df_metr_boot_c_est[df_metr_boot_c_est["Candidato"]==candidato_cober_men_est].
 ↪head(1), x="Porcenta_tama", y="Cobertura", ax=axx[1]
)
```

```
ax_1.set(xlabel='Porcentaje casillas', ylabel='Longitud intervalo');
ax_2.set(xlabel='Porcentaje casillas', ylabel='Cobertura');
```



[41]:
```
# %lprun -f boot_estratificado boot_estratificado(df_act_yuc_i, num_boot=20,␣
↪n_sample=250)
```

2 Bootstrap por votos (en las casillas)

2.1 Boostrap por votos (borrador)

[21]:
```
# Número de votos seleccionados (submuestra)
n_vot_sub=2000

# Número total de votos
n_votos=int(df_act_yuc_i[["JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRE
↪sum().sum())

df_stra_sam = df_act_yuc_i.sample(n=250, frac=None)
```

```
df_stra_sam=np.
 ↪array(df_stra_sam[["JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA",

# Votos por candidato en la muestra
n_votos_cand=df_stra_sam.sum(0).astype(int)

rng_2 = np.random.default_rng(j)

# Submuestra de votos
votos_subm=rng_2.multinomial(n=n_vot_sub,  pvals=n_votos_cand/n_votos_cand.
 ↪sum())

# Los pesos de la distribución Dirichlet
pesos_bootstrap=np.random.dirichlet(votos_subm, 1000)

# Data frame con los resultados
df_prop_list=[]

for j in range(1000):
    #print(X_test.shape[0])
    #rng_2 = np.random.default_rng(j)

    # Guardamos la semilla para que el código se reproducible
    rng_2 = np.random.default_rng()
    # La casilla seleccionada por el bootstrap

    # Para completar toda la población
    #cas_sel_boot=rng_2.choice(a=pesos_bootstrap.shape[1], ␣
 ↪size=n_votos-int(df_stra_sam.sum()),  p=pesos_bootstrap[j])

    # Total de votos de la muestra simulada (utilizando una multinomial)
    votos_sim=rng_2.multinomial(n=n_votos,  pvals=pesos_bootstrap[j])

    # La estimación puntual
    array_prop_i=(votos_sim)/(votos_sim).sum()

    df_prop_list.append(array_prop_i)

array_res=np.array(df_prop_list)

# Los valores de los intervalos de probabilidad lower y upper
inter_prob=np.concatenate((np.apply_along_axis(inter_prob_l, axis=0,␣
 ↪arr=array_res),np.apply_along_axis(inter_prob_u, axis=0, arr=array_res)))

# Estimaciones puntuales
est_puntual=array_res.mean(0)
```

[22]:
```
inter_prob=np.concatenate((np.apply_along_axis(inter_prob_l, axis=0,␣
↪arr=array_res),np.apply_along_axis(inter_prob_u, axis=0, arr=array_res)))
```

[23]:
```
est_puntual
```

[23]:
```
array([0.53246574, 0.40542693, 0.03895929, 0.00608284, 0.01706519])
```

2.2 Bootstrap por votos (sin estratificar)

[42]:
```
def bootstrap_votos_simple(df, num_boot: int=1000, n_sample=None, frac=None,␣
↪n_vot_sub=2000):

    est_punt_list=[]

    inter_list=[]

    # Número total de votos
    ␣
↪n_votos=int(df[["JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA","YA
↪sum().sum())

    # Semilla para las muestras de la Dirichlet
    rng = np.random.default_rng()

    # Semilla para los conteos de votos (eventualmente se tiene que mover␣
↪adentro del for para que sea reproducible el código)
    rng_2 = np.random.default_rng()

    for i in range(num_boot):

        df_stra_sam = df_act_yuc_i.sample(n=n_sample, frac=frac)

        df_stra_sam=np.
↪array(df_stra_sam[["JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA",

        # Votos por candidato en la muestra
        n_votos_cand=df_stra_sam.sum(0).astype(int)

        # Submuestra de votos (en este caso son 2000)
        votos_subm=rng.multinomial(n=n_vot_sub,  pvals=n_votos_cand/
↪n_votos_cand.sum())

        # Los pesos de la distribución Dirichlet
        pesos_bootstrap=rng.dirichlet(votos_subm, 1000)

        # Data frame con los resultados
        df_prop_list=[]
```

```
        for j in range(1000):

            # Guardamos la semilla para que el código se reproducible
            #rng_2 = np.random.default_rng()
            # La casilla seleccionada por el bootstrap

            # Total de votos de la muestra simulada (utilizando una multinomial)
            votos_sim=rng_2.multinomial(n=n_votos,  pvals=pesos_bootstrap[j])
            array_prop_i=(votos_sim)/(votos_sim).sum()

            df_prop_list.append(array_prop_i)

        array_res=np.array(df_prop_list)

        # Los valores de los intervalos de probabilidad lower y upper
        inter_prob=np.concatenate((np.apply_along_axis(inter_prob_l, axis=0,␣
↪arr=array_res),np.apply_along_axis(inter_prob_u, axis=0, arr=array_res)))

        # Estimaciones puntuales
        est_puntual=array_res.mean(0)

        # Guardamos las estimaciones puntuales
        est_punt_list.append(est_puntual)

        # Guardamos los intervalos
        inter_list.append(inter_prob)
        #print(i)

    print(df_stra_sam.shape[0])

    # Todas las estimaciones puntuales
    array_est_punt=np.array(est_punt_list)

    array_inter_prob=np.array(inter_list)

    return array_inter_prob,array_est_punt
```

[43]:
```
# Boostrap por votos
inter_prob_v, est_punt_v=bootstrap_votos_simple(df_act_yuc_i, num_boot=1000,␣
↪n_sample=2000, n_vot_sub=500)
```

2000

[44]:
```
# Guardamos las bases
df_metr_boot_v, df_error_max_v, df_can_cob_v = metricas_bootstrap(prop_reales,␣
↪inter_prob_v, est_punt_v, porcenta_tama=250)
```

[45]:
```
df_can_cob_v
```

[45]:

	Candidato	Numero_total_cob	Cobertura	Porcenta_tama
0	JOAQUIN_DIAZ_MENA	946	0.946	250
1	RENAN_BARRERA_CONCHA	940	0.940	250
2	VIDA_ARAVARI_GOMEZ_HERRERA	932	0.932	250
3	VOTOS_NULOS_CAND_NO_REGIS	922	0.922	250
4	YAMIL_JASMIN_LOPEZ_MANRIQUE	918	0.918	250

[46]:
```
# Base para poder hacer las gráficas de mínima cobertura e intervalos más largos
df_metr_boot_c_v=df_metr_boot_v.drop(columns="Cobertura").
 ↪merge(df_can_cob_v[["Candidato","Cobertura"]], on=["Candidato"], how='left')
df_metr_boot_c_v
```

[46]:

	Num_bootstrap	cuantil_025	cuantil_975	Longitud_intervalo	\
0	1	0.454683	0.542702	0.088019	
1	2	0.462941	0.551474	0.088533	
2	3	0.513000	0.598913	0.085913	
3	4	0.501991	0.585687	0.083696	
4	5	0.457490	0.547336	0.089847	
…	…	…	…	…	
4995	996	0.001989	0.017690	0.015702	
4996	997	0.008419	0.030325	0.021907	
4997	998	0.013803	0.041952	0.028150	
4998	999	0.014648	0.040564	0.025916	
4999	1000	0.007990	0.030957	0.022968	

	Estamacion_puntual	Porcentaje_real	Candidato	\
0	0.499993	0.515125	JOAQUIN_DIAZ_MENA	
1	0.507580	0.515125	JOAQUIN_DIAZ_MENA	
2	0.555389	0.515125	JOAQUIN_DIAZ_MENA	
3	0.544281	0.515125	JOAQUIN_DIAZ_MENA	
4	0.502875	0.515125	JOAQUIN_DIAZ_MENA	
…	…	…	…	
4995	0.007996	0.021467	VOTOS_NULOS_CAND_NO_REGIS	
4996	0.017759	0.021467	VOTOS_NULOS_CAND_NO_REGIS	
4997	0.026249	0.021467	VOTOS_NULOS_CAND_NO_REGIS	
4998	0.025880	0.021467	VOTOS_NULOS_CAND_NO_REGIS	
4999	0.018198	0.021467	VOTOS_NULOS_CAND_NO_REGIS	

	Error	Porcenta_tama	Cobertura
0	0.015132	250	0.946
1	0.007546	250	0.946
2	0.040264	250	0.946
3	0.029156	250	0.946
4	0.012251	250	0.946
…	…	…	…

```
4995  0.013471            250      0.922
4996  0.003708            250      0.922
4997  0.004782            250      0.922
4998  0.004413            250      0.922
4999  0.003269            250      0.922

[5000 rows x 10 columns]
```

[47]:
```
# Candidato con intervalo más largo
df_inter_lar_v=df_metr_boot_v.groupby(["Candidato"]).agg({'Longitud_intervalo':
 ↪'mean'}).reset_index()
df_inter_lar_v["Porcenta_tama"]=250
candidato_mas_largo_v=df_inter_lar_v[df_inter_lar_v["Longitud_intervalo"]==df_inter_lar_v["Lon
 ↪max()]["Candidato"].values[0]
df_metr_boot_c_v[df_metr_boot_c_v["Candidato"]==candidato_mas_largo_v]["Longitud_intervalo"].
 ↪describe()
```

[47]:
```
count    1000.000000
mean        0.087000
std         0.002525
min         0.078624
25%         0.085280
50%         0.087033
75%         0.088739
max         0.095053
Name: Longitud_intervalo, dtype: float64
```

[48]:
```
# Candidato con intervalo más largo
df_inter_lar_v=df_metr_boot_v.groupby(["Candidato"]).agg({'Longitud_intervalo':
 ↪'mean'}).reset_index()
df_inter_lar_v["Porcenta_tama"]=250
candidato_mas_largo_v=df_inter_lar_v[df_inter_lar_v["Longitud_intervalo"]==df_inter_lar_v["Lon
 ↪max()]["Candidato"].values[0]
```

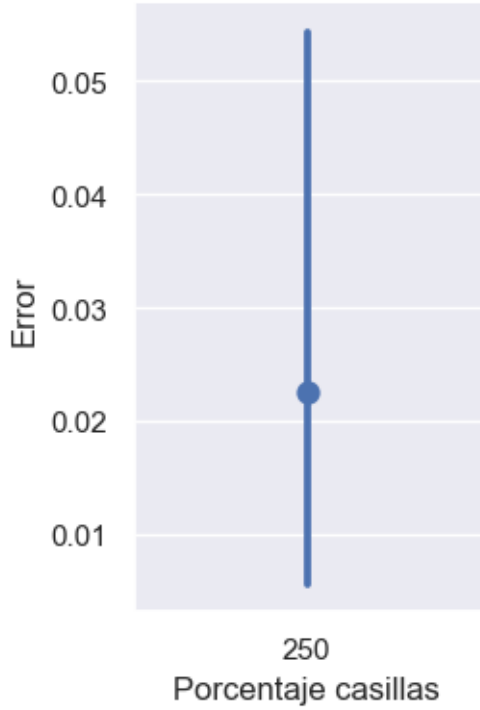
[49]:
```
# Candidato con menor cobertura
candidato_cober_men_v=df_can_cob_v[df_can_cob_v["Cobertura"]==df_can_cob_v["Cobertura"].
 ↪min()]["Candidato"].values[0]
```

[50]:
```
df_error_max_v.describe(percentiles=[0.025,0.975])
```

[50]:
```
       Num_bootstrap    Error_max  Porcenta_tama
count    1000.000000  1000.000000         1000.0
mean      500.500000     0.022580          250.0
std       288.819436     0.013207            0.0
min         1.000000     0.001336          250.0
2.5%       25.975000     0.005612          250.0
50%       500.500000     0.019438          250.0
```

```
97.5%     975.025000     0.054397          250.0
max      1000.000000     0.081494          250.0
```

[51]:
```
sns.set_theme();
ax = sns.catplot(
    data=df_error_max_v, x="Porcenta_tama", y="Error_max",
    kind="point", height=4, aspect=.7, errorbar='pi'
);
ax.set(xlabel='Porcentaje casillas', ylabel='Error');
```

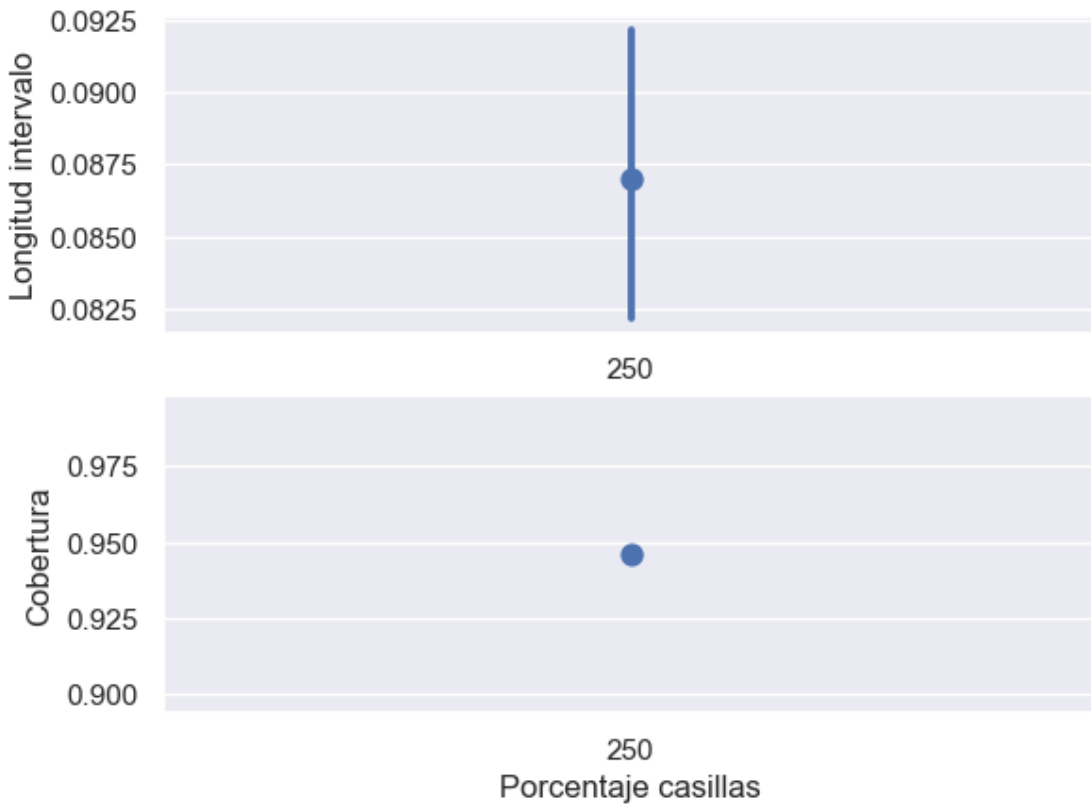


[52]:
```
# Base para poder hacer las gráficas de mínima cobertura e intervalos más largos
df_metr_boot_c_v=df_metr_boot_v.drop(columns="Cobertura").
 ↪merge(df_can_cob_v[["Candidato","Cobertura"]], on=["Candidato"], how='left')
df_metr_boot_c_v
```

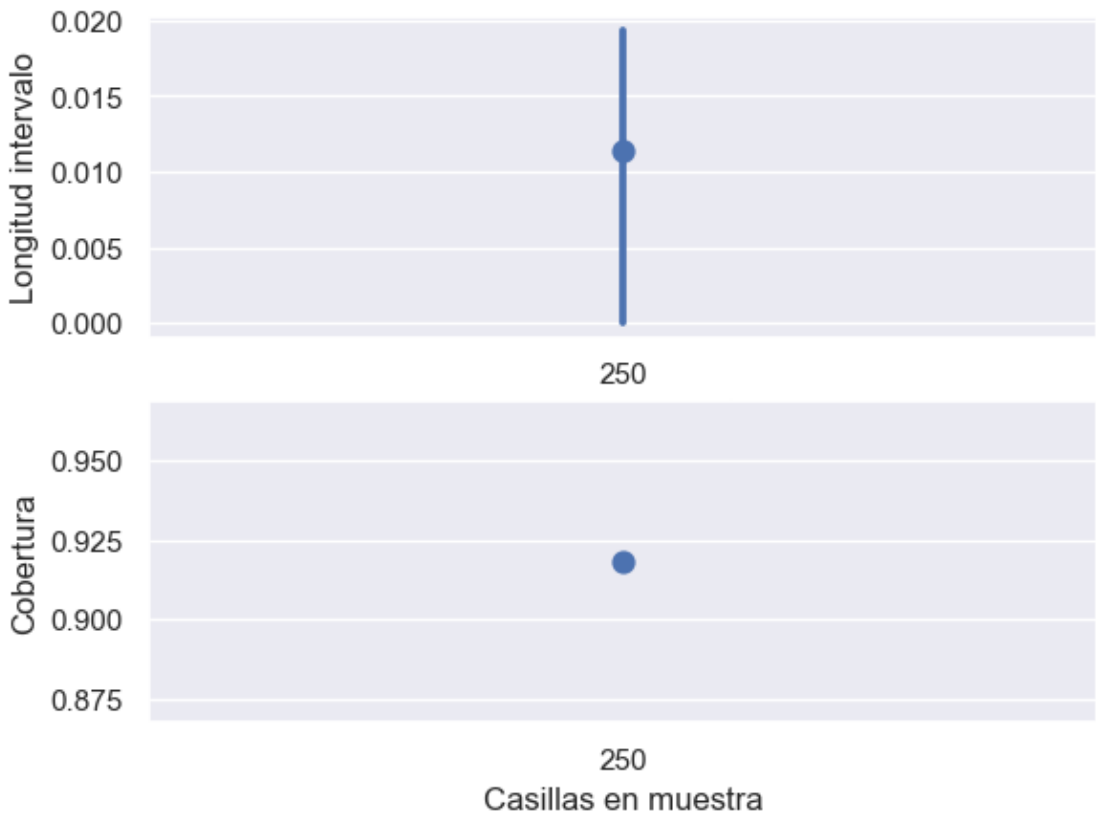
[52]:
```
      Num_bootstrap  cuantil_025  cuantil_975  Longitud_intervalo  \
0                 1     0.454683     0.542702            0.088019
1                 2     0.462941     0.551474            0.088533
2                 3     0.513000     0.598913            0.085913
3                 4     0.501991     0.585687            0.083696
4                 5     0.457490     0.547336            0.089847
…               …           …            …                   …
4995            996     0.001989     0.017690            0.015702
```

```
4996            997     0.008419     0.030325            0.021907
4997            998     0.013803     0.041952            0.028150
4998            999     0.014648     0.040564            0.025916
4999           1000     0.007990     0.030957            0.022968

      Estamacion_puntual  Porcentaje_real                  Candidato  \
0               0.499993         0.515125          JOAQUIN_DIAZ_MENA
1               0.507580         0.515125          JOAQUIN_DIAZ_MENA
2               0.555389         0.515125          JOAQUIN_DIAZ_MENA
3               0.544281         0.515125          JOAQUIN_DIAZ_MENA
4               0.502875         0.515125          JOAQUIN_DIAZ_MENA
…                  …              …                          …
4995            0.007996         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4996            0.017759         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4997            0.026249         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4998            0.025880         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4999            0.018198         0.021467  VOTOS_NULOS_CAND_NO_REGIS

         Error  Porcenta_tama  Cobertura
0     0.015132            250      0.946
1     0.007546            250      0.946
2     0.040264            250      0.946
3     0.029156            250      0.946
4     0.012251            250      0.946
…        …            …        …
4995  0.013471            250      0.922
4996  0.003708            250      0.922
4997  0.004782            250      0.922
4998  0.004413            250      0.922
4999  0.003269            250      0.922

[5000 rows x 10 columns]
```

[53]:
```
facet_kws={'sharey': False, 'sharex': True}
fig, axx =plt.subplots(2,1)
ax_1=sns.pointplot(
    ␣
 ↪data=df_metr_boot_c_v[df_metr_boot_c_v["Candidato"]==candidato_mas_largo_v],␣
 ↪x="Porcenta_tama", y="Longitud_intervalo", errorbar='pi',ax=axx[0]
);
ax_2=sns.pointplot(
    data=df_metr_boot_c_v[df_metr_boot_c_v["Candidato"]==candidato_mas_largo_v].
 ↪head(1), x="Porcenta_tama", y="Cobertura", ax=axx[1]
)
ax_1.set(xlabel='Porcentaje casillas', ylabel='Longitud intervalo');
ax_2.set(xlabel='Porcentaje casillas', ylabel='Cobertura');
```



[54]:
```
facet_kws={'sharey': False, 'sharex': True}
fig, axx =plt.subplots(2,1)
ax_1=sns.pointplot(
    ␣
 ↪data=df_metr_boot_c_v[df_metr_boot_c_v["Candidato"]==candidato_cober_men_v],␣
 ↪x="Porcenta_tama", y="Longitud_intervalo", errorbar='pi', ax=axx[0]
);
ax_2=sns.pointplot(
    data=df_metr_boot_c_v[df_metr_boot_c_v["Candidato"]==candidato_cober_men_v].
 ↪head(1), x="Porcenta_tama", y="Cobertura", ax=axx[1]
)
ax_1.set(xlabel='Casillas en muestra', ylabel='Longitud intervalo');
ax_2.set(xlabel='Casillas en muestra', ylabel='Cobertura');
```



[55]:
```
# %load_ext line_profiler
# %lprun -f bootstrap_votos_simple bootstrap_votos_simple(df_act_yuc_i,␣
↪num_boot=10, n_sample=250)
```

2.3 Boostrap por votos estratificado (borrador)

[37]:
```
n_sample=250

num_boot=10

# Listas auxiliares para guardar los datos
est_punt_list=[]

inter_list=[]

# Cuantas votos hay por distrito
array_frec_est=np.array(np.array(df_act_yuc_i.groupby('DISTRITO').agg({'TOTAL':
↪'sum'}).reset_index(drop=True).astype(int)["TOTAL"]))

# La submuestra de votos
```

```
n_vot_sub=2000

# Número de poblaciones
n_boot=1000

# Tamaño de la población (la muestra)
t_p=2000

# Semilla para las muestras de la Dirichlet
rng = np.random.default_rng()
rng_2 = np.random.default_rng()

# Hacemos el muestreo estratificado
if n_sample==None:
    # Hacemos el muestreo de frac de las casilas estratificado por distrito
    X_train, df_stra_sam = train_test_split(df_act_yuc_i[["DISTRITO","ID␣
 ↪CASILLA","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA","YAMIL_JAS
 ↪stratify=df_act_yuc_i['DISTRITO'], test_size=0.1)
else:
    # Hacemos el muestreo de tamaño n_sample de las casilas estratificado por␣
 ↪distrito
    df_stra_sam=muestreo_estratificado_mej(df_act_yuc_i, ["DISTRITO"],␣
 ↪sample_size=n_sample, seed=None)

# Guardamos en un numpy array el dataframe de la muestra
array_muest=np.
 ↪array(df_stra_sam[["DISTRITO","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOME

# Hacemos la suma agrupando por distritos
#valores_votos_dist=[np.insert(array_muest[array_muest[:,0]==val][:,1:].sum(0).
 ↪astype(int), 0, val) for val in val_dist]

# Los valores distintos de los distritos
val_dist=df_stra_sam["DISTRITO"].sort_values().unique()

# Sirve para calcular el número de votos por estrato (distrito) en la muestra␣
 ↪(tal vez sirva para después)
#valores_votos_dist=[array_muest[array_muest[:,0]==val][:,1:].sum(0).sum(0).
 ↪astype(int) for val in val_dist]

# Separamos todos los distritos en distintos arrays
valores_sep_est=[array_muest[array_muest[:,0]==val] for val in val_dist]

# Los porcentajes de cada distrito en la muestra
array_pesos_normalizados_dist=np.array(df_stra_sam["DISTRITO"].
 ↪value_counts(normalize=True).sort_index())
```

```
# Obtenemos los totales de las proporciones para cada estrato (distrito)
totales_prop_est=obtener_totales(array_pesos_normalizados_dist, n_vot_sub)

# Submuestra de votos por distrito (estrato)
votos_subm_d=[rng.multinomial(n=totales_prop_est[val-1], ␣
 ↪pvals=array_muest[array_muest[:,0]==val][:,1:].sum(0)/
 ↪array_muest[array_muest[:,0]==val][:,1:].sum()) for val in val_dist]


# Vector de pesos de la distribución Dirichlet

# Hay que arreglar la función para que pueda aceptar valores con alfa igual a␣
 ↪cero (que tengan probabilidad 0)
vectores_dir=[f_a(xi,n_boot) for xi in votos_subm_d]

# Calculamos el tamaño del vector de candidatos
tam_vect=array_muest.shape[1]-1

# Índices de la base por distritos
#array_index_est=np.array(df_stra_sam.reset_index().
 ↪groupby('DISTRITO')['index'].apply(list))

#pesos_bootstrap=np.random.dirichlet(np.ones(df_stra_sam.shape[0],␣
 ↪len(df_stra_sam["DISTRITO"].sort_values().unique())), n_boot)
# Data frame con los resultados
df_prop_list=[]


for j in range(n_boot):

    #rng_2 = np.random.default_rng()

    # Lista de las casillas muestreadas por el bootstrap
    df_casillas_boot=[]

    # Completamos la muestra mediante el bootstrap bayesiano en cada estrato
    for k in val_dist:

        # Total de votos de la muestra simulada (utilizando una multinomial)
        votos_sim=rng_2.multinomial(n=array_frec_est[k-1], ␣
 ↪pvals=vectores_dir[k-1][j])
        array_prop_i=(votos_sim)/(votos_sim).sum()

        # Calculamos los porcentajes ponderados con su respectiva proporción␣
 ↪por distrito
```

```
        porcentaje_dist_v=(array_prop_i)*(array_pesos_normalizados_dist[k-1])
        df_casillas_boot.append(porcentaje_dist_v)

    # # Juntamos todas las simulaciones
    df_combined_m=np.vstack(df_casillas_boot)

    # Obtenemos la estimacion puntual por población
    estim_pun=df_combined_m.sum(0)
    df_prop_list.append(estim_pun)

    # Guardamos todo en un array con las estimaciones
    array_res=np.array(df_prop_list)

# Los valores de los intervalos de probabilidad lower y upper
inter_prob=np.concatenate((np.apply_along_axis(inter_prob_l, axis=0,␣
 ↪arr=array_res),np.apply_along_axis(inter_prob_u, axis=0, arr=array_res)))

# Estimaciones puntuales (del bootstrap)
est_puntual=array_res.mean(0)
```

2.4 Bootstrap por votos estratificado

[56]:
```
# Versión función
def boot_votos_estratificado(df_act_yuc_i, num_boot: int=1000, n_sample=None,␣
 ↪frac=None, n_vot_sub=2000):

    # Listas auxiliares para guardar los datos
    est_punt_list=[]

    inter_list=[]

    # Cuantas votos hay por distrito
    array_frec_est=np.array(np.array(df_act_yuc_i.groupby('DISTRITO').
 ↪agg({'TOTAL':'sum'}).reset_index(drop=True).astype(int)["TOTAL"]))

    # Número de poblaciones
    n_boot=1000

    # Semilla para las muestras de la Dirichlet
    rng = np.random.default_rng()

    # Semilla para las muestras de la Dirichlet
    rng_2 = np.random.default_rng()

    for i in range(num_boot):

        # Hacemos el muestreo estratificado
```

```
        if n_sample==None:
            # Hacemos el muestreo de frac de las casilas estratificado por␣
↪distrito
            X_train, df_stra_sam =␣
↪train_test_split(df_act_yuc_i[["DISTRITO","ID␣
↪CASILLA","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOMEZ_HERRERA","YAMIL_JAS
↪stratify=df_act_yuc_i['DISTRITO'], test_size=frac)
        else:
            # Hacemos el muestreo de tamaño n_sample de las casilas␣
↪estratificado por distrito
            df_stra_sam=muestreo_estratificado_mej(df_act_yuc_i, ["DISTRITO"],␣
↪sample_size=n_sample, seed=None)

        # Guardamos en un numpy array el dataframe de la muestra
        array_muest=np.
↪array(df_stra_sam[["DISTRITO","JOAQUIN_DIAZ_MENA","RENAN_BARRERA_CONCHA","VIDA_ARAVARI_GOME

        # Los valores distintos de los distritos
        val_dist=df_stra_sam["DISTRITO"].sort_values().unique()

        # Sirve para calcular el número de votos por estrato (distrito) en la␣
↪muestra (tal vez sirva para después)
        #valores_votos_dist=[array_muest[array_muest[:,0]==val][:,1:].sum(0).
↪sum(0).astype(int) for val in val_dist]

        # Los porcentajes de cada distrito en la muestra
        array_pesos_normalizados_dist=np.array(df_stra_sam["DISTRITO"].
↪value_counts(normalize=True).sort_index())

        # Obtenemos los totales de las proporciones para cada estrato (distrito)
        totales_prop_est=obtener_totales(array_pesos_normalizados_dist,␣
↪n_vot_sub)

        # Submuestra de votos por distrito (estrato)
        votos_subm_d=[rng.multinomial(n=totales_prop_est[val-1], ␣
↪pvals=array_muest[array_muest[:,0]==val][:,1:].sum(0)/
↪array_muest[array_muest[:,0]==val][:,1:].sum()) for val in val_dist]

        # Vector de pesos de la distribución Dirichlet

        # Hay que arreglar la función para que pueda aceptar valores con alfa␣
↪igual a cero (que tengan probabilidad 0)
        vectores_dir=[f_a(xi,n_boot) for xi in votos_subm_d]

        # Data frame con los resultados
        df_prop_list=[]
```

```
        for j in range(n_boot):

            # Lista de las casillas muestreadas por el bootstrap
            df_casillas_boot=[]

            # Completamos la muestra mediante el bootstrap bayesiano en cada␣
↪estrato
            for k in val_dist:

                # Total de votos de la muestra simulada (utilizando una␣
↪multinomial)
                votos_sim=rng_2.multinomial(n=array_frec_est[k-1], ␣
↪pvals=vectores_dir[k-1][j])
                # Para calcular el porcentaje de votos
                # (votos_sim)/(votos_sim).sum()
                # Calculamos los porcentajes ponderados con su respectiva␣
↪proporción por distrito
                # ((votos_sim)/(votos_sim).
↪sum())*(array_pesos_normalizados_dist[k-1])
                # y los vamos guardando
                df_casillas_boot.append(((votos_sim)/(votos_sim).
↪sum())*(array_pesos_normalizados_dist[k-1]))

            # Juntamos todas las simulaciones
            # np.vstack(df_casillas_boot)
            # y obtenemos la estimacion puntual por población
            estim_pun=np.vstack(df_casillas_boot).sum(0)
            df_prop_list.append(estim_pun)

        # Guardamos todo en un array con las estimaciones
        array_res=np.array(df_prop_list)

        # Los valores de los intervalos de probabilidad lower y upper
        inter_prob=np.concatenate((np.apply_along_axis(inter_prob_l, axis=0,␣
↪arr=array_res),np.apply_along_axis(inter_prob_u, axis=0, arr=array_res)))

        # Estimaciones puntuales (del bootstrap)
        est_puntual=array_res.mean(0)

        # Guardamos las estimaciones puntuales
        est_punt_list.append(est_puntual)

        # Guardamos los intervalos
        inter_list.append(inter_prob)
```

```
    print(df_stra_sam.shape[0])

    # Todas las estimaciones puntuales
    array_est_punt=np.array(est_punt_list)

    array_inter_prob=np.array(inter_list)

    return array_inter_prob,array_est_punt
```

[57]:
```
inter_prob_v_est, est_punt_v_est=boot_votos_estratificado(df_act_yuc_i,␣
↪num_boot=1000, n_sample=250, n_vot_sub=2000)
```

250

[58]:
```
# Cálculamos las métricas
df_metr_boot_v_est, df_error_max_v_est, df_can_cob_v_est =␣
↪metricas_bootstrap(prop_reales, inter_prob_v_est, est_punt_v_est,␣
↪porcenta_tama=250)
```

[59]:
```
# Candidato con intervalo más largo
df_inter_lar_v_est=df_metr_boot_v_est.groupby(["Candidato"]).
↪agg({'Longitud_intervalo':'mean'}).reset_index()
df_inter_lar_v_est["Porcenta_tama"]=250
candidato_mas_largo_v_est=df_inter_lar_v_est[df_inter_lar_v_est["Longitud_intervalo"]==df_inte
↪max()]["Candidato"].values[0]
```

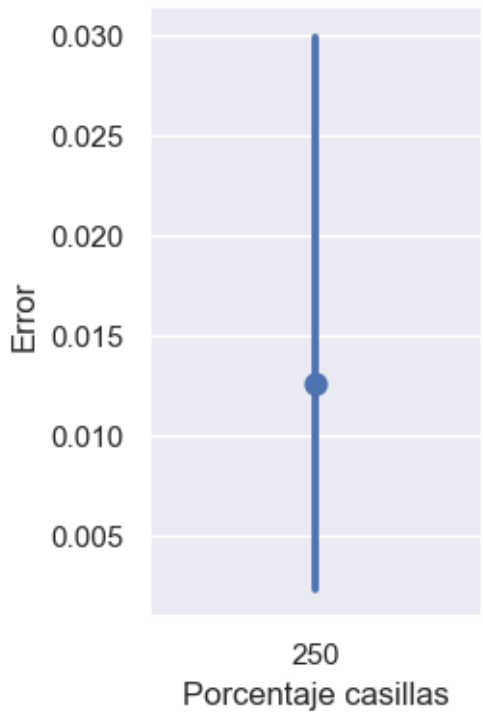
[60]:
```
# Candidato con menor cobertura
candidato_cober_men_v_est=df_can_cob_v_est[df_can_cob_v_est["Cobertura"]==df_can_cob_v_est["Co
↪min()]["Candidato"].values[0]
```

[61]:
```
df_error_max_v_est.describe(percentiles=[0.025,0.975])
```

[61]:

	Num_bootstrap	Error_max	Porcenta_tama
count	1000.000000	1000.000000	1000.0
mean	500.500000	0.012572	250.0
std	288.819436	0.007362	0.0
min	1.000000	0.000791	250.0
2.5%	25.975000	0.002301	250.0
50%	500.500000	0.011141	250.0
97.5%	975.025000	0.029932	250.0
max	1000.000000	0.045654	250.0

[62]:
```
sns.set_theme();
ax = sns.catplot(
    data=df_error_max_v_est, x="Porcenta_tama", y="Error_max",
    kind="point", height=4, aspect=.7, errorbar='pi'
);
ax.set(xlabel='Porcentaje casillas', ylabel='Error');
```

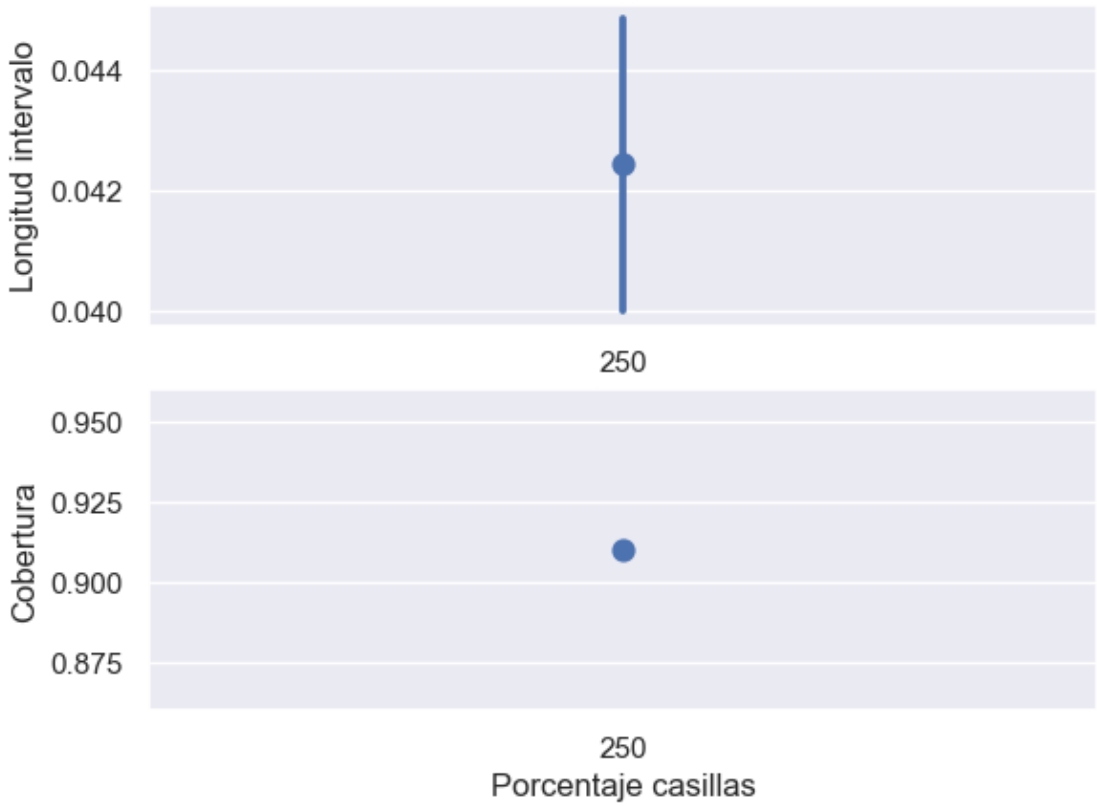


[63]:
```
# Base para poder hacer las gráficas de mínima cobertura e intervalos más largos
df_metr_boot_c_v_est=df_metr_boot_v_est.drop(columns="Cobertura").
 ↪merge(df_can_cob_v_est[["Candidato","Cobertura"]], on=["Candidato"],␣
 ↪how='left')
df_metr_boot_c_v_est
```

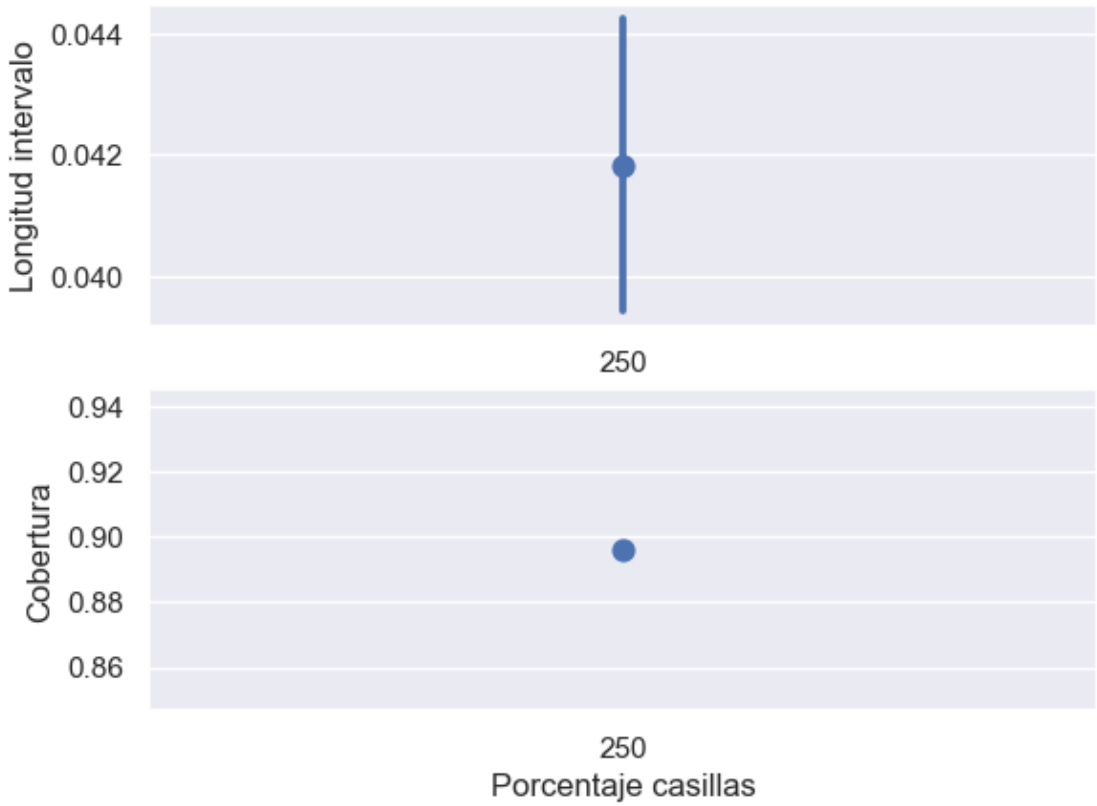
[63]:
```
      Num_bootstrap  cuantil_025  cuantil_975  Longitud_intervalo  \
0                 1     0.497661     0.540644            0.042984
1                 2     0.498525     0.540947            0.042423
2                 3     0.511973     0.554100            0.042127
3                 4     0.500341     0.542362            0.042021
4                 5     0.486716     0.531513            0.044797
…               …           …            …                   …
4995            996     0.017124     0.029780            0.012656
4996            997     0.016572     0.029038            0.012466
4997            998     0.022442     0.037669            0.015227
4998            999     0.019415     0.032641            0.013226
4999           1000     0.018352     0.031254            0.012902

      Estamacion_puntual  Porcentaje_real                  Candidato  \
0               0.519384         0.515125          JOAQUIN_DIAZ_MENA
1               0.519175         0.515125          JOAQUIN_DIAZ_MENA
2               0.532674         0.515125          JOAQUIN_DIAZ_MENA
```

```
3                0.521621         0.515125          JOAQUIN_DIAZ_MENA
4                0.508624         0.515125          JOAQUIN_DIAZ_MENA
…                   …                …                          …
4995             0.023026         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4996             0.022437         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4997             0.029654         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4998             0.025426         0.021467  VOTOS_NULOS_CAND_NO_REGIS
4999             0.024569         0.021467  VOTOS_NULOS_CAND_NO_REGIS

         Error  Porcenta_tama  Cobertura
0     0.004258            250      0.910
1     0.004049            250      0.910
2     0.017548            250      0.910
3     0.006496            250      0.910
4     0.006501            250      0.910
…        …             …         …
4995  0.001559            250      0.937
4996  0.000970            250      0.937
4997  0.008187            250      0.937
4998  0.003959            250      0.937
4999  0.003102            250      0.937

[5000 rows x 10 columns]
```

[64]:
```
facet_kws={'sharey': False, 'sharex': True}
fig, axx =plt.subplots(2,1)
ax_1=sns.pointplot(
   ␣
 ↪data=df_metr_boot_c_v_est[df_metr_boot_c_v_est["Candidato"]==candidato_mas_largo_v_est],␣
 ↪x="Porcenta_tama", y="Longitud_intervalo", errorbar='pi',ax=axx[0]
);
ax_2=sns.pointplot(
   ␣
 ↪data=df_metr_boot_c_v_est[df_metr_boot_c_v_est["Candidato"]==candidato_mas_largo_v_est].
 ↪head(1), x="Porcenta_tama", y="Cobertura", ax=axx[1]
)
ax_1.set(xlabel='Porcentaje casillas', ylabel='Longitud intervalo');
ax_2.set(xlabel='Porcentaje casillas', ylabel='Cobertura');
```



[65]:
```
facet_kws={'sharey': False, 'sharex': True}
fig, axx =plt.subplots(2,1)
ax_1=sns.pointplot(
    ␣
 ↪data=df_metr_boot_c_v_est[df_metr_boot_c_v_est["Candidato"]==candidato_cober_men_v_est],␣
 ↪x="Porcenta_tama", y="Longitud_intervalo", errorbar='pi', ax=axx[0]
);
ax_2=sns.pointplot(
    ␣
 ↪data=df_metr_boot_c_v_est[df_metr_boot_c_v_est["Candidato"]==candidato_cober_men_v_est].
 ↪head(1), x="Porcenta_tama", y="Cobertura", ax=axx[1]
)
ax_1.set(xlabel='Porcentaje casillas', ylabel='Longitud intervalo');
ax_2.set(xlabel='Porcentaje casillas', ylabel='Cobertura');
```



[66]:
```
# %load_ext line_profiler
# %lprun -f boot_votos_estratificado boot_votos_estratificado(df_act_yuc_i,␣
↪num_boot=10, n_sample=250, n_vot_sub=2000)
```

3 Comparación de los dos bootstraps

3.1 Análisis de los bootstrap por casillas

3.1.1 Sin estratificar

[67]:
```
# Probamos distintos valores de tamaño de la muestra de las casillas y la␣
↪submuestra de votos
muestra_casillas = [100, 150, 200, 250, 300, 350, 400]

# Dataframe donde se guardará todo
df_boostraps_se=pd.DataFrame()

# Contador para el for
k=0

for mc in muestra_casillas:
```

```
        # Boostrap por votos (sin estratificar)
        inter_prob, est_punt=bootstrap_simple(df_act_yuc_i, num_boot=1000,␣
↪n_sample=mc)
        # Calculamos las métricas
        df_metr_boot, df_error_max, df_can_cob= metricas_bootstrap(prop_reales,␣
↪inter_prob, est_punt, porcenta_tama=mc)

        # Los valores de las muestra y submuestras
        df_boostraps_se.loc[k,"Tamaño_muestra_casillas"]= mc

        # Las métricas para el error máximo
        df_boostraps_se.
↪loc[k,["Error_maximo_q_025","Error_maximo_mediana","Error_maximo_q_975"]]=df_error_max["Err
↪quantile([0.025,0.5,0.975]).to_list()

        # Calculamos el candidato que tenga intervalos más largos (en promedio)
        candidato_int_m_l_c=df_metr_boot.groupby(["Candidato"]).
↪agg({'Longitud_intervalo':'mean'}).reset_index()["Candidato"].values[0]
        # Candidato con el intervalos más grandes
        df_boostraps_se.loc[k,␣
↪"Intervalo_M_grande_candidato_nom"]=candidato_int_m_l_c
        # Cobertura del candidato con el intervalos más grandes
        df_boostraps_se.loc[k,␣
↪["Intervalo_M_grande_candidato_cob"]]=df_can_cob[df_can_cob["Candidato"]==candidato_int_m_l
↪values[0]
        # Las métricas del candidato con los intervalos más grandes
        df_boostraps_se.
↪loc[k,["Intervalo_M_grande_inter_q_025","Intervalo_M_grande_inter_mediana","Intervalo_M_gra
↪quantile([0.025,0.5,0.975]).to_list()

        # Calculamos el candidato con la menor cobertura
        # Candidato con menor cobertura
       ␣
↪candidato_cober_men=df_can_cob[df_can_cob["Cobertura"]==df_can_cob["Cobertura"].
↪min()]["Candidato"].values[0]

        # Candidato con el intervalos más grandes
        df_boostraps_se.loc[k, "Cobertura_m_candidato_nom"]=candidato_cober_men
        # Cobertura del candidato con menor cobertura
        df_boostraps_se.loc[k,␣
↪["Cobertura_m_candidato"]]=df_can_cob[df_can_cob["Candidato"]==candidato_cober_men]["Cobert
↪values[0]
        # Las métricas del candidato con la menor cobertura
```

```
        df_boostraps_se.
↪loc[k,["Cobertura_m_inter_q_025","Cobertura_m_inter_mediana","Cobertura_m_inter_q_975"]]=df
↪quantile([0.025,0.5,0.975]).to_list()


        # Coberturas de los candidatos en el bootstrap
        df_boostraps_se.loc[k,␣
↪["Cob_JOAQUIN_DIAZ_MENA","Cob_RENAN_BARRERA_CONCHA","Cob_VIDA_ARAVARI_GOMEZ_HERRERA","Cob_V
↪sort_values(by="Candidato")["Cobertura"].to_list()

        k=k+1

        print("Muestra casillas:", mc)
```

```
100
Muestra casillas: 100
150
Muestra casillas: 150
200
Muestra casillas: 200
250
Muestra casillas: 250
300
Muestra casillas: 300
350
Muestra casillas: 350
400
Muestra casillas: 400
```

[241]:
```
# Guardamos un csv
df_boostraps_se.to_csv(r"/Users/damian/Documents/Tesis maestria/Bootstraps/
↪Bases/Boostrap por casillas/
↪Distintas_muestras_submuestras_bootstrap_casillas_sin_est_1.csv")
```

[124]:
```
# Agregamos una columna para identificar que es sin estratificar
df_boostraps_se["Estratificación"] = "Sin estratificar (casillas)"
df_boostraps_se
```

[124]:

	Tamaño_muestra_casillas	Error_maximo_q_025	Error_maximo_mediana	\
0	100.0	0.002082	0.011726	
1	150.0	0.001481	0.009102	
2	200.0	0.001470	0.007852	
3	250.0	0.001444	0.006472	
4	300.0	0.001181	0.005938	
5	350.0	0.001161	0.005860	
6	400.0	0.001071	0.005357	

```
   Error_maximo_q_975 Intervalo_M_grande_candidato_nom  \
0            0.032818                JOAQUIN_DIAZ_MENA
1            0.027687                JOAQUIN_DIAZ_MENA
2            0.022880                JOAQUIN_DIAZ_MENA
3            0.020072                JOAQUIN_DIAZ_MENA
4            0.018555                JOAQUIN_DIAZ_MENA
5            0.016537                JOAQUIN_DIAZ_MENA
6            0.016599                JOAQUIN_DIAZ_MENA

   Intervalo_M_grande_candidato_cob  Intervalo_M_grande_inter_q_025  \
0                             0.948                        0.043906
1                             0.945                        0.037443
2                             0.955                        0.034174
3                             0.963                        0.031203
4                             0.963                        0.029041
5                             0.964                        0.027405
6                             0.964                        0.025817

   Intervalo_M_grande_inter_mediana  Intervalo_M_grande_inter_q_975  \
0                          0.054279                        0.066212
1                          0.044915                        0.052655
2                          0.039497                        0.044929
3                          0.035486                        0.040258
4                          0.032597                        0.036722
5                          0.030672                        0.033889
6                          0.028835                        0.031960

     Cobertura_m_candidato_nom  Cobertura_m_candidato  \
0  YAMIL_JASMIN_LOPEZ_MANRIQUE                  0.730
1  YAMIL_JASMIN_LOPEZ_MANRIQUE                  0.809
2  YAMIL_JASMIN_LOPEZ_MANRIQUE                  0.826
3  YAMIL_JASMIN_LOPEZ_MANRIQUE                  0.864
4  YAMIL_JASMIN_LOPEZ_MANRIQUE                  0.880
5  YAMIL_JASMIN_LOPEZ_MANRIQUE                  0.909
6  YAMIL_JASMIN_LOPEZ_MANRIQUE                  0.929

   Cobertura_m_inter_q_025  Cobertura_m_inter_mediana  \
0                 0.001281                   0.003450
1                 0.001186                   0.003373
2                 0.001083                   0.002989
3                 0.001044                   0.002901
4                 0.000993                   0.002691
5                 0.001028                   0.002552
6                 0.001045                   0.002429

   Cobertura_m_inter_q_975  Cob_JOAQUIN_DIAZ_MENA  Cob_RENAN_BARRERA_CONCHA  \
0                 0.011382                  0.948                     0.941
```

```
1                 0.008262                  0.945                      0.950
2                 0.006531                  0.955                      0.961
3                 0.005749                  0.963                      0.956
4                 0.005168                  0.963                      0.966
5                 0.004640                  0.964                      0.971
6                 0.004210                  0.964                      0.962

   Cob_VIDA_ARAVARI_GOMEZ_HERRERA  Cob_VOTOS_NULOS_CAND_NO_REGIS  \
0                           0.917                          0.928
1                           0.936                          0.941
2                           0.946                          0.941
3                           0.946                          0.941
4                           0.952                          0.937
5                           0.943                          0.954
6                           0.960                          0.951

   Cob_YAMIL_JASMIN_LOPEZ_MANRIQUE              Estratificación
0                            0.730  Sin estratificar (casillas)
1                            0.809  Sin estratificar (casillas)
2                            0.826  Sin estratificar (casillas)
3                            0.864  Sin estratificar (casillas)
4                            0.880  Sin estratificar (casillas)
5                            0.909  Sin estratificar (casillas)
6                            0.929  Sin estratificar (casillas)
```

3.1.2 Estratificado

[242]:
```
# Probamos distintos valores de tamaño de la muestra de las casillas y la␣
↪submuestra de votos
muestra_casillas = [100, 150, 200, 250, 300, 350, 400]

# Dataframe donde se guardará todo
df_boostraps_e=pd.DataFrame()

# Contador para el for
k=0

# Hacemos los for sobre el tamaño de las muestras y submuestras
for mc in muestra_casillas:
    # Para medir cuanto tiempo se tarda en correr
    loop_time = time.time()
    # Boostrap por votos (estratificado)
    inter_prob_v_est, est_punt_v_est=boot_estratificado(df_act_yuc_i,␣
↪num_boot=1000, n_sample=mc)

    # Calculamos las métricas
```

```
    df_metr_boot_est, df_error_max_est, df_can_cob_est=␣
↪metricas_bootstrap(prop_reales, inter_prob_v_est, est_punt_v_est,␣
↪porcenta_tama=mc)

    # Los valores de las muestra y submuestras
    df_boostraps_e.loc[k,"Tamaño_muestra_casillas"]=mc

    # Las métricas para el error máximo
    df_boostraps_e.
↪loc[k,["Error_maximo_q_025","Error_maximo_mediana","Error_maximo_q_975"]]=df_error_max_est[
↪quantile([0.025,0.5,0.975]).to_list()

    # Calculamos el candidato que tenga intervalos más largos (en promedio)
    candidato_int_m_l_c_est=df_metr_boot_est.groupby(["Candidato"]).
↪agg({'Longitud_intervalo':'mean'}).reset_index()["Candidato"].values[0]
    # Candidato con el intervalos más grandes
    df_boostraps_e.loc[k,␣
↪"Intervalo_M_grande_candidato_nom"]=candidato_int_m_l_c_est
    # Cobertura del candidato con el intervalos más grandes
    df_boostraps_e.loc[k,␣
↪["Intervalo_M_grande_candidato_cob"]]=df_can_cob_est[df_can_cob_est["Candidato"]==candidato
↪values[0]
    # Las métricas del candidato con los intervalos más grandes
    df_boostraps_e.
↪loc[k,["Intervalo_M_grande_inter_q_025","Intervalo_M_grande_inter_mediana","Intervalo_M_gra
↪quantile([0.025,0.5,0.975]).to_list()

    # Calculamos el candidato con la menor cobertura
    # Candidato con menor cobertura
  ␣
↪candidato_cober_men_est=df_can_cob_est[df_can_cob_est["Cobertura"]==df_can_cob_est["Cobertu
↪min()]["Candidato"].values[0]

    # Candidato con el intervalos más grandes
    df_boostraps_e.loc[k, "Cobertura_m_candidato_nom"]=candidato_cober_men_est
    # Cobertura del candidato con menor cobertura
    df_boostraps_e.loc[k,␣
↪["Cobertura_m_candidato"]]=df_can_cob_est[df_can_cob_est["Candidato"]==candidato_cober_men_
↪values[0]
    # Las métricas del candidato con la menor cobertura
    df_boostraps_e.
↪loc[k,["Cobertura_m_inter_q_025","Cobertura_m_inter_mediana","Cobertura_m_inter_q_975"]]=df
↪quantile([0.025,0.5,0.975]).to_list()


    # Coberturas de los candidatos en el bootstrap
```

```
    df_boostraps_e.loc[k,␣
↪["Cob_JOAQUIN_DIAZ_MENA","Cob_RENAN_BARRERA_CONCHA","Cob_VIDA_ARAVARI_GOMEZ_HERRERA","Cob_V
↪sort_values(by="Candidato")["Cobertura"].to_list()

    k=k+1

    print("Muestra casillas:", mc)
    print('Tiempo que tarda correr una iteración del for {}'.format((time.
↪time() - loop_time)))

# Lo guardamos en otra base copia por si sin querer pongo a correr el código
df_boostraps_e_f=df_boostraps_e.copy()
df_boostraps_e_f["Estratificación"] = "Estratificación por distritos (casillas)"
```

```
100
Muestra casillas: 100
Tiempo que tarda correr una iteración del for 212.46737217903137
150
Muestra casillas: 150
Tiempo que tarda correr una iteración del for 220.97292184829712
200
Muestra casillas: 200
Tiempo que tarda correr una iteración del for 226.76472091674805
250
Muestra casillas: 250
Tiempo que tarda correr una iteración del for 232.69735670089722
300
Muestra casillas: 300
Tiempo que tarda correr una iteración del for 239.30866193771362
350
Muestra casillas: 350
Tiempo que tarda correr una iteración del for 244.79878187179565
400
Muestra casillas: 400
Tiempo que tarda correr una iteración del for 253.10825181007385
```

[251]:
```
# Guardamos un csv
df_boostraps_e_f.to_csv(r"/Users/damian/Documents/Tesis maestria/Bootstraps/
↪Bases/Boostrap por casillas/
↪Distintas_muestras_submuestras_bootstrap_casillas_estratificado_1.csv")
```

[252]:
```
df_boostraps_e_f
```

[252]:

	Tamaño_muestra_casillas	Error_maximo_q_025	Error_maximo_mediana	\
0	100.0	0.001909	0.008420	
1	150.0	0.001521	0.007042	
2	200.0	0.001215	0.005826	

3	250.0	0.001011	0.005408
4	300.0	0.000930	0.004685
5	350.0	0.000902	0.004381
6	400.0	0.000882	0.004242

	Error_maximo_q_975	Intervalo_M_grande_candidato_nom \
0	0.024146	JOAQUIN_DIAZ_MENA
1	0.019725	JOAQUIN_DIAZ_MENA
2	0.016717	JOAQUIN_DIAZ_MENA
3	0.014872	JOAQUIN_DIAZ_MENA
4	0.012806	JOAQUIN_DIAZ_MENA
5	0.011695	JOAQUIN_DIAZ_MENA
6	0.011940	JOAQUIN_DIAZ_MENA

	Intervalo_M_grande_candidato_cob	Intervalo_M_grande_inter_q_025 \
0	0.872	0.025369
1	0.919	0.023360
2	0.931	0.021536
3	0.928	0.020119
4	0.959	0.019086
5	0.964	0.017997
6	0.954	0.017338

	Intervalo_M_grande_inter_mediana	Intervalo_M_grande_inter_q_975 \
0	0.030919	0.036931
1	0.027289	0.031747
2	0.024856	0.028727
3	0.022862	0.025752
4	0.021477	0.024154
5	0.020195	0.022511
6	0.019241	0.021361

	Cobertura_m_candidato_nom	Cobertura_m_candidato \
0	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.680
1	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.771
2	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.804
3	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.866
4	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.885
5	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.887
6	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.908

	Cobertura_m_inter_q_025	Cobertura_m_inter_mediana \
0	0.001046	0.002652
1	0.000997	0.002603
2	0.000941	0.002465
3	0.000947	0.002429
4	0.000920	0.002311

```
5                 0.000906                   0.002236
6                 0.000883                   0.002081

   Cobertura_m_inter_q_975  Cob_JOAQUIN_DIAZ_MENA  Cob_RENAN_BARRERA_CONCHA  \
0                 0.009110                  0.872                     0.868
1                 0.007019                  0.919                     0.900
2                 0.005696                  0.931                     0.919
3                 0.004988                  0.928                     0.920
4                 0.004472                  0.959                     0.944
5                 0.003962                  0.964                     0.956
6                 0.003616                  0.954                     0.952

   Cob_VIDA_ARAVARI_GOMEZ_HERRERA  Cob_VOTOS_NULOS_CAND_NO_REGIS  \
0                           0.847                          0.870
1                           0.870                          0.907
2                           0.897                          0.933
3                           0.919                          0.931
4                           0.930                          0.935
5                           0.936                          0.938
6                           0.941                          0.951

   Cob_YAMIL_JASMIN_LOPEZ_MANRIQUE                          Estratificación
0                            0.680  Estratificación por distritos (casillas)
1                            0.771  Estratificación por distritos (casillas)
2                            0.804  Estratificación por distritos (casillas)
3                            0.866  Estratificación por distritos (casillas)
4                            0.885  Estratificación por distritos (casillas)
5                            0.887  Estratificación por distritos (casillas)
6                            0.908  Estratificación por distritos (casillas)
```

[253]:
```
# Juntamos las bases
df_boostraps_c=pd.concat([df_boostraps_se, df_boostraps_e_f]).
 ↪reset_index(drop=True)
df_boostraps_c
```

[253]:
```
    Tamaño_muestra_casillas  Error_maximo_q_025  Error_maximo_mediana  \
0                     100.0            0.002082              0.011726
1                     150.0            0.001481              0.009102
2                     200.0            0.001470              0.007852
3                     250.0            0.001444              0.006472
4                     300.0            0.001181              0.005938
5                     350.0            0.001161              0.005860
6                     400.0            0.001071              0.005357
7                     100.0            0.001909              0.008420
8                     150.0            0.001521              0.007042
9                     200.0            0.001215              0.005826
10                    250.0            0.001011              0.005408
```

11	300.0	0.000930	0.004685
12	350.0	0.000902	0.004381
13	400.0	0.000882	0.004242

	Error_maximo_q_975	Intervalo_M_grande_candidato_nom \
0	0.032818	JOAQUIN_DIAZ_MENA
1	0.027687	JOAQUIN_DIAZ_MENA
2	0.022880	JOAQUIN_DIAZ_MENA
3	0.020072	JOAQUIN_DIAZ_MENA
4	0.018555	JOAQUIN_DIAZ_MENA
5	0.016537	JOAQUIN_DIAZ_MENA
6	0.016599	JOAQUIN_DIAZ_MENA
7	0.024146	JOAQUIN_DIAZ_MENA
8	0.019725	JOAQUIN_DIAZ_MENA
9	0.016717	JOAQUIN_DIAZ_MENA
10	0.014872	JOAQUIN_DIAZ_MENA
11	0.012806	JOAQUIN_DIAZ_MENA
12	0.011695	JOAQUIN_DIAZ_MENA
13	0.011940	JOAQUIN_DIAZ_MENA

	Intervalo_M_grande_candidato_cob	Intervalo_M_grande_inter_q_025 \
0	0.948	0.043906
1	0.945	0.037443
2	0.955	0.034174
3	0.963	0.031203
4	0.963	0.029041
5	0.964	0.027405
6	0.964	0.025817
7	0.872	0.025369
8	0.919	0.023360
9	0.931	0.021536
10	0.928	0.020119
11	0.959	0.019086
12	0.964	0.017997
13	0.954	0.017338

	Intervalo_M_grande_inter_mediana	Intervalo_M_grande_inter_q_975 \
0	0.054279	0.066212
1	0.044915	0.052655
2	0.039497	0.044929
3	0.035486	0.040258
4	0.032597	0.036722
5	0.030672	0.033889
6	0.028835	0.031960
7	0.030919	0.036931
8	0.027289	0.031747
9	0.024856	0.028727

10	0.022862	0.025752
11	0.021477	0.024154
12	0.020195	0.022511
13	0.019241	0.021361

	Cobertura_m_candidato_nom	Cobertura_m_candidato \
0	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.730
1	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.809
2	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.826
3	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.864
4	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.880
5	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.909
6	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.929
7	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.680
8	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.771
9	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.804
10	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.866
11	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.885
12	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.887
13	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.908

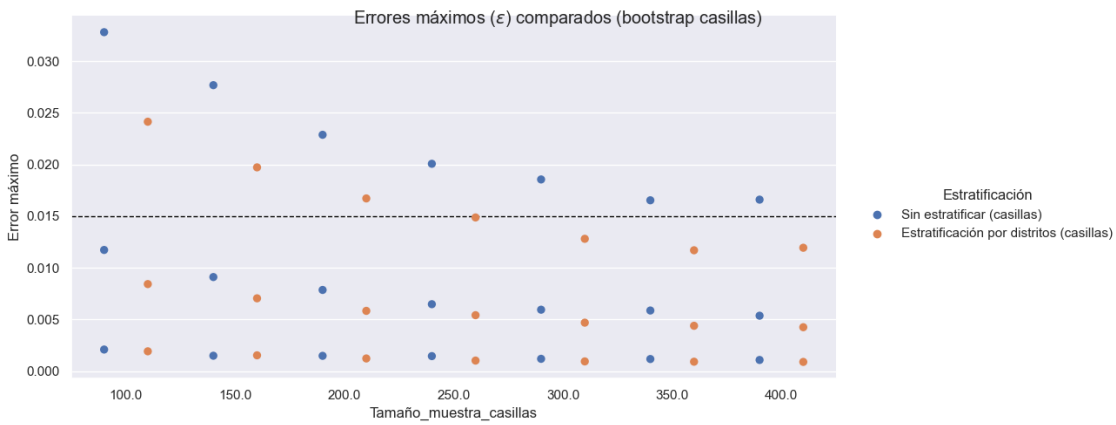
	Cobertura_m_inter_q_025	Cobertura_m_inter_mediana \
0	0.001281	0.003450
1	0.001186	0.003373
2	0.001083	0.002989
3	0.001044	0.002901
4	0.000993	0.002691
5	0.001028	0.002552
6	0.001045	0.002429
7	0.001046	0.002652
8	0.000997	0.002603
9	0.000941	0.002465
10	0.000947	0.002429
11	0.000920	0.002311
12	0.000906	0.002236
13	0.000883	0.002081

	Cobertura_m_inter_q_975	Cob_JOAQUIN_DIAZ_MENA	Cob_RENAN_BARRERA_CONCHA \
0	0.011382	0.948	0.941
1	0.008262	0.945	0.950
2	0.006531	0.955	0.961
3	0.005749	0.963	0.956
4	0.005168	0.963	0.966
5	0.004640	0.964	0.971
6	0.004210	0.964	0.962
7	0.009110	0.872	0.868
8	0.007019	0.919	0.900

```
9                  0.005696                  0.931                     0.919
10                 0.004988                  0.928                     0.920
11                 0.004472                  0.959                     0.944
12                 0.003962                  0.964                     0.956
13                 0.003616                  0.954                     0.952

    Cob_VIDA_ARAVARI_GOMEZ_HERRERA  Cob_VOTOS_NULOS_CAND_NO_REGIS  \
0                            0.917                          0.928
1                            0.936                          0.941
2                            0.946                          0.941
3                            0.946                          0.941
4                            0.952                          0.937
5                            0.943                          0.954
6                            0.960                          0.951
7                            0.847                          0.870
8                            0.870                          0.907
9                            0.897                          0.933
10                           0.919                          0.931
11                           0.930                          0.935
12                           0.936                          0.938
13                           0.941                          0.951

    Cob_YAMIL_JASMIN_LOPEZ_MANRIQUE                          Estratificación
0                             0.730               Sin estratificar (casillas)
1                             0.809               Sin estratificar (casillas)
2                             0.826               Sin estratificar (casillas)
3                             0.864               Sin estratificar (casillas)
4                             0.880               Sin estratificar (casillas)
5                             0.909               Sin estratificar (casillas)
6                             0.929               Sin estratificar (casillas)
7                             0.680  Estratificación por distritos (casillas)
8                             0.771  Estratificación por distritos (casillas)
9                             0.804  Estratificación por distritos (casillas)
10                            0.866  Estratificación por distritos (casillas)
11                            0.885  Estratificación por distritos (casillas)
12                            0.887  Estratificación por distritos (casillas)
13                            0.908  Estratificación por distritos (casillas)
```

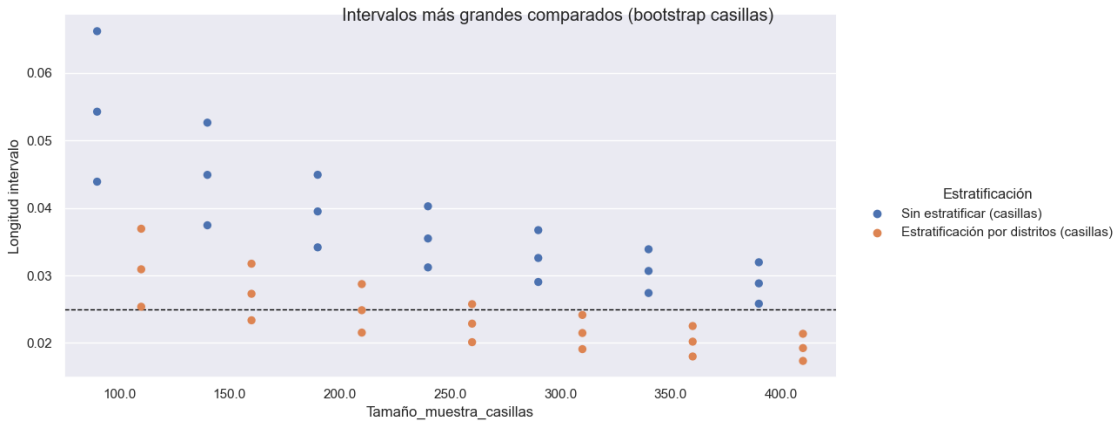
[259]:
```
# Para hacer la gráfica del error con los distintos tamaños de la submuestra de␣
↪votos
df_grafica_error_aux_c=pd.melt(df_boostraps_c[["Tamaño_muestra_casillas",␣
↪"Estratificación", "Error_maximo_q_025", "Error_maximo_mediana",␣
↪"Error_maximo_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Estratificación"], value_vars=["Error_maximo_q_025",␣
↪"Error_maximo_mediana", "Error_maximo_q_975"])
```

```
# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per=sns.catplot(data=df_grafica_error_aux_c, x="Tamaño_muestra_casillas",␣
↪y="value", hue="Estratificación", kind='strip', aspect=2/1, palette="deep",␣
↪jitter=False, dodge=True, s = 45);


per.set(xlabel='Tamaño_muestra_casillas', ylabel='Error máximo')
per.figure.subplots_adjust(top=0.9)
per.refline(y=0.015, linewidth=1, color='black', ls='--');
per.figure.suptitle('Errores máximos ($\epsilon$) comparados (bootstrap␣
↪casillas)');
```

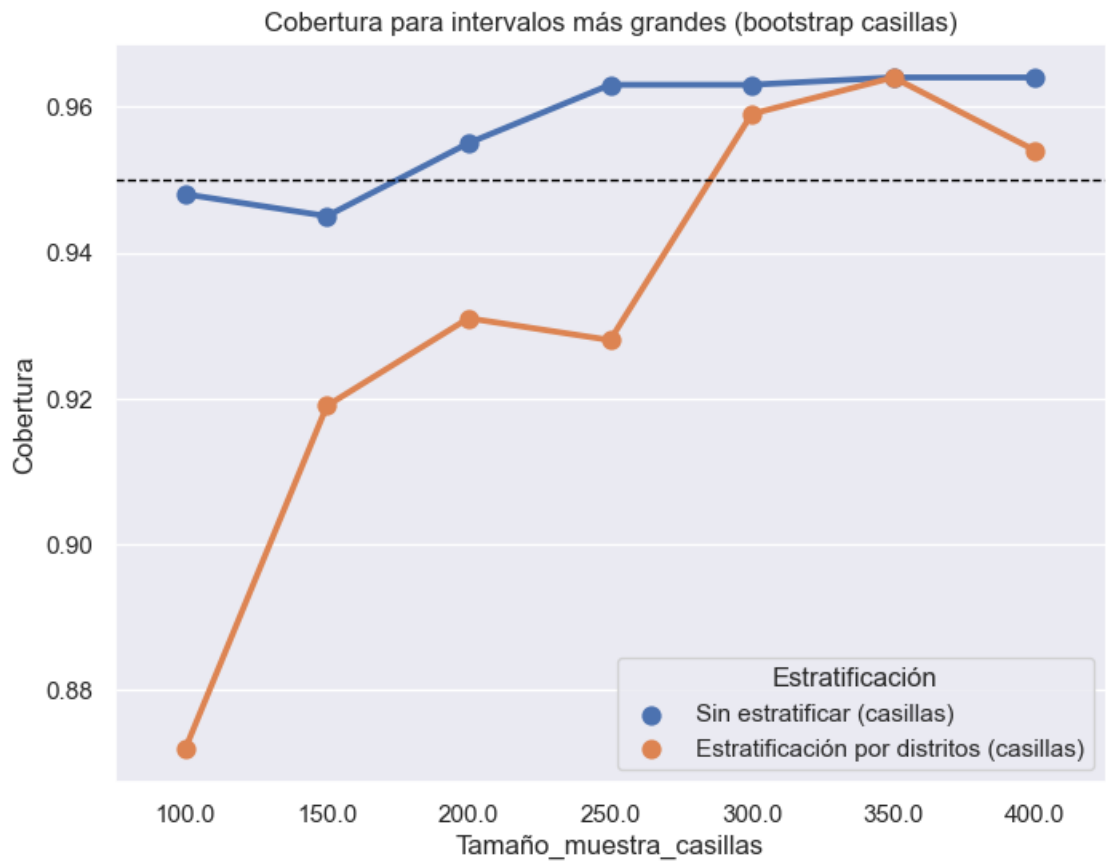


[260]:
```
# Para hacer la gráfica de longitud de intervalo (para el candidato con los␣
↪intervalos más grandes) con los distintos tamños de la submuestra de votos
df_grafica_int_im_aux_c=pd.melt(df_boostraps_c[["Tamaño_muestra_casillas",␣
↪"Estratificación", "Intervalo_M_grande_inter_q_025",␣
↪"Intervalo_M_grande_inter_mediana",␣
↪"Intervalo_M_grande_inter_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Estratificación"], value_vars=["Intervalo_M_grande_inter_q_025",␣
↪"Intervalo_M_grande_inter_mediana", "Intervalo_M_grande_inter_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_i=sns.catplot(data=df_grafica_int_im_aux_c, x="Tamaño_muestra_casillas",␣
↪y="value", hue="Estratificación", kind='strip', aspect=2/1, palette="deep",␣
↪jitter=False, dodge=True, s = 45);

per_i.set(xlabel='Tamaño_muestra_casillas', ylabel='Longitud intervalo')
```

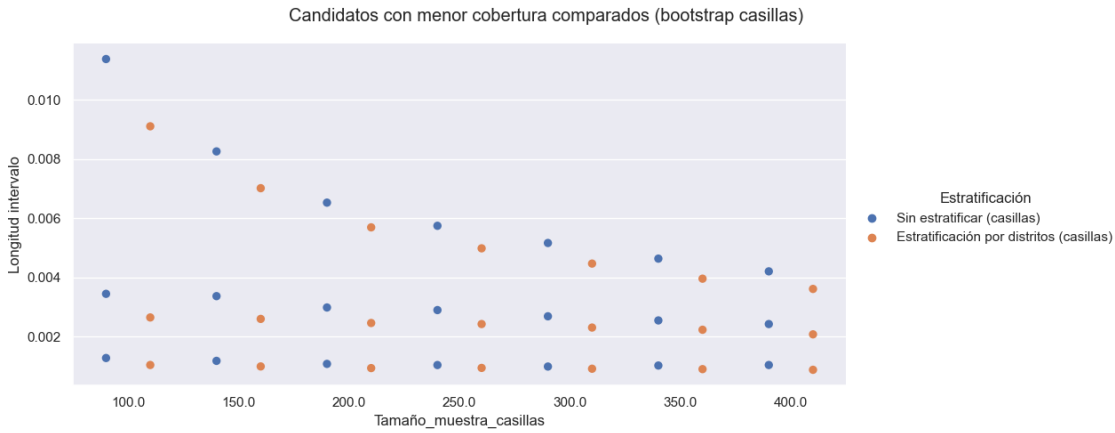
```
per_i.figure.subplots_adjust(top=0.9)
per_i.refline(y=0.025, linewidth=1, color='black', ls='--');
per_i.figure.suptitle('Intervalos más grandes comparados (bootstrap casillas)');
```



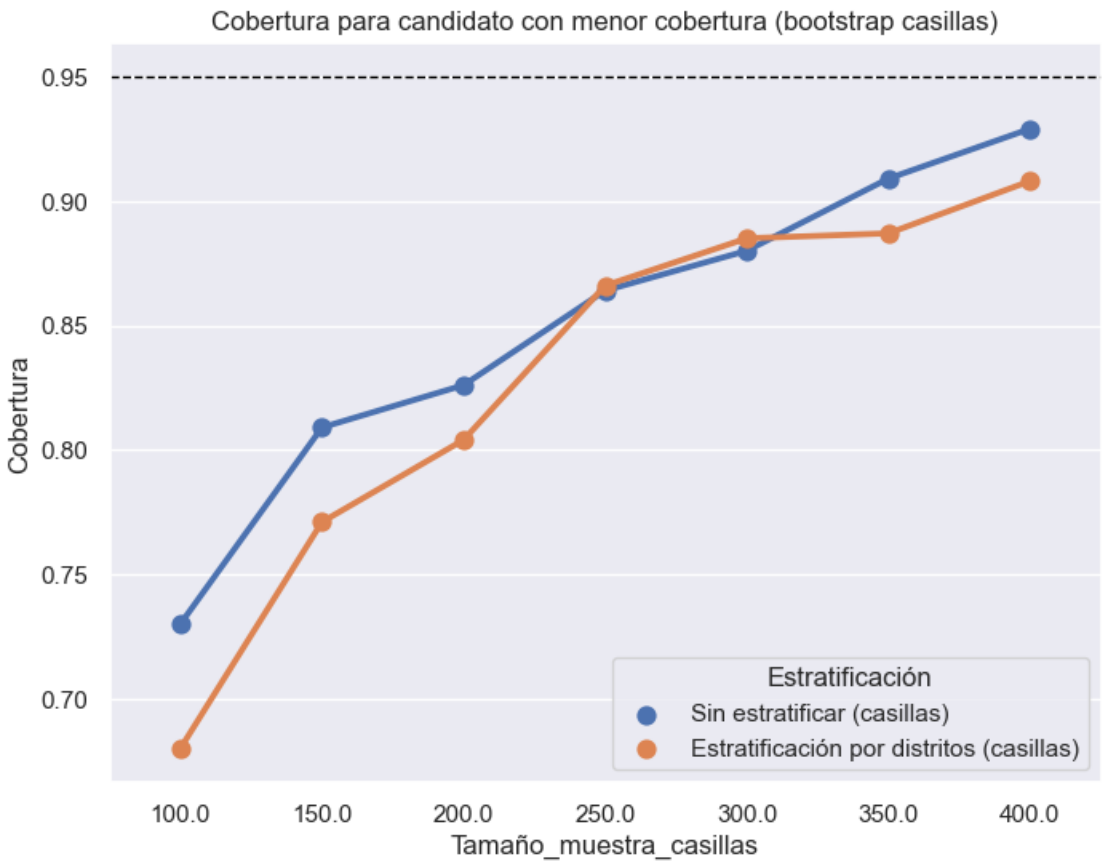
[262]:
```
fig, ax = plt.subplots(figsize=(8,6))

per_c=sns.pointplot(ax=ax,
    data=df_boostraps_c, x="Tamaño_muestra_casillas",␣
 ↪y="Intervalo_M_grande_candidato_cob", hue="Estratificación"
).set_title("Cobertura para intervalos más grandes (bootstrap casillas)");
ax.set(ylabel='Cobertura')
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
```



[257]:
```
# Para hacer la gráfica de longitud de intervalo (para el candidato con los␣
↪intervalos más grandes) con los distintos tamños de la submuestra de votos
df_grafica_int_mc_aux_c=pd.melt(df_boostraps_c[["Tamaño_muestra_casillas",␣
↪"Estratificación", "Cobertura_m_inter_q_025", "Cobertura_m_inter_mediana",␣
↪"Cobertura_m_inter_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Estratificación"], value_vars=["Cobertura_m_inter_q_025",␣
↪"Cobertura_m_inter_mediana", "Cobertura_m_inter_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_mc=sns.catplot(data=df_grafica_int_mc_aux_c, x="Tamaño_muestra_casillas",␣
↪y="value", hue="Estratificación", kind='strip', aspect=2/1, palette="deep",␣
↪jitter=False, dodge=True, s = 45);

per_mc.set(xlabel='Tamaño_muestra_casillas', ylabel='Longitud intervalo')
per_mc.figure.subplots_adjust(top=0.9)
per_mc.figure.suptitle('Candidatos con menor cobertura comparados (bootstrap␣
↪casillas)');
```



[264]:
```
fig, ax = plt.subplots(figsize=(8,6))

sns.pointplot(ax=ax, data=df_boostraps_c, x="Tamaño_muestra_casillas",␣
 ↪y="Cobertura_m_candidato", hue="Estratificación").set_title("Cobertura para␣
 ↪candidato con menor cobertura (bootstrap casillas)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
ax.set(ylabel='Cobertura');
```



3.2 Análisis de los bootstrap por votos

3.2.1 Sin estratificar

[93]:
```
# Probamos distintos valores de tamaño de la muestra de las casillas y la␣
↪submuestra de votos
muestra_casillas = [100, 150, 200, 250, 300, 350, 400]
submuestra_votos = [500, 1000, 1500, 2000, 3000, 4000]

# Dataframe donde se guardará todo
df_boostraps_votos_se=pd.DataFrame()

# Contador para el for
k=0

for mc in muestra_casillas:
    for smv in submuestra_votos:
        # Boostrap por votos (sin estratificar)
        inter_prob_v, est_punt_v=bootstrap_votos_simple(df_act_yuc_i,␣
↪num_boot=1000, n_sample=mc, n_vot_sub=smv)
        # Calculamos las métricas
        df_metr_boot_v, df_error_max_v, df_can_cob_v =␣
↪metricas_bootstrap(prop_reales, inter_prob_v, est_punt_v, porcenta_tama=mc)

        # Los valores de las muestra y submuestras
        df_boostraps_votos_se.
↪loc[k,["Tamaño_muestra_casillas","Tamaño_submuestra_votos"]]=[mc,smv]

        # Las métricas para el error máximo
        df_boostraps_votos_se.
↪loc[k,["Error_maximo_q_025","Error_maximo_mediana","Error_maximo_q_975"]]=df_error_max_v["E
↪quantile([0.025,0.5,0.975]).to_list()

        # Calculamos el candidato que tenga intervalos más largos (en promedio)
        candidato_int_m_l=df_metr_boot_v.groupby(["Candidato"]).
↪agg({'Longitud_intervalo':'mean'}).reset_index()["Candidato"].values[0]
        # Candidato con el intervalos más grandes
        df_boostraps_votos_se.loc[k,␣
↪"Intervalo_M_grande_candidato_nom"]=candidato_int_m_l
        # Cobertura del candidato con el intervalos más grandes
        df_boostraps_votos_se.loc[k,␣
↪["Intervalo_M_grande_candidato_cob"]]=df_can_cob_v[df_can_cob_v["Candidato"]==candidato_int
↪values[0]
        # Las métricas del candidato con los intervalos más grandes
```

```
        df_boostraps_votos_se.
↪loc[k,["Intervalo_M_grande_inter_q_025","Intervalo_M_grande_inter_mediana","Intervalo_M_gra
↪quantile([0.025,0.5,0.975]).to_list()

        # Calculamos el candidato con la menor cobertura
        # Candidato con menor cobertura
       ␣
↪candidato_cober_men_v=df_can_cob_v[df_can_cob_v["Cobertura"]==df_can_cob_v["Cobertura"].
↪min()]["Candidato"].values[0]

        # Candidato con el intervalos más grandes
        df_boostraps_votos_se.loc[k,␣
↪"Cobertura_m_candidato_nom"]=candidato_cober_men_v
        # Cobertura del candidato con menor cobertura
        df_boostraps_votos_se.loc[k,␣
↪["Cobertura_m_candidato"]]=df_can_cob_v[df_can_cob_v["Candidato"]==candidato_cober_men_v]["
↪values[0]
        # Las métricas del candidato con la menor cobertura
        df_boostraps_votos_se.
↪loc[k,["Cobertura_m_inter_q_025","Cobertura_m_inter_mediana","Cobertura_m_inter_q_975"]]=df
↪quantile([0.025,0.5,0.975]).to_list()


        # Coberturas de los candidatos en el bootstrap
        df_boostraps_votos_se.loc[k,␣
↪["Cob_JOAQUIN_DIAZ_MENA","Cob_RENAN_BARRERA_CONCHA","Cob_VIDA_ARAVARI_GOMEZ_HERRERA","Cob_V
↪sort_values(by="Candidato")["Cobertura"].to_list()

        k=k+1

        print("Muestra casillas:", mc, "Submuestra votos:", smv)
```

```
100
Muestra casillas: 100 Submuestra votos: 500
100
Muestra casillas: 100 Submuestra votos: 1000
100
Muestra casillas: 100 Submuestra votos: 1500
100
Muestra casillas: 100 Submuestra votos: 2000
100
Muestra casillas: 100 Submuestra votos: 3000
100
Muestra casillas: 100 Submuestra votos: 4000
150
Muestra casillas: 150 Submuestra votos: 500
150
```

```
Muestra casillas: 150 Submuestra votos: 1000
150
Muestra casillas: 150 Submuestra votos: 1500
150
Muestra casillas: 150 Submuestra votos: 2000
150
Muestra casillas: 150 Submuestra votos: 3000
150
Muestra casillas: 150 Submuestra votos: 4000
200
Muestra casillas: 200 Submuestra votos: 500
200
Muestra casillas: 200 Submuestra votos: 1000
200
Muestra casillas: 200 Submuestra votos: 1500
200
Muestra casillas: 200 Submuestra votos: 2000
200
Muestra casillas: 200 Submuestra votos: 3000
200
Muestra casillas: 200 Submuestra votos: 4000
250
Muestra casillas: 250 Submuestra votos: 500
250
Muestra casillas: 250 Submuestra votos: 1000
250
Muestra casillas: 250 Submuestra votos: 1500
250
Muestra casillas: 250 Submuestra votos: 2000
250
Muestra casillas: 250 Submuestra votos: 3000
250
Muestra casillas: 250 Submuestra votos: 4000
300
Muestra casillas: 300 Submuestra votos: 500
300
Muestra casillas: 300 Submuestra votos: 1000
300
Muestra casillas: 300 Submuestra votos: 1500
300
Muestra casillas: 300 Submuestra votos: 2000
300
Muestra casillas: 300 Submuestra votos: 3000
300
Muestra casillas: 300 Submuestra votos: 4000
350
Muestra casillas: 350 Submuestra votos: 500
350
```

```
Muestra casillas: 350 Submuestra votos: 1000
350
Muestra casillas: 350 Submuestra votos: 1500
350
Muestra casillas: 350 Submuestra votos: 2000
350
Muestra casillas: 350 Submuestra votos: 3000
350
Muestra casillas: 350 Submuestra votos: 4000
400
Muestra casillas: 400 Submuestra votos: 500
400
Muestra casillas: 400 Submuestra votos: 1000
400
Muestra casillas: 400 Submuestra votos: 1500
400
Muestra casillas: 400 Submuestra votos: 2000
400
Muestra casillas: 400 Submuestra votos: 3000
400
Muestra casillas: 400 Submuestra votos: 4000
```

[236]:
```
df_boostraps_votos_se
```

[236]:

	Tamaño_muestra_casillas	Tamaño_submuestra_votos	Error_maximo_q_025	\
0	100.0	500.0	0.005308	
1	100.0	1000.0	0.004519	
2	100.0	1500.0	0.003510	
3	100.0	2000.0	0.003194	
4	100.0	3000.0	0.003040	
5	100.0	4000.0	0.002851	
6	150.0	500.0	0.005375	
7	150.0	1000.0	0.003785	
8	150.0	1500.0	0.003415	
9	150.0	2000.0	0.003013	
10	150.0	3000.0	0.002494	
11	150.0	4000.0	0.002409	
12	200.0	500.0	0.005746	
13	200.0	1000.0	0.003611	
14	200.0	1500.0	0.003381	
15	200.0	2000.0	0.003209	
16	200.0	3000.0	0.002804	
17	200.0	4000.0	0.002246	
18	250.0	500.0	0.005007	
19	250.0	1000.0	0.004335	
20	250.0	1500.0	0.003142	
21	250.0	2000.0	0.002893	

22	250.0	3000.0	0.002423
23	250.0	4000.0	0.002141
24	300.0	500.0	0.005249
25	300.0	1000.0	0.004186
26	300.0	1500.0	0.003210
27	300.0	2000.0	0.002317
28	300.0	3000.0	0.002378
29	300.0	4000.0	0.002413
30	350.0	500.0	0.005249
31	350.0	1000.0	0.003800
32	350.0	1500.0	0.003092
33	350.0	2000.0	0.002923
34	350.0	3000.0	0.002134
35	350.0	4000.0	0.002120
36	400.0	500.0	0.005342
37	400.0	1000.0	0.003435
38	400.0	1500.0	0.003184
39	400.0	2000.0	0.002819
40	400.0	3000.0	0.002419
41	400.0	4000.0	0.002076

	Error_maximo_mediana	Error_maximo_q_975	Intervalo_M_grande_candidato_nom \
0	0.022929	0.062799	JOAQUIN_DIAZ_MENA
1	0.018041	0.051593	JOAQUIN_DIAZ_MENA
2	0.015530	0.044627	JOAQUIN_DIAZ_MENA
3	0.014859	0.041947	JOAQUIN_DIAZ_MENA
4	0.013795	0.036948	JOAQUIN_DIAZ_MENA
5	0.012261	0.037064	JOAQUIN_DIAZ_MENA
6	0.020910	0.061563	JOAQUIN_DIAZ_MENA
7	0.016508	0.046723	JOAQUIN_DIAZ_MENA
8	0.014536	0.039845	JOAQUIN_DIAZ_MENA
9	0.013179	0.036363	JOAQUIN_DIAZ_MENA
10	0.012063	0.033380	JOAQUIN_DIAZ_MENA
11	0.011209	0.032000	JOAQUIN_DIAZ_MENA
12	0.020769	0.057699	JOAQUIN_DIAZ_MENA
13	0.015410	0.046722	JOAQUIN_DIAZ_MENA
14	0.013534	0.038136	JOAQUIN_DIAZ_MENA
15	0.012380	0.034714	JOAQUIN_DIAZ_MENA
16	0.011524	0.030711	JOAQUIN_DIAZ_MENA
17	0.009845	0.028473	JOAQUIN_DIAZ_MENA
18	0.019205	0.057466	JOAQUIN_DIAZ_MENA
19	0.015512	0.041949	JOAQUIN_DIAZ_MENA
20	0.013396	0.036685	JOAQUIN_DIAZ_MENA
21	0.011819	0.032724	JOAQUIN_DIAZ_MENA
22	0.010620	0.029421	JOAQUIN_DIAZ_MENA
23	0.009668	0.027467	JOAQUIN_DIAZ_MENA
24	0.019547	0.058287	JOAQUIN_DIAZ_MENA

25	0.015413	0.041537	JOAQUIN_DIAZ_MENA
26	0.013056	0.038632	JOAQUIN_DIAZ_MENA
27	0.011005	0.031314	JOAQUIN_DIAZ_MENA
28	0.009668	0.030096	JOAQUIN_DIAZ_MENA
29	0.009134	0.026298	JOAQUIN_DIAZ_MENA
30	0.019680	0.053022	JOAQUIN_DIAZ_MENA
31	0.014759	0.040228	JOAQUIN_DIAZ_MENA
32	0.012955	0.036795	JOAQUIN_DIAZ_MENA
33	0.011262	0.030247	JOAQUIN_DIAZ_MENA
34	0.009696	0.025402	JOAQUIN_DIAZ_MENA
35	0.009072	0.024705	JOAQUIN_DIAZ_MENA
36	0.020852	0.058007	JOAQUIN_DIAZ_MENA
37	0.014388	0.040297	JOAQUIN_DIAZ_MENA
38	0.012240	0.032924	JOAQUIN_DIAZ_MENA
39	0.011289	0.031850	JOAQUIN_DIAZ_MENA
40	0.009589	0.026717	JOAQUIN_DIAZ_MENA
41	0.008985	0.026969	JOAQUIN_DIAZ_MENA

	Intervalo_M_grande_candidato_cob	Intervalo_M_grande_inter_q_025	\
0	0.905	0.082074	
1	0.844	0.058127	
2	0.831	0.047558	
3	0.784	0.041231	
4	0.738	0.033602	
5	0.700	0.029056	
6	0.909	0.082307	
7	0.890	0.058188	
8	0.856	0.047509	
9	0.831	0.040856	
10	0.782	0.033527	
11	0.736	0.029051	
12	0.932	0.081999	
13	0.891	0.058083	
14	0.882	0.047396	
15	0.863	0.041048	
16	0.821	0.033442	
17	0.795	0.029010	
18	0.931	0.082655	
19	0.895	0.058169	
20	0.896	0.047364	
21	0.892	0.040949	
22	0.860	0.033569	
23	0.844	0.029062	
24	0.935	0.082221	
25	0.914	0.057898	
26	0.897	0.047177	
27	0.900	0.041066	

28	0.867	0.033614
29	0.848	0.029128
30	0.939	0.082058
31	0.927	0.058001
32	0.919	0.047420
33	0.912	0.041154
34	0.875	0.033528
35	0.856	0.029200
36	0.937	0.082124
37	0.933	0.058251
38	0.930	0.047445
39	0.894	0.041141
40	0.897	0.033549
41	0.864	0.029125

	Intervalo_M_grande_inter_mediana	Intervalo_M_grande_inter_q_975 \
0	0.087004	0.092082
1	0.061491	0.065346
2	0.050283	0.053183
3	0.043561	0.046106
4	0.035642	0.037684
5	0.030801	0.032752
6	0.087046	0.092177
7	0.061695	0.065283
8	0.050301	0.053091
9	0.043587	0.046302
10	0.035614	0.037694
11	0.030848	0.032720
12	0.086981	0.092646
13	0.061456	0.065104
14	0.050414	0.053401
15	0.043549	0.046089
16	0.035618	0.037643
17	0.030869	0.032709
18	0.087081	0.092315
19	0.061633	0.065039
20	0.050308	0.053012
21	0.043673	0.046378
22	0.035731	0.037802
23	0.030883	0.032749
24	0.087064	0.092381
25	0.061671	0.065087
26	0.050314	0.053358
27	0.043563	0.046162
28	0.035614	0.037785
29	0.030809	0.032609
30	0.086925	0.092143

31	0.061546	0.065109
32	0.050307	0.053376
33	0.043576	0.046201
34	0.035626	0.037589
35	0.030807	0.032815
36	0.086926	0.092213
37	0.061669	0.065165
38	0.050384	0.053355
39	0.043613	0.046318
40	0.035611	0.037701
41	0.030872	0.032840

	Cobertura_m_candidato_nom	Cobertura_m_candidato	\
0	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.881	
1	JOAQUIN_DIAZ_MENA	0.844	
2	RENAN_BARRERA_CONCHA	0.815	
3	RENAN_BARRERA_CONCHA	0.781	
4	RENAN_BARRERA_CONCHA	0.716	
5	RENAN_BARRERA_CONCHA	0.674	
6	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.892	
7	RENAN_BARRERA_CONCHA	0.888	
8	RENAN_BARRERA_CONCHA	0.843	
9	RENAN_BARRERA_CONCHA	0.814	
10	RENAN_BARRERA_CONCHA	0.773	
11	JOAQUIN_DIAZ_MENA	0.736	
12	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.891	
13	RENAN_BARRERA_CONCHA	0.890	
14	RENAN_BARRERA_CONCHA	0.880	
15	RENAN_BARRERA_CONCHA	0.844	
16	RENAN_BARRERA_CONCHA	0.794	
17	RENAN_BARRERA_CONCHA	0.771	
18	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.912	
19	JOAQUIN_DIAZ_MENA	0.895	
20	RENAN_BARRERA_CONCHA	0.878	
21	RENAN_BARRERA_CONCHA	0.886	
22	RENAN_BARRERA_CONCHA	0.852	
23	RENAN_BARRERA_CONCHA	0.827	
24	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.888	
25	JOAQUIN_DIAZ_MENA	0.914	
26	RENAN_BARRERA_CONCHA	0.889	
27	JOAQUIN_DIAZ_MENA	0.900	
28	RENAN_BARRERA_CONCHA	0.854	
29	RENAN_BARRERA_CONCHA	0.834	
30	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.905	
31	RENAN_BARRERA_CONCHA	0.921	
32	RENAN_BARRERA_CONCHA	0.900	
33	RENAN_BARRERA_CONCHA	0.898	

34	RENAN_BARRERA_CONCHA	0.867
35	RENAN_BARRERA_CONCHA	0.839
36	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.907
37	RENAN_BARRERA_CONCHA	0.916
38	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.909
39	JOAQUIN_DIAZ_MENA	0.894
40	JOAQUIN_DIAZ_MENA	0.897
41	JOAQUIN_DIAZ_MENA	0.864

	Cobertura_m_inter_q_025	Cobertura_m_inter_mediana	\
0	0.000000	0.011118	
1	0.058127	0.061491	
2	0.046686	0.049606	
3	0.040456	0.043119	
4	0.033201	0.035232	
5	0.028797	0.030490	
6	0.000000	0.011123	
7	0.057133	0.061015	
8	0.046862	0.049682	
9	0.040614	0.043115	
10	0.033255	0.035143	
11	0.029051	0.030848	
12	0.000000	0.011214	
13	0.057527	0.060661	
14	0.046989	0.049731	
15	0.040616	0.043029	
16	0.033253	0.035222	
17	0.028707	0.030567	
18	0.000000	0.011212	
19	0.058169	0.061633	
20	0.046789	0.049667	
21	0.040485	0.043085	
22	0.033181	0.035219	
23	0.028637	0.030427	
24	0.000000	0.011137	
25	0.057898	0.061671	
26	0.046782	0.049693	
27	0.041066	0.043563	
28	0.033001	0.035163	
29	0.028734	0.030435	
30	0.000000	0.011519	
31	0.057232	0.060806	
32	0.046687	0.049649	
33	0.040638	0.043050	
34	0.033239	0.035158	
35	0.028770	0.030471	
36	0.000000	0.011494	

37	0.057378	0.060910
38	0.004240	0.007075
39	0.041141	0.043613
40	0.033549	0.035611
41	0.029125	0.030872

	Cobertura_m_inter_q_975	Cob_JOAQUIN_DIAZ_MENA	Cob_RENAN_BARRERA_CONCHA	\
0	0.019452	0.905	0.907	
1	0.065346	0.844	0.845	
2	0.052836	0.831	0.815	
3	0.045656	0.784	0.781	
4	0.037317	0.738	0.716	
5	0.032241	0.700	0.674	
6	0.019556	0.909	0.907	
7	0.064654	0.890	0.888	
8	0.052786	0.856	0.843	
9	0.045963	0.831	0.814	
10	0.037501	0.782	0.773	
11	0.032720	0.736	0.741	
12	0.019519	0.932	0.937	
13	0.064408	0.891	0.890	
14	0.052700	0.882	0.880	
15	0.045583	0.863	0.844	
16	0.037369	0.821	0.794	
17	0.032369	0.795	0.771	
18	0.018610	0.931	0.925	
19	0.065039	0.895	0.913	
20	0.052706	0.896	0.878	
21	0.045741	0.892	0.886	
22	0.037400	0.860	0.852	
23	0.032328	0.844	0.827	
24	0.019116	0.935	0.935	
25	0.065087	0.914	0.914	
26	0.052525	0.897	0.889	
27	0.046162	0.900	0.904	
28	0.037254	0.867	0.854	
29	0.032204	0.848	0.834	
30	0.019297	0.939	0.941	
31	0.064603	0.927	0.921	
32	0.052698	0.919	0.900	
33	0.045669	0.912	0.898	
34	0.037384	0.875	0.867	
35	0.032450	0.856	0.839	
36	0.019454	0.937	0.932	
37	0.064779	0.933	0.916	
38	0.009867	0.930	0.925	
39	0.046318	0.894	0.899	

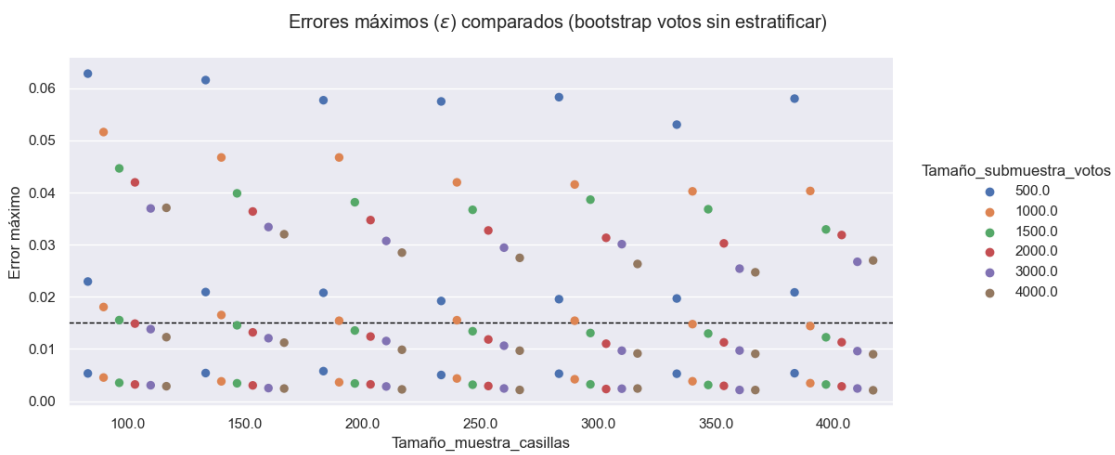
40	0.037701	0.897	0.907
41	0.032840	0.864	0.867

	Cob_VIDA_ARAVARI_GOMEZ_HERRERA	Cob_VOTOS_NULOS_CAND_NO_REGIS	\
0	0.920	0.933	
1	0.899	0.942	
2	0.864	0.919	
3	0.837	0.912	
4	0.804	0.906	
5	0.748	0.892	
6	0.925	0.938	
7	0.930	0.922	
8	0.905	0.925	
9	0.885	0.930	
10	0.849	0.934	
11	0.818	0.908	
12	0.932	0.936	
13	0.920	0.937	
14	0.911	0.937	
15	0.901	0.924	
16	0.873	0.926	
17	0.830	0.899	
18	0.924	0.942	
19	0.919	0.936	
20	0.913	0.942	
21	0.906	0.926	
22	0.886	0.923	
23	0.858	0.915	
24	0.919	0.940	
25	0.938	0.950	
26	0.912	0.940	
27	0.925	0.949	
28	0.906	0.932	
29	0.878	0.923	
30	0.937	0.927	
31	0.949	0.943	
32	0.918	0.952	
33	0.912	0.941	
34	0.902	0.924	
35	0.891	0.945	
36	0.942	0.927	
37	0.926	0.940	
38	0.933	0.924	
39	0.926	0.939	
40	0.915	0.939	
41	0.889	0.928	

```
    Cob_YAMIL_JASMIN_LOPEZ_MANRIQUE
0                             0.881
1                             0.923
2                             0.885
3                             0.857
4                             0.837
5                             0.811
6                             0.892
7                             0.924
8                             0.901
9                             0.893
10                            0.869
11                            0.852
12                            0.891
13                            0.926
14                            0.905
15                            0.905
16                            0.869
17                            0.881
18                            0.912
19                            0.916
20                            0.910
21                            0.888
22                            0.901
23                            0.888
24                            0.888
25                            0.925
26                            0.921
27                            0.910
28                            0.907
29                            0.894
30                            0.905
31                            0.921
32                            0.919
33                            0.907
34                            0.926
35                            0.910
36                            0.907
37                            0.941
38                            0.909
39                            0.910
40                            0.924
41                            0.914
```

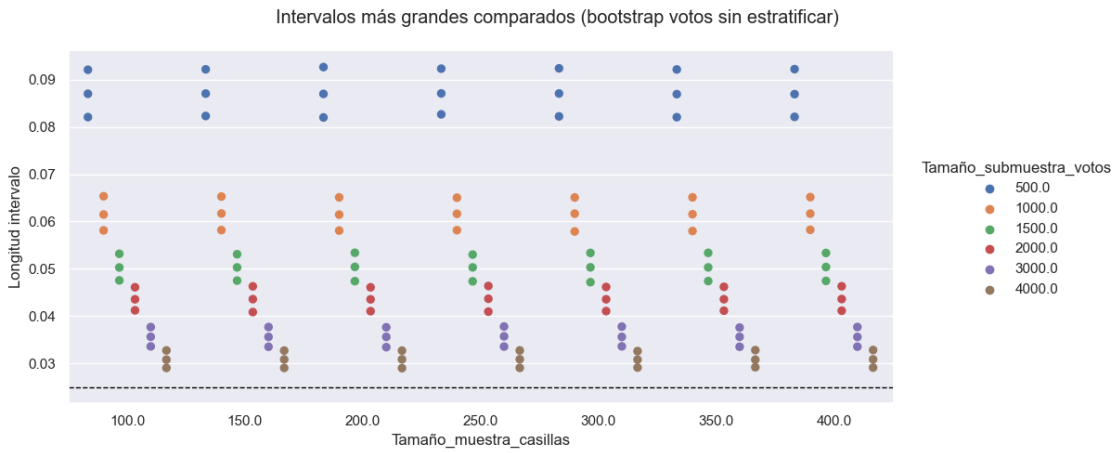
[265]:
```
# Para hacer la gráfica del error con los distintos tamaños de la submuestra de␣
↪votos
```

```
df_grafica_error_aux=pd.melt(df_boostraps_votos_se[["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos", "Error_maximo_q_025", "Error_maximo_mediana",␣
↪"Error_maximo_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos"], value_vars=["Error_maximo_q_025",␣
↪"Error_maximo_mediana", "Error_maximo_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per=sns.catplot(data=df_grafica_error_aux, x="Tamaño_muestra_casillas",␣
↪y="value", hue="Tamaño_submuestra_votos", kind='strip', aspect=2/1,␣
↪palette="deep", jitter=False, dodge=True, s = 45);


per.set(xlabel='Tamaño_muestra_casillas', ylabel='Error máximo')
per.figure.subplots_adjust(top=0.9)
per.figure.suptitle('Errores máximos ($\epsilon$) comparados (bootstrap votos␣
↪sin estratificar)');
per.refline(y=0.015, linewidth=1, color='black', ls='--');
```

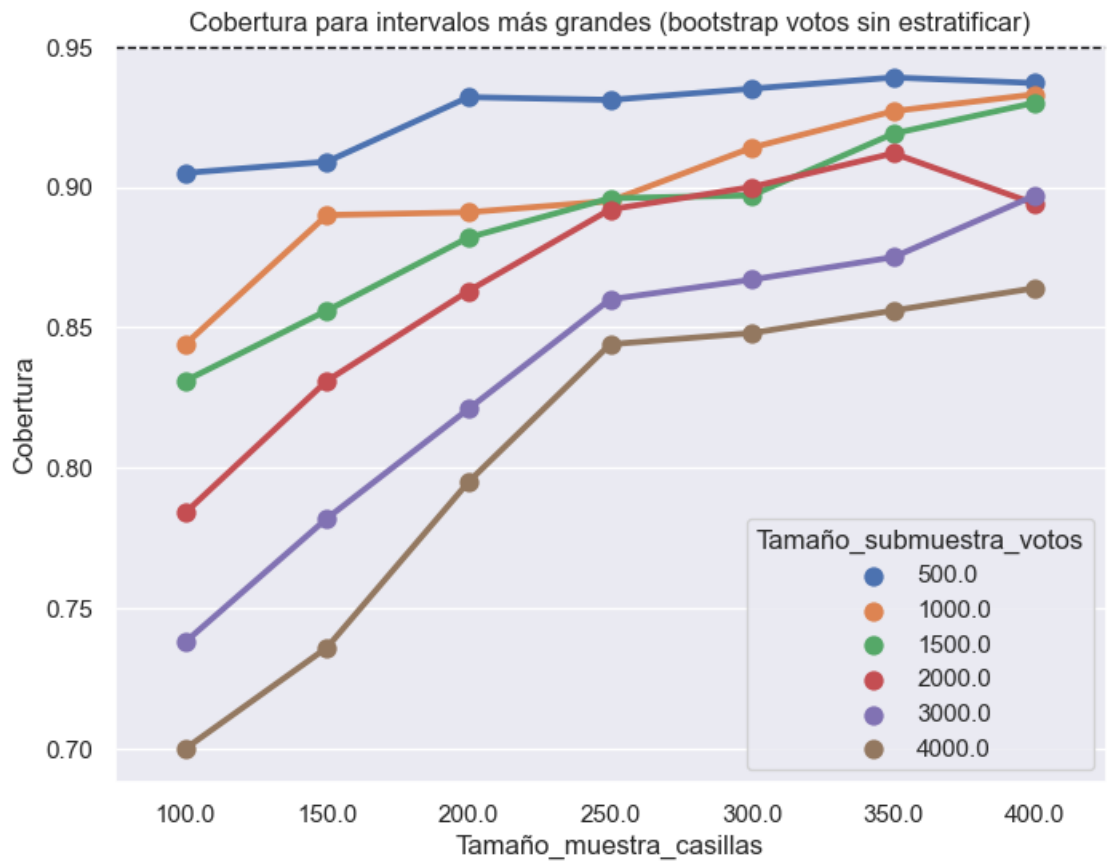


[267]:
```
# Para hacer la gráfica de longitud de intervalo (para el candidato con los␣
↪intervalos más grandes) con los distintos tamños de la submuestra de votos
df_grafica_int_im_aux=pd.melt(df_boostraps_votos_se[["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos", "Intervalo_M_grande_inter_q_025",␣
↪"Intervalo_M_grande_inter_mediana",␣
↪"Intervalo_M_grande_inter_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos"], value_vars=["Intervalo_M_grande_inter_q_025",␣
↪"Intervalo_M_grande_inter_mediana", "Intervalo_M_grande_inter_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)
```

```
per_i=sns.catplot(data=df_grafica_int_im_aux, x="Tamaño_muestra_casillas",␣
 ↪y="value", hue="Tamaño_submuestra_votos", kind='strip', aspect=2/1,␣
 ↪palette="deep", jitter=False, dodge=True, s = 45);

per_i.set(xlabel='Tamaño_muestra_casillas', ylabel='Longitud intervalo')
per_i.figure.subplots_adjust(top=0.9)
per_i.figure.suptitle('Intervalos más grandes comparados (bootstrap votos sin␣
 ↪estratificar)');
per_i.refline(y=0.025, linewidth=1, color='black', ls='--');
```

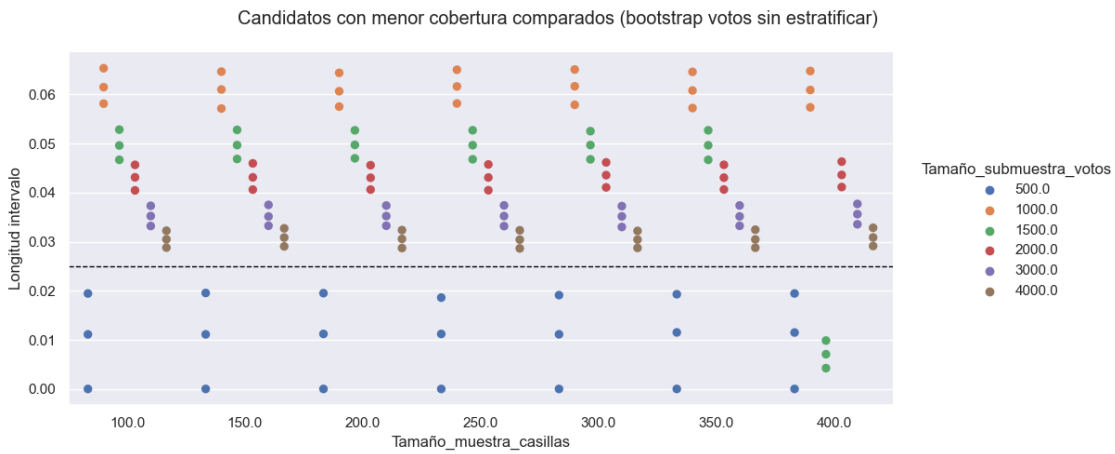


[269]:
```
fig, ax = plt.subplots(figsize=(8,6))

sns.pointplot(ax=ax,
    data=df_boostraps_votos_se, x="Tamaño_muestra_casillas",␣
 ↪y="Intervalo_M_grande_candidato_cob", hue="Tamaño_submuestra_votos"
).set_title("Cobertura para intervalos más grandes (bootstrap votos sin␣
 ↪estratificar)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
ax.set(ylabel='Cobertura');
```

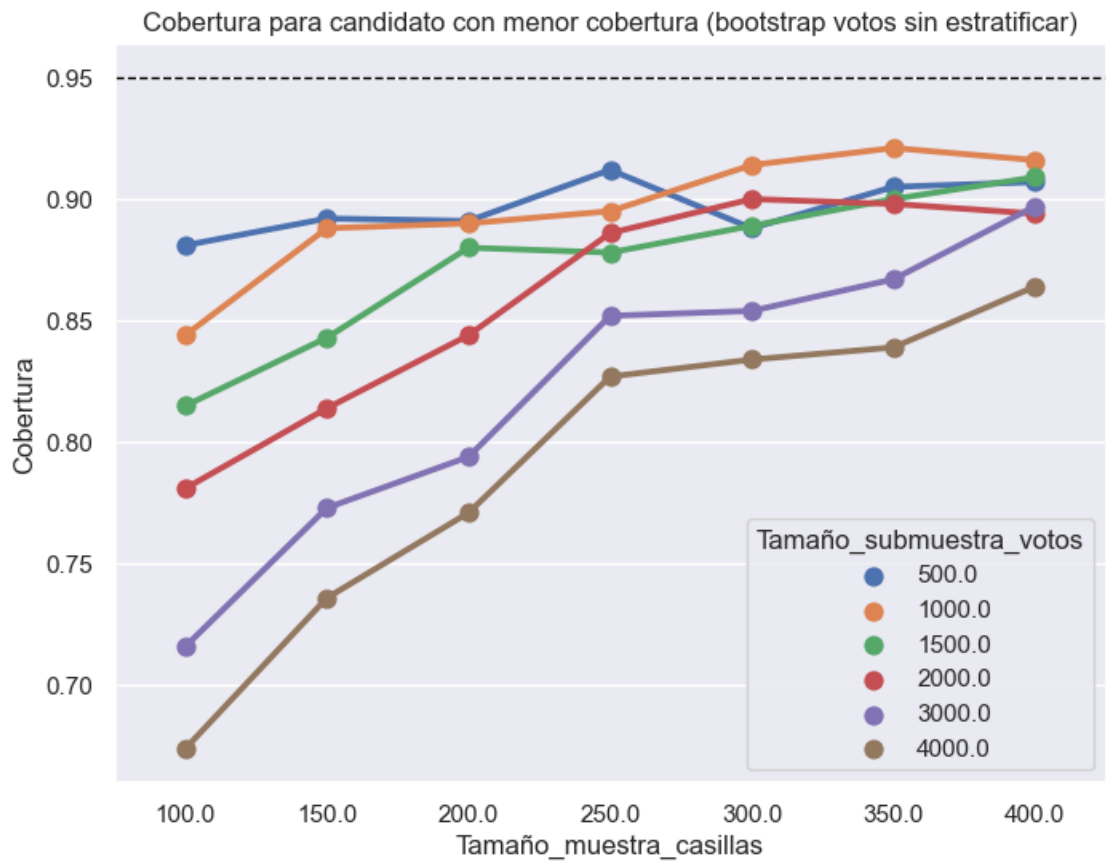


[271]:
```
# Para hacer la gráfica de longitud de intervalo (para el candidato con los␣
↪intervalos más grandes) con los distintos tamños de la submuestra de votos
df_grafica_int_mc_aux=pd.melt(df_boostraps_votos_se[["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos", "Cobertura_m_inter_q_025",␣
↪"Cobertura_m_inter_mediana",␣
↪"Cobertura_m_inter_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos"], value_vars=["Cobertura_m_inter_q_025",␣
↪"Cobertura_m_inter_mediana", "Cobertura_m_inter_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_mc=sns.catplot(data=df_grafica_int_mc_aux, x="Tamaño_muestra_casillas",␣
↪y="value", hue="Tamaño_submuestra_votos", kind='strip', aspect=2/1,␣
↪palette="deep", jitter=False, dodge=True, s = 45);

per_mc.set(xlabel='Tamaño_muestra_casillas', ylabel='Longitud intervalo')
per_mc.figure.subplots_adjust(top=0.9)
```

```
per_mc.figure.suptitle('Candidatos con menor cobertura comparados (bootstrap␣
↪votos sin estratificar)');
per_mc.refline(y=0.025, linewidth=1, color='black', ls='--');
```



[273]:
```
fig, ax = plt.subplots(figsize=(8,6))

sns.pointplot(ax=ax, data=df_boostraps_votos_se, x="Tamaño_muestra_casillas",␣
↪y="Cobertura_m_candidato", hue="Tamaño_submuestra_votos").
↪set_title("Cobertura para candidato con menor cobertura (bootstrap votos sin␣
↪estratificar)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
ax.set(ylabel='Cobertura');
```



3.2.2 Estratificado (por distrito)

[123]:
```
# Probamos distintos valores de tamaño de la muestra de las casillas y la␣
↪submuestra de votos
muestra_casillas = [100, 150, 200, 250, 300, 350, 400]
submuestra_votos = [500, 1000, 1500, 2000, 3000, 4000]

# Dataframe donde se guardará todo
df_boostraps_votos_ec=pd.DataFrame()

# Contador para el for
k=0

# Hacemos los for sobre el tamaño de las muestras y submuestras
for mc in muestra_casillas:
    for smv in submuestra_votos:
        # Para medir cuanto tiempo se tarda en correr
        loop_time = time.time()
        # Boostrap por votos (estratificado)
```

```
        inter_prob_v_est, est_punt_v_est=boot_votos_estratificado(df_act_yuc_i,␣
↪num_boot=1000, n_sample=mc, n_vot_sub=smv)

        # Calculamos las métricas
        df_metr_boot_v_est, df_error_max_v_est, df_can_cob_v_est =␣
↪metricas_bootstrap(prop_reales, inter_prob_v_est, est_punt_v_est,␣
↪porcenta_tama=mc)

        # Los valores de las muestra y submuestras
        df_boostraps_votos_ec.
↪loc[k,["Tamaño_muestra_casillas","Tamaño_submuestra_votos"]]=[mc,smv]

        # Las métricas para el error máximo
        df_boostraps_votos_ec.
↪loc[k,["Error_maximo_q_025","Error_maximo_mediana","Error_maximo_q_975"]]=df_error_max_v_es
↪quantile([0.025,0.5,0.975]).to_list()

        # Calculamos el candidato que tenga intervalos más largos (en promedio)
        candidato_int_est_m_l=df_metr_boot_v_est.groupby(["Candidato"]).
↪agg({'Longitud_intervalo':'mean'}).reset_index()["Candidato"].values[0]
        # Candidato con el intervalos más grandes
        df_boostraps_votos_ec.loc[k,␣
↪"Intervalo_M_grande_candidato_nom"]=candidato_int_est_m_l
        # Cobertura del candidato con el intervalos más grandes
        df_boostraps_votos_ec.loc[k,␣
↪["Intervalo_M_grande_candidato_cob"]]=df_can_cob_v_est[df_can_cob_v_est["Candidato"]==candi
↪values[0]
        # Las métricas del candidato con los intervalos más grandes
        df_boostraps_votos_ec.
↪loc[k,["Intervalo_M_grande_inter_q_025","Intervalo_M_grande_inter_mediana","Intervalo_M_gra
↪quantile([0.025,0.5,0.975]).to_list()

        # Calculamos el candidato con la menor cobertura
        # Candidato con menor cobertura
       ␣
↪candidato_cober_est_men_v=df_can_cob_v_est[df_can_cob_v_est["Cobertura"]==df_can_cob_v_est[
↪min()]["Candidato"].values[0]

        # Candidato con el intervalos más grandes
        df_boostraps_votos_ec.loc[k,␣
↪"Cobertura_m_candidato_nom"]=candidato_cober_est_men_v
        # Cobertura del candidato con menor cobertura
        df_boostraps_votos_ec.loc[k,␣
↪["Cobertura_m_candidato"]]=df_can_cob_v_est[df_can_cob_v_est["Candidato"]==candidato_cober_
↪values[0]
        # Las métricas del candidato con la menor cobertura
```

```
        df_boostraps_votos_ec.
↪loc[k,["Cobertura_m_inter_q_025","Cobertura_m_inter_mediana","Cobertura_m_inter_q_975"]]=df
↪quantile([0.025,0.5,0.975]).to_list()


        # Coberturas de los candidatos en el bootstrap
        df_boostraps_votos_ec.loc[k,␣
↪["Cob_JOAQUIN_DIAZ_MENA","Cob_RENAN_BARRERA_CONCHA","Cob_VIDA_ARAVARI_GOMEZ_HERRERA","Cob_V
↪sort_values(by="Candidato")["Cobertura"].to_list()

        k=k+1

        print("Muestra casillas:", mc, "Submuestra votos:", smv)
        print('Tiempo que tarda correr una iteración del for {}'.format((time.
↪time() - loop_time)))

# Lo guardamos en otra base copia por si sin querer pongo a correr el código
df_boostraps_votos_ec_f=df_boostraps_votos_ec.copy()
```

```
100
Muestra casillas: 100 Submuestra votos: 500
Tiempo que tarda correr una iteración del for 117.71354818344116
100
Muestra casillas: 100 Submuestra votos: 1000
Tiempo que tarda correr una iteración del for 117.87102794647217
100
Muestra casillas: 100 Submuestra votos: 1500
Tiempo que tarda correr una iteración del for 117.51008582115173
100
Muestra casillas: 100 Submuestra votos: 2000
Tiempo que tarda correr una iteración del for 117.62462997436523
100
Muestra casillas: 100 Submuestra votos: 3000
Tiempo que tarda correr una iteración del for 117.83214712142944
100
Muestra casillas: 100 Submuestra votos: 4000
Tiempo que tarda correr una iteración del for 434.48600482940674
150
Muestra casillas: 150 Submuestra votos: 500
Tiempo que tarda correr una iteración del for 117.98713207244873
150
Muestra casillas: 150 Submuestra votos: 1000
Tiempo que tarda correr una iteración del for 119.52670001983643
150
Muestra casillas: 150 Submuestra votos: 1500
Tiempo que tarda correr una iteración del for 124.20759296417236
150
```

```
Muestra casillas: 150 Submuestra votos: 2000
Tiempo que tarda correr una iteración del for 120.9193959236145
150
Muestra casillas: 150 Submuestra votos: 3000
Tiempo que tarda correr una iteración del for 119.81950402259827
150
Muestra casillas: 150 Submuestra votos: 4000
Tiempo que tarda correr una iteración del for 121.4541540145874
200
Muestra casillas: 200 Submuestra votos: 500
Tiempo que tarda correr una iteración del for 117.92127203941345
200
Muestra casillas: 200 Submuestra votos: 1000
Tiempo que tarda correr una iteración del for 122.36299180984497
200
Muestra casillas: 200 Submuestra votos: 1500
Tiempo que tarda correr una iteración del for 127.16438794136047
200
Muestra casillas: 200 Submuestra votos: 2000
Tiempo que tarda correr una iteración del for 121.91371607780457
200
Muestra casillas: 200 Submuestra votos: 3000
Tiempo que tarda correr una iteración del for 120.60905289649963
200
Muestra casillas: 200 Submuestra votos: 4000
Tiempo que tarda correr una iteración del for 121.41330409049988
250
Muestra casillas: 250 Submuestra votos: 500
Tiempo que tarda correr una iteración del for 119.80784392356873
250
Muestra casillas: 250 Submuestra votos: 1000
Tiempo que tarda correr una iteración del for 120.72400784492493
250
Muestra casillas: 250 Submuestra votos: 1500
Tiempo que tarda correr una iteración del for 122.55476880073547
250
Muestra casillas: 250 Submuestra votos: 2000
Tiempo que tarda correr una iteración del for 120.64412903785706
250
Muestra casillas: 250 Submuestra votos: 3000
Tiempo que tarda correr una iteración del for 120.65024304389954
250
Muestra casillas: 250 Submuestra votos: 4000
Tiempo que tarda correr una iteración del for 121.84681224822998
300
Muestra casillas: 300 Submuestra votos: 500
Tiempo que tarda correr una iteración del for 118.51657009124756
300
```

```
Muestra casillas: 300 Submuestra votos: 1000
Tiempo que tarda correr una iteración del for 118.81075930595398
300
Muestra casillas: 300 Submuestra votos: 1500
Tiempo que tarda correr una iteración del for 119.34661817550659
300
Muestra casillas: 300 Submuestra votos: 2000
Tiempo que tarda correr una iteración del for 120.96960091590881
300
Muestra casillas: 300 Submuestra votos: 3000
Tiempo que tarda correr una iteración del for 120.99717497825623
300
Muestra casillas: 300 Submuestra votos: 4000
Tiempo que tarda correr una iteración del for 127.7669107913971
350
Muestra casillas: 350 Submuestra votos: 500
Tiempo que tarda correr una iteración del for 119.46384906768799
350
Muestra casillas: 350 Submuestra votos: 1000
Tiempo que tarda correr una iteración del for 123.20504307746887
350
Muestra casillas: 350 Submuestra votos: 1500
Tiempo que tarda correr una iteración del for 124.60997676849365
350
Muestra casillas: 350 Submuestra votos: 2000
Tiempo que tarda correr una iteración del for 121.84983086585999
350
Muestra casillas: 350 Submuestra votos: 3000
Tiempo que tarda correr una iteración del for 120.73050427436829
350
Muestra casillas: 350 Submuestra votos: 4000
Tiempo que tarda correr una iteración del for 120.08476781845093
400
Muestra casillas: 400 Submuestra votos: 500
Tiempo que tarda correr una iteración del for 117.921217918396
400
Muestra casillas: 400 Submuestra votos: 1000
Tiempo que tarda correr una iteración del for 118.7878258228302
400
Muestra casillas: 400 Submuestra votos: 1500
Tiempo que tarda correr una iteración del for 119.83775591850281
400
Muestra casillas: 400 Submuestra votos: 2000
Tiempo que tarda correr una iteración del for 121.85913491249084
400
Muestra casillas: 400 Submuestra votos: 3000
Tiempo que tarda correr una iteración del for 120.43954205513
400
```

```
Muestra casillas: 400 Submuestra votos: 4000
Tiempo que tarda correr una iteración del for 120.17651987075806
```

[131]:
```
# Guardamos la base en un csv
#df_boostraps_votos_ec_f.to_csv("/Users/damian/Documents/Tesis maestria/
 ↪Bootstraps/Bases/Bootstrap votos/
 ↪Distintas_muestras_submuestras_bootstrap_votos_estratificado_e_3.csv")
```

[237]:
```
# Leemos la base en el csv
df_boostraps_votos_ec_f=pd.read_csv("/Users/damian/Documents/Tesis maestria/
 ↪Bootstraps/Bases/Bootstrap votos/
 ↪Distintas_muestras_submuestras_bootstrap_votos_estratificado_e_3.csv",␣
 ↪index_col=0)
# Quitamos la primer columna
#df_boostraps_votos_ec_f=df_boostraps_votos_ec_f.drop(df_boostraps_votos_ec_f.
 ↪columns[0], axis=1)
df_boostraps_votos_ec_f
```

[237]:

	Tamaño_muestra_casillas	Tamaño_submuestra_votos	Error_maximo_q_025	\
0	100.0	500.0	0.005224	
1	100.0	1000.0	0.003589	
2	100.0	1500.0	0.003411	
3	100.0	2000.0	0.002992	
4	100.0	3000.0	0.002796	
5	100.0	4000.0	0.002315	
6	150.0	500.0	0.005577	
7	150.0	1000.0	0.003999	
8	150.0	1500.0	0.003412	
9	150.0	2000.0	0.003202	
10	150.0	3000.0	0.002620	
11	150.0	4000.0	0.002455	
12	200.0	500.0	0.005005	
13	200.0	1000.0	0.003760	
14	200.0	1500.0	0.003381	
15	200.0	2000.0	0.002406	
16	200.0	3000.0	0.002472	
17	200.0	4000.0	0.002213	
18	250.0	500.0	0.004752	
19	250.0	1000.0	0.003732	
20	250.0	1500.0	0.003127	
21	250.0	2000.0	0.002293	
22	250.0	3000.0	0.002401	
23	250.0	4000.0	0.002287	
24	300.0	500.0	0.004979	
25	300.0	1000.0	0.003500	
26	300.0	1500.0	0.002951	
27	300.0	2000.0	0.002765	

28	300.0	3000.0	0.002074
29	300.0	4000.0	0.002278
30	350.0	500.0	0.005140
31	350.0	1000.0	0.003459
32	350.0	1500.0	0.003081
33	350.0	2000.0	0.002438
34	350.0	3000.0	0.002208
35	350.0	4000.0	0.001865
36	400.0	500.0	0.005094
37	400.0	1000.0	0.003499
38	400.0	1500.0	0.002786
39	400.0	2000.0	0.002762
40	400.0	3000.0	0.002283
41	400.0	4000.0	0.002006

	Error_maximo_mediana	Error_maximo_q_975	Intervalo_M_grande_candidato_nom \
0	0.020914	0.054767	JOAQUIN_DIAZ_MENA
1	0.016246	0.041720	JOAQUIN_DIAZ_MENA
2	0.013748	0.039078	JOAQUIN_DIAZ_MENA
3	0.013061	0.033661	JOAQUIN_DIAZ_MENA
4	0.011646	0.031995	JOAQUIN_DIAZ_MENA
5	0.010649	0.028842	JOAQUIN_DIAZ_MENA
6	0.021123	0.055494	JOAQUIN_DIAZ_MENA
7	0.015438	0.042980	JOAQUIN_DIAZ_MENA
8	0.012813	0.037242	JOAQUIN_DIAZ_MENA
9	0.012156	0.032395	JOAQUIN_DIAZ_MENA
10	0.010291	0.027803	JOAQUIN_DIAZ_MENA
11	0.009913	0.026215	JOAQUIN_DIAZ_MENA
12	0.020565	0.056468	JOAQUIN_DIAZ_MENA
13	0.014349	0.039393	JOAQUIN_DIAZ_MENA
14	0.012657	0.034159	JOAQUIN_DIAZ_MENA
15	0.010889	0.030407	JOAQUIN_DIAZ_MENA
16	0.009771	0.026736	JOAQUIN_DIAZ_MENA
17	0.008741	0.024181	JOAQUIN_DIAZ_MENA
18	0.019928	0.052234	JOAQUIN_DIAZ_MENA
19	0.014011	0.039500	JOAQUIN_DIAZ_MENA
20	0.012267	0.034981	JOAQUIN_DIAZ_MENA
21	0.011114	0.030506	JOAQUIN_DIAZ_MENA
22	0.009644	0.025856	JOAQUIN_DIAZ_MENA
23	0.009059	0.023746	JOAQUIN_DIAZ_MENA
24	0.018005	0.056078	JOAQUIN_DIAZ_MENA
25	0.013803	0.039788	JOAQUIN_DIAZ_MENA
26	0.011871	0.033644	JOAQUIN_DIAZ_MENA
27	0.011065	0.027129	JOAQUIN_DIAZ_MENA
28	0.008756	0.023935	JOAQUIN_DIAZ_MENA
29	0.008024	0.022944	JOAQUIN_DIAZ_MENA
30	0.020699	0.055524	JOAQUIN_DIAZ_MENA

31	0.014373	0.039718	JOAQUIN_DIAZ_MENA
32	0.011960	0.031115	JOAQUIN_DIAZ_MENA
33	0.010513	0.029329	JOAQUIN_DIAZ_MENA
34	0.008821	0.026122	JOAQUIN_DIAZ_MENA
35	0.007908	0.022585	JOAQUIN_DIAZ_MENA
36	0.018702	0.053000	JOAQUIN_DIAZ_MENA
37	0.014076	0.038654	JOAQUIN_DIAZ_MENA
38	0.011666	0.032697	JOAQUIN_DIAZ_MENA
39	0.010269	0.028049	JOAQUIN_DIAZ_MENA
40	0.008916	0.025229	JOAQUIN_DIAZ_MENA
41	0.007737	0.020432	JOAQUIN_DIAZ_MENA

	Intervalo_M_grande_candidato_cob	Intervalo_M_grande_inter_q_025 \
0	0.924	0.076358
1	0.887	0.055619
2	0.870	0.045693
3	0.841	0.039763
4	0.817	0.032640
5	0.795	0.028249
6	0.918	0.076883
7	0.898	0.055365
8	0.897	0.045911
9	0.881	0.040086
10	0.851	0.032704
11	0.841	0.028395
12	0.924	0.077090
13	0.924	0.055694
14	0.899	0.045717
15	0.894	0.039842
16	0.872	0.032719
17	0.867	0.028493
18	0.932	0.077371
19	0.927	0.055972
20	0.918	0.045823
21	0.902	0.039967
22	0.901	0.032758
23	0.858	0.028459
24	0.933	0.076779
25	0.921	0.055922
26	0.917	0.045952
27	0.919	0.040061
28	0.910	0.032806
29	0.895	0.028581
30	0.933	0.077032
31	0.932	0.056014
32	0.929	0.045899
33	0.921	0.039931

34	0.908	0.032662
35	0.906	0.028281
36	0.945	0.076987
37	0.930	0.055920
38	0.917	0.045840
39	0.921	0.039959
40	0.903	0.032830
41	0.918	0.028576

	Intervalo_M_grande_inter_mediana	Intervalo_M_grande_inter_q_975 \
0	0.081740	0.087228
1	0.059236	0.062937
2	0.048666	0.051772
3	0.042330	0.044892
4	0.034654	0.036788
5	0.030085	0.032017
6	0.081919	0.087109
7	0.059242	0.063135
8	0.048692	0.051822
9	0.042290	0.044889
10	0.034655	0.036672
11	0.030143	0.032057
12	0.081910	0.086973
13	0.059253	0.063148
14	0.048703	0.051764
15	0.042394	0.045065
16	0.034731	0.036972
17	0.030050	0.031976
18	0.082003	0.087159
19	0.059433	0.063054
20	0.048854	0.051780
21	0.042277	0.044981
22	0.034772	0.036963
23	0.030190	0.032110
24	0.081957	0.087295
25	0.059235	0.062966
26	0.048774	0.051860
27	0.042331	0.044931
28	0.034762	0.036852
29	0.030223	0.032206
30	0.082129	0.087307
31	0.059361	0.063020
32	0.048639	0.051579
33	0.042283	0.044818
34	0.034758	0.036844
35	0.030178	0.031979
36	0.082233	0.087256

37	0.059344	0.063085
38	0.048772	0.051897
39	0.042474	0.045017
40	0.034817	0.036956
41	0.030196	0.032110

	Cobertura_m_candidato_nom	Cobertura_m_candidato	\
0	VIDA_ARAVARI_GOMEZ_HERRERA	0.891	
1	RENAN_BARRERA_CONCHA	0.882	
2	RENAN_BARRERA_CONCHA	0.863	
3	RENAN_BARRERA_CONCHA	0.827	
4	RENAN_BARRERA_CONCHA	0.783	
5	RENAN_BARRERA_CONCHA	0.765	
6	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.861	
7	RENAN_BARRERA_CONCHA	0.896	
8	RENAN_BARRERA_CONCHA	0.894	
9	RENAN_BARRERA_CONCHA	0.857	
10	RENAN_BARRERA_CONCHA	0.838	
11	RENAN_BARRERA_CONCHA	0.816	
12	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.903	
13	VIDA_ARAVARI_GOMEZ_HERRERA	0.910	
14	RENAN_BARRERA_CONCHA	0.895	
15	JOAQUIN_DIAZ_MENA	0.894	
16	RENAN_BARRERA_CONCHA	0.851	
17	VIDA_ARAVARI_GOMEZ_HERRERA	0.849	
18	VOTOS_NULOS_CAND_NO_REGIS	0.924	
19	JOAQUIN_DIAZ_MENA	0.927	
20	RENAN_BARRERA_CONCHA	0.917	
21	JOAQUIN_DIAZ_MENA	0.902	
22	RENAN_BARRERA_CONCHA	0.867	
23	RENAN_BARRERA_CONCHA	0.845	
24	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.903	
25	JOAQUIN_DIAZ_MENA	0.921	
26	RENAN_BARRERA_CONCHA	0.916	
27	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.915	
28	RENAN_BARRERA_CONCHA	0.892	
29	RENAN_BARRERA_CONCHA	0.877	
30	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.895	
31	RENAN_BARRERA_CONCHA	0.926	
32	RENAN_BARRERA_CONCHA	0.924	
33	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.913	
34	VIDA_ARAVARI_GOMEZ_HERRERA	0.888	
35	RENAN_BARRERA_CONCHA	0.878	
36	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.907	
37	RENAN_BARRERA_CONCHA	0.921	
38	JOAQUIN_DIAZ_MENA	0.917	
39	RENAN_BARRERA_CONCHA	0.917	

40	RENAN_BARRERA_CONCHA	0.899
41	YAMIL_JASMIN_LOPEZ_MANRIQUE	0.884

	Cobertura_m_inter_q_025	Cobertura_m_inter_mediana	\
0	0.023178	0.031241	
1	0.054702	0.058213	
2	0.045209	0.047868	
3	0.039164	0.041625	
4	0.032146	0.034135	
5	0.027894	0.029581	
6	0.000000	0.010719	
7	0.055047	0.058484	
8	0.045285	0.048034	
9	0.039330	0.041658	
10	0.032151	0.034217	
11	0.027982	0.029704	
12	0.000000	0.010711	
13	0.018805	0.022709	
14	0.044923	0.048062	
15	0.039842	0.042394	
16	0.032181	0.034289	
17	0.010431	0.011676	
18	0.015510	0.023938	
19	0.055972	0.059433	
20	0.045218	0.048112	
21	0.039967	0.042277	
22	0.032330	0.034251	
23	0.027968	0.029751	
24	0.000000	0.010810	
25	0.055922	0.059235	
26	0.045255	0.048081	
27	0.003924	0.006102	
28	0.032211	0.034227	
29	0.028089	0.029758	
30	0.000000	0.010900	
31	0.055264	0.058597	
32	0.045253	0.048023	
33	0.004225	0.006143	
34	0.011792	0.013388	
35	0.028039	0.029814	
36	0.000000	0.011681	
37	0.055312	0.058579	
38	0.045840	0.048772	
39	0.039463	0.041887	
40	0.032319	0.034368	
41	0.003244	0.004388	

	Cobertura_m_inter_q_975	Cob_JOAQUIN_DIAZ_MENA	Cob_RENAN_BARRERA_CONCHA	\
0	0.038323	0.924	0.909	
1	0.061971	0.887	0.882	
2	0.051035	0.870	0.863	
3	0.044283	0.841	0.827	
4	0.036129	0.817	0.783	
5	0.031480	0.795	0.765	
6	0.019321	0.918	0.917	
7	0.062342	0.898	0.896	
8	0.050948	0.897	0.894	
9	0.044157	0.881	0.857	
10	0.036144	0.851	0.838	
11	0.031558	0.841	0.816	
12	0.018729	0.924	0.918	
13	0.026865	0.924	0.921	
14	0.051110	0.899	0.895	
15	0.045065	0.894	0.900	
16	0.036227	0.872	0.851	
17	0.012898	0.867	0.858	
18	0.030681	0.932	0.928	
19	0.063054	0.927	0.934	
20	0.050986	0.918	0.917	
21	0.044981	0.902	0.902	
22	0.036345	0.901	0.867	
23	0.031684	0.858	0.845	
24	0.018790	0.933	0.920	
25	0.062966	0.921	0.930	
26	0.050977	0.917	0.916	
27	0.008141	0.919	0.921	
28	0.036241	0.910	0.892	
29	0.031608	0.895	0.877	
30	0.018821	0.933	0.924	
31	0.062114	0.932	0.926	
32	0.051088	0.929	0.924	
33	0.008294	0.921	0.921	
34	0.014903	0.908	0.906	
35	0.031619	0.906	0.878	
36	0.018864	0.945	0.938	
37	0.062347	0.930	0.921	
38	0.051897	0.917	0.919	
39	0.044413	0.921	0.917	
40	0.036371	0.903	0.899	
41	0.005524	0.918	0.901	

	Cob_VIDA_ARAVARI_GOMEZ_HERRERA	Cob_VOTOS_NULOS_CAND_NO_REGIS	\
0	0.891	0.932	
1	0.897	0.921	

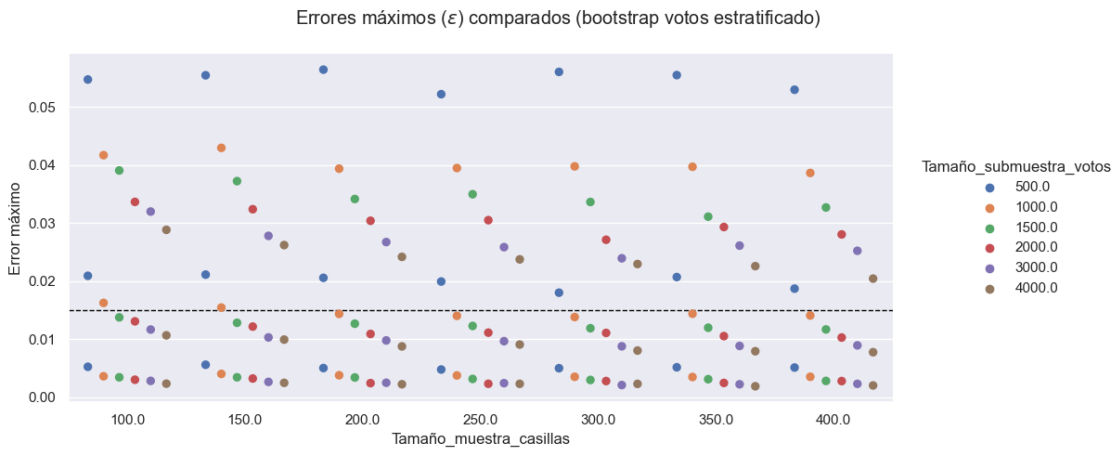
2	0.892	0.922
3	0.874	0.892
4	0.836	0.915
5	0.822	0.899
6	0.928	0.938
7	0.916	0.939
8	0.895	0.936
9	0.900	0.919
10	0.857	0.908
11	0.833	0.921
12	0.932	0.944
13	0.910	0.929
14	0.922	0.937
15	0.897	0.935
16	0.873	0.934
17	0.849	0.918
18	0.927	0.924
19	0.931	0.944
20	0.929	0.929
21	0.922	0.930
22	0.904	0.935
23	0.888	0.926
24	0.923	0.935
25	0.934	0.933
26	0.916	0.936
27	0.916	0.940
28	0.894	0.922
29	0.897	0.939
30	0.924	0.928
31	0.928	0.929
32	0.925	0.945
33	0.918	0.915
34	0.888	0.951
35	0.887	0.929
36	0.927	0.924
37	0.933	0.926
38	0.937	0.940
39	0.921	0.935
40	0.925	0.930
41	0.901	0.935

	Cob_YAMIL_JASMIN_LOPEZ_MANRIQUE
0	0.896
1	0.893
2	0.875
3	0.872
4	0.847

```
5                                0.841
6                                0.861
7                                0.930
8                                0.902
9                                0.889
10                               0.883
11                               0.856
12                               0.903
13                               0.921
14                               0.920
15                               0.906
16                               0.889
17                               0.889
18                               0.924
19                               0.929
20                               0.927
21                               0.909
22                               0.921
23                               0.904
24                               0.903
25                               0.928
26                               0.922
27                               0.915
28                               0.897
29                               0.905
30                               0.895
31                               0.933
32                               0.928
33                               0.913
34                               0.920
35                               0.912
36                               0.907
37                               0.934
38                               0.928
39                               0.927
40                               0.914
41                               0.884
```

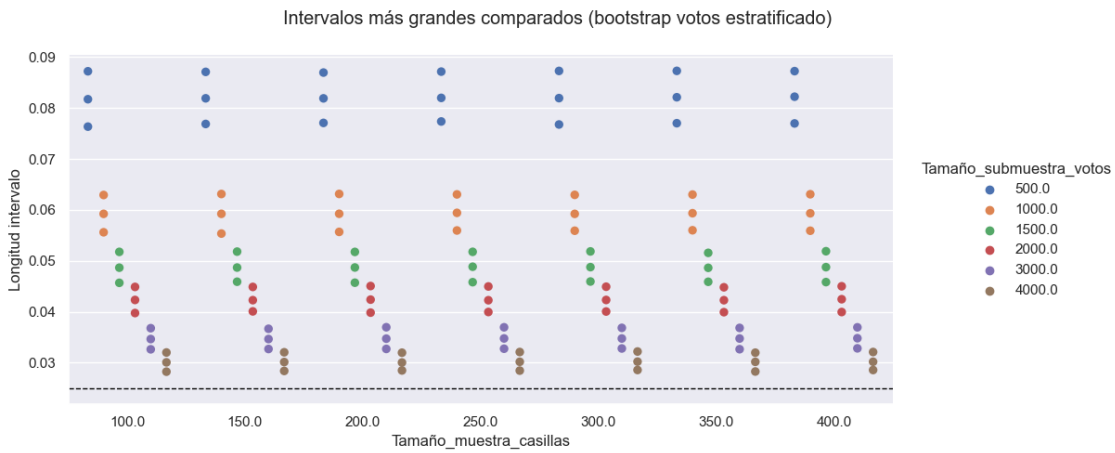
[274]:
```
# Para hacer la gráfica del error con los distintos tamaños de la submuestra de␣
↪votos
df_grafica_error_aux_est=pd.
↪melt(df_boostraps_votos_ec_f[["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos", "Error_maximo_q_025", "Error_maximo_mediana",␣
↪"Error_maximo_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos"], value_vars=["Error_maximo_q_025",␣
↪"Error_maximo_mediana", "Error_maximo_q_975"])
```

```
# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_e=sns.catplot(data=df_grafica_error_aux_est, x="Tamaño_muestra_casillas",␣
↪y="value", hue="Tamaño_submuestra_votos", kind='strip', aspect=2/1,␣
↪palette="deep", jitter=False, dodge=True, s = 45);


per_e.set(xlabel='Tamaño_muestra_casillas', ylabel='Error máximo')
per_e.figure.subplots_adjust(top=0.9)
per_e.figure.suptitle('Errores máximos ($\epsilon$) comparados (bootstrap votos␣
↪estratificado)');
per_e.refline(y=0.015, linewidth=1, color='black', ls='--');
```

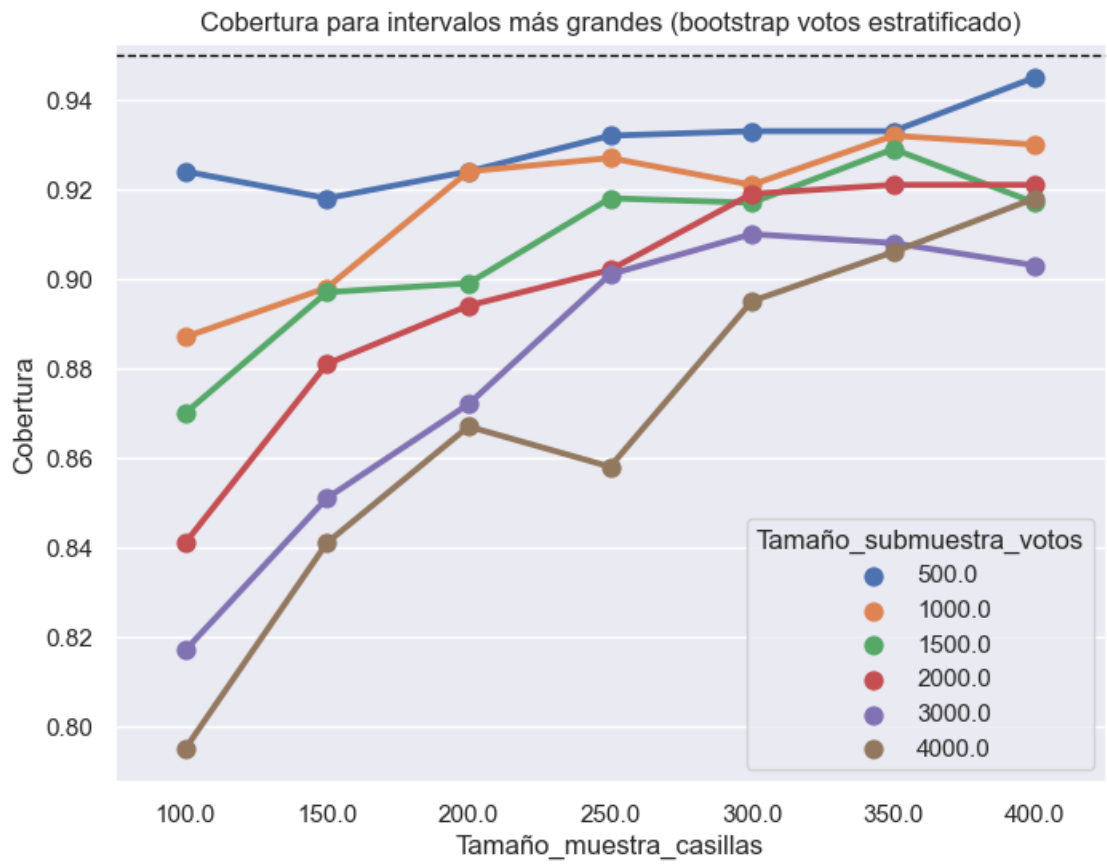


[275]:
```
# Para hacer la gráfica de longitud de intervalo (para el candidato con los␣
↪intervalos más grandes) con los distintos tamños de la submuestra de votos
df_grafica_int_im_aux_est=pd.
↪melt(df_boostraps_votos_ec_f[["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos", "Intervalo_M_grande_inter_q_025",␣
↪"Intervalo_M_grande_inter_mediana",␣
↪"Intervalo_M_grande_inter_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos"], value_vars=["Intervalo_M_grande_inter_q_025",␣
↪"Intervalo_M_grande_inter_mediana", "Intervalo_M_grande_inter_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_i_e=sns.catplot(data=df_grafica_int_im_aux_est,␣
↪x="Tamaño_muestra_casillas", y="value", hue="Tamaño_submuestra_votos",␣
↪kind='strip', aspect=2/1, palette="deep", jitter=False, dodge=True, s = 45);
```

```
per_i_e.set(xlabel='Tamaño_muestra_casillas', ylabel='Longitud intervalo')
per_i_e.figure.subplots_adjust(top=0.9)
per_i_e.figure.suptitle('Intervalos más grandes comparados (bootstrap votos␣
↪estratificado)');
per_i_e.refline(y=0.025, linewidth=1, color='black', ls='--');
```

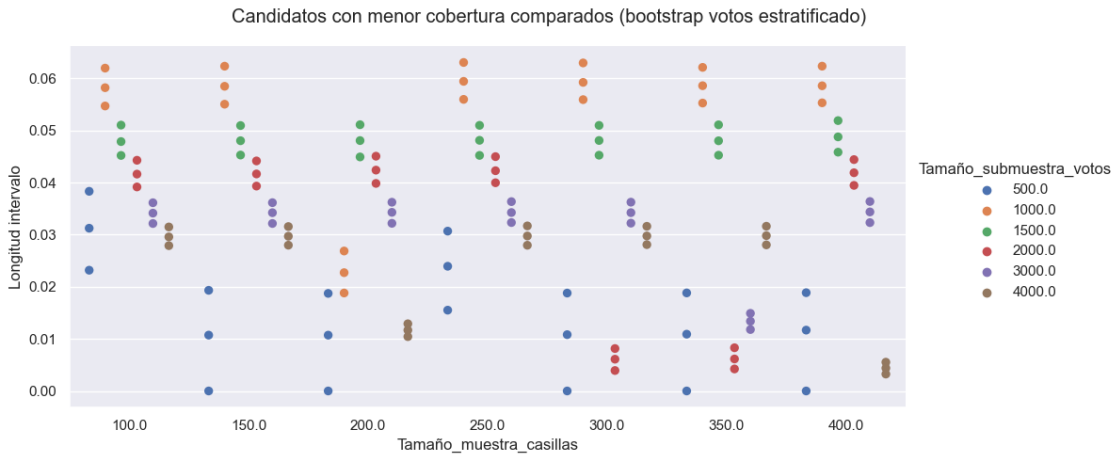


[280]:
```
fig, ax = plt.subplots(figsize=(8,6))

sns.pointplot(ax=ax,
    data=df_boostraps_votos_ec_f, x="Tamaño_muestra_casillas",␣
↪y="Intervalo_M_grande_candidato_cob", hue="Tamaño_submuestra_votos"
).set_title("Cobertura para intervalos más grandes (bootstrap votos␣
↪estratificado)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
ax.set(ylabel='Cobertura');
```

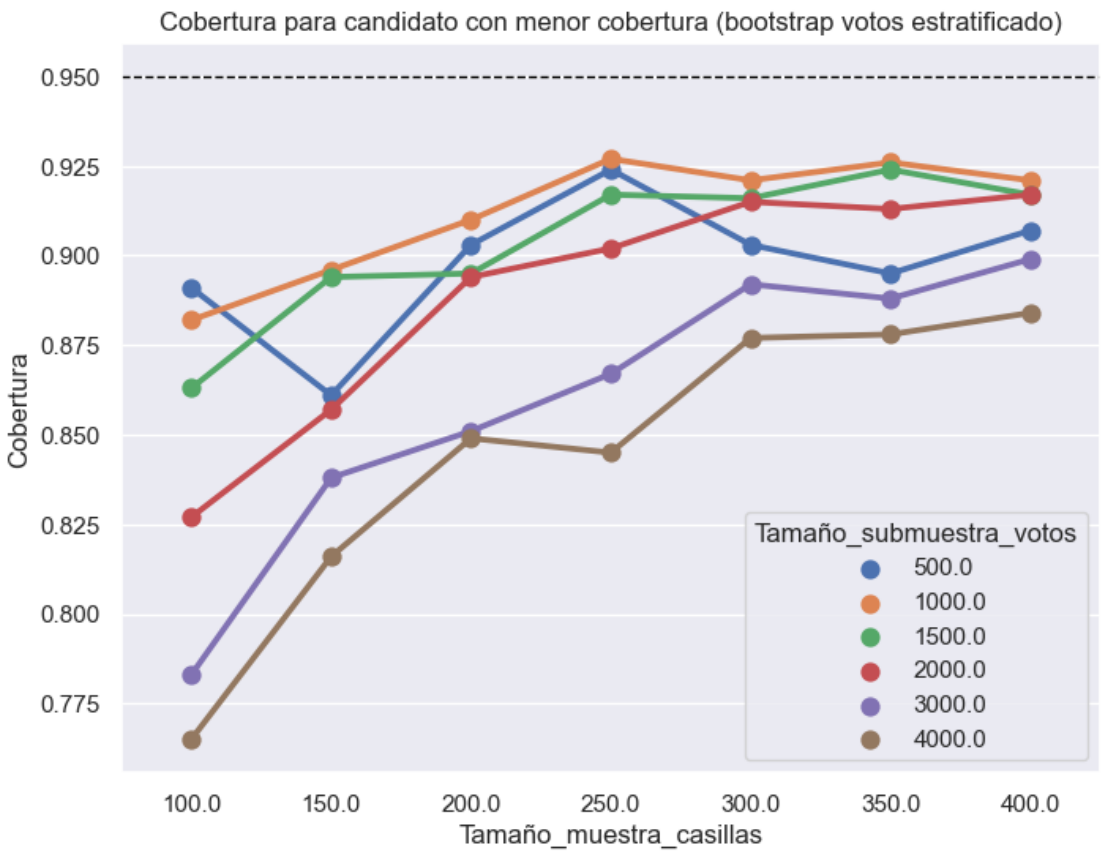


[ ]:
```
# Para hacer la gráfica de longitud de intervalo (para el candidato con los␣
↪intervalos más grandes) con los distintos tamños de la submuestra de votos
df_grafica_int_mc_aux_est=pd.
↪melt(df_boostraps_votos_ec_f[["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos", "Cobertura_m_inter_q_025",␣
↪"Cobertura_m_inter_mediana",␣
↪"Cobertura_m_inter_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Tamaño_submuestra_votos"], value_vars=["Cobertura_m_inter_q_025",␣
↪"Cobertura_m_inter_mediana", "Cobertura_m_inter_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_mc_e=sns.catplot(data=df_grafica_int_mc_aux_est,␣
↪x="Tamaño_muestra_casillas", y="value", hue="Tamaño_submuestra_votos",␣
↪kind='strip', aspect=2/1, palette="deep", jitter=False, dodge=True, s = 45);

per_mc_e.set(xlabel='Tamaño_muestra_casillas', ylabel='Longitud intervalo')
per_mc_e.figure.subplots_adjust(top=0.9)
```

```
per_mc_e.figure.suptitle('Candidatos con menor cobertura comparados (bootstrap␣
↪votos estratificado)');
```



[282]:
```
fig, ax = plt.subplots(figsize=(8,6))

sns.pointplot(ax=ax, data=df_boostraps_votos_ec_f, x="Tamaño_muestra_casillas",␣
↪y="Cobertura_m_candidato", hue="Tamaño_submuestra_votos").
↪set_title("Cobertura para candidato con menor cobertura (bootstrap votos␣
↪estratificado)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
ax.set(ylabel='Cobertura');
```

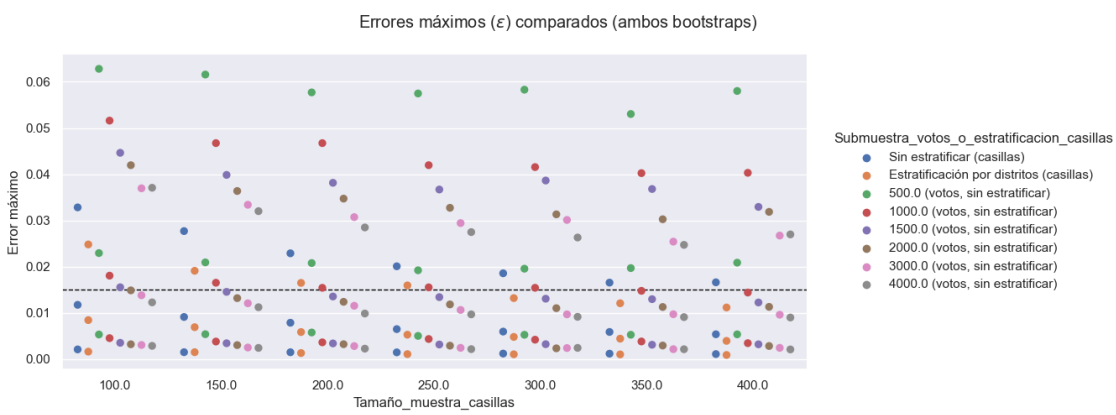


3.3 Juntamos el análisis

Primero juntamos el bootstrap de casillas (estratificado y no estratificado) con el bootstrap de votos no estratificado.

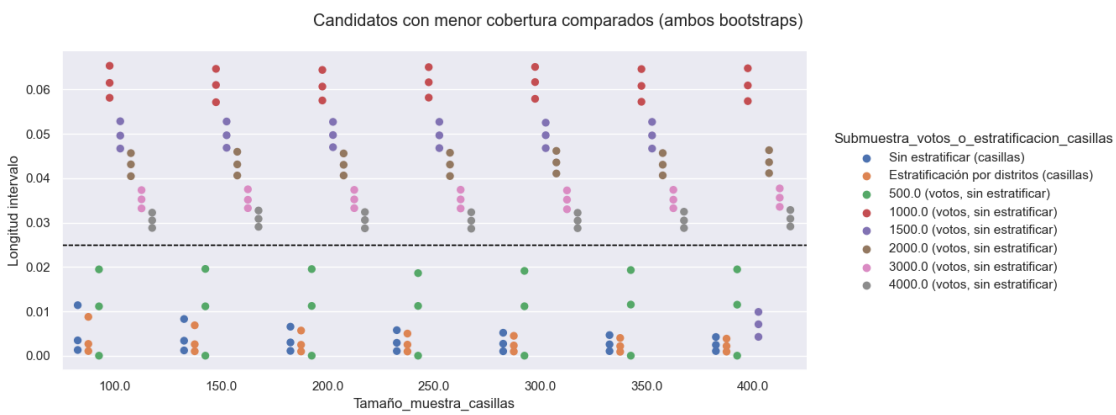
[152]:
```
# Guardamos todo en nuevas bases
df_boostraps_c_f=df_boostraps_c.copy().rename(columns={'Estratificación':
 ↪'Submuestra_votos_o_estratificacion_casillas'})
df_boostraps_votos_se_f=df_boostraps_votos_se.copy().
 ↪rename(columns={'Tamaño_submuestra_votos':
 ↪'Submuestra_votos_o_estratificacion_casillas'})
df_boostraps_votos_se_f["Submuestra_votos_o_estratificacion_casillas"]=df_boostraps_votos_se_f
 ↪astype(str)

df_boostraps_votos_se_f["Submuestra_votos_o_estratificacion_casillas"]=df_boostraps_votos_se_f
 ↪(votos, sin estratificar)'
# Juntamos las bases
df_bootraps_comb_1=pd.concat([df_boostraps_c_f,df_boostraps_votos_se_f]).
 ↪reset_index(drop=True)
```

[234]:
```
# Para hacer la gráfica del error con los distintos tamaños de la submuestra de␣
↪votos
df_grafica_error_aux_comb_1=pd.
↪melt(df_bootraps_comb_1[["Tamaño_muestra_casillas",␣
↪"Submuestra_votos_o_estratificacion_casillas", "Error_maximo_q_025",␣
↪"Error_maximo_mediana",␣
↪"Error_maximo_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Submuestra_votos_o_estratificacion_casillas"],␣
↪value_vars=["Error_maximo_q_025", "Error_maximo_mediana",␣
↪"Error_maximo_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_e=sns.catplot(data=df_grafica_error_aux_comb_1,␣
↪x="Tamaño_muestra_casillas", y="value",␣
↪hue="Submuestra_votos_o_estratificacion_casillas", kind='strip', aspect=2/1,␣
↪palette="deep", jitter=False, dodge=True, s = 45);
per_e.set(xlabel='Tamaño_muestra_casillas', ylabel='Error máximo')
per_e.figure.subplots_adjust(top=0.9)
per_e.figure.suptitle('Errores máximos ($\epsilon$) comparados (ambos␣
↪bootstraps)');
per_e.refline(y=0.015, linewidth=1, color='black', ls='--');
```

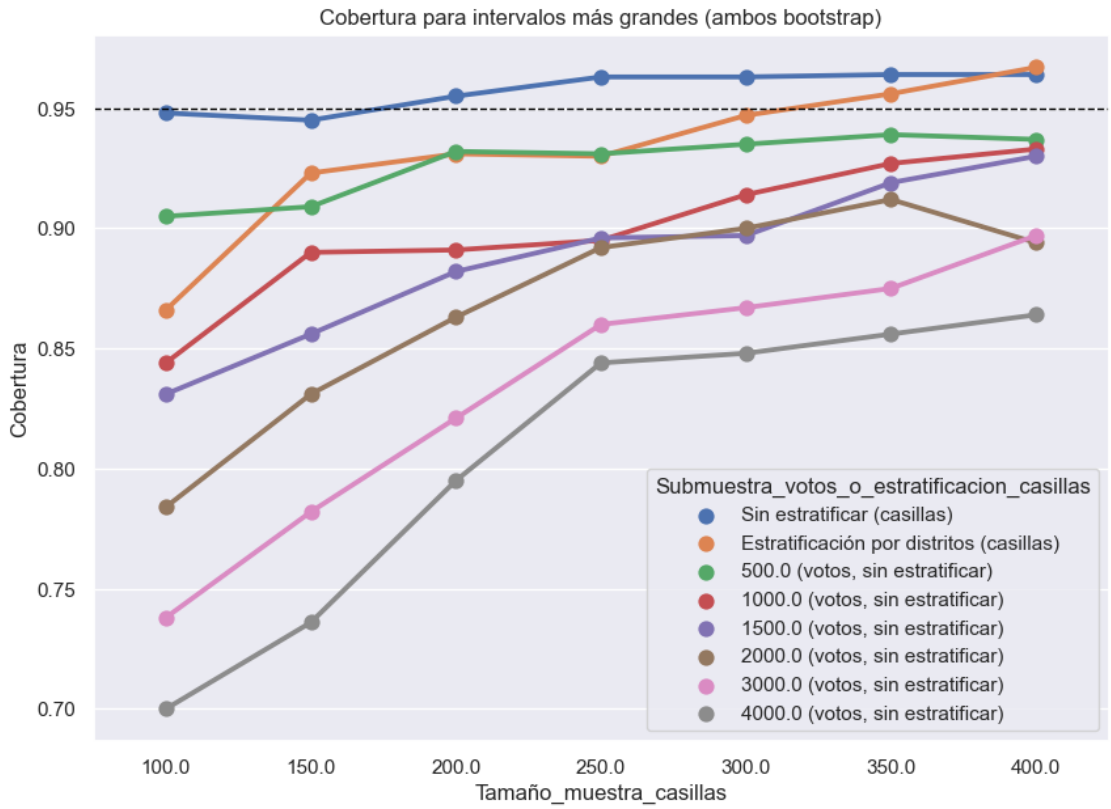


[233]:
```
# Para hacer la gráfica de longitud de intervalo (para el candidato con los␣
↪intervalos más grandes) con los distintos tamños de la submuestra de votos
```

```
df_grafica_int_im_aux_comb_1=pd.
 ↪melt(df_bootraps_comb_1[["Tamaño_muestra_casillas",␣
 ↪"Submuestra_votos_o_estratificacion_casillas",␣
 ↪"Intervalo_M_grande_inter_q_025", "Intervalo_M_grande_inter_mediana",␣
 ↪"Intervalo_M_grande_inter_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
 ↪"Submuestra_votos_o_estratificacion_casillas"],␣
 ↪value_vars=["Intervalo_M_grande_inter_q_025",␣
 ↪"Intervalo_M_grande_inter_mediana", "Intervalo_M_grande_inter_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
 ↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_i_e=sns.catplot(data=df_grafica_int_im_aux_comb_1,␣
 ↪x="Tamaño_muestra_casillas", y="value",␣
 ↪hue="Submuestra_votos_o_estratificacion_casillas", kind='strip', aspect=2/1,␣
 ↪palette="deep", jitter=False, dodge=True, s = 45);

per_i_e.set(xlabel='Tamaño_muestra_casillas', ylabel='Longitud intervalo')
per_i_e.figure.subplots_adjust(top=0.9)
per_i_e.figure.suptitle('Intervalos más grandes comparados (ambos bootstrap)');
per_mc_e.refline(y=0.025, linewidth=1, color='black', ls='--');
```

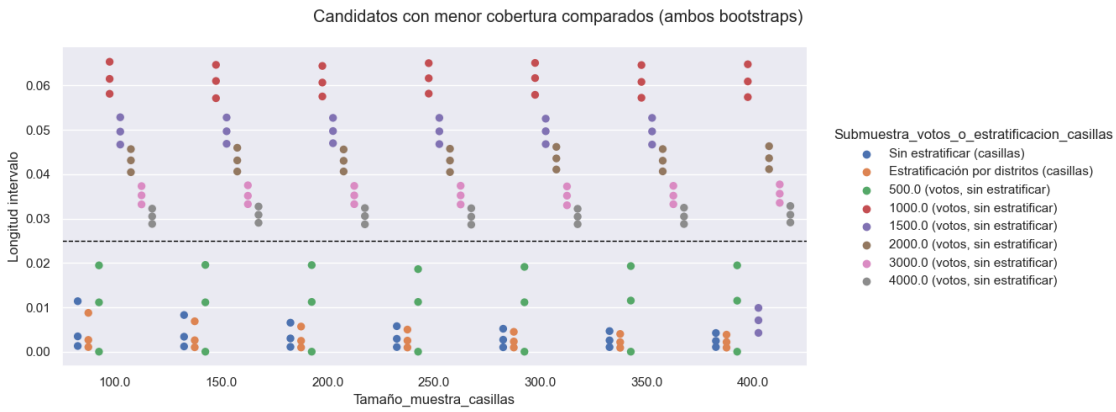


[283]:
```
fig, ax = plt.subplots(figsize=(10,7))

sns.pointplot(ax=ax,
    data=df_bootraps_comb_1, x="Tamaño_muestra_casillas",␣
 ↪y="Intervalo_M_grande_candidato_cob",␣
 ↪hue="Submuestra_votos_o_estratificacion_casillas"
).set_title("Cobertura para intervalos más grandes (ambos bootstrap)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
ax.set(ylabel='Cobertura');
```

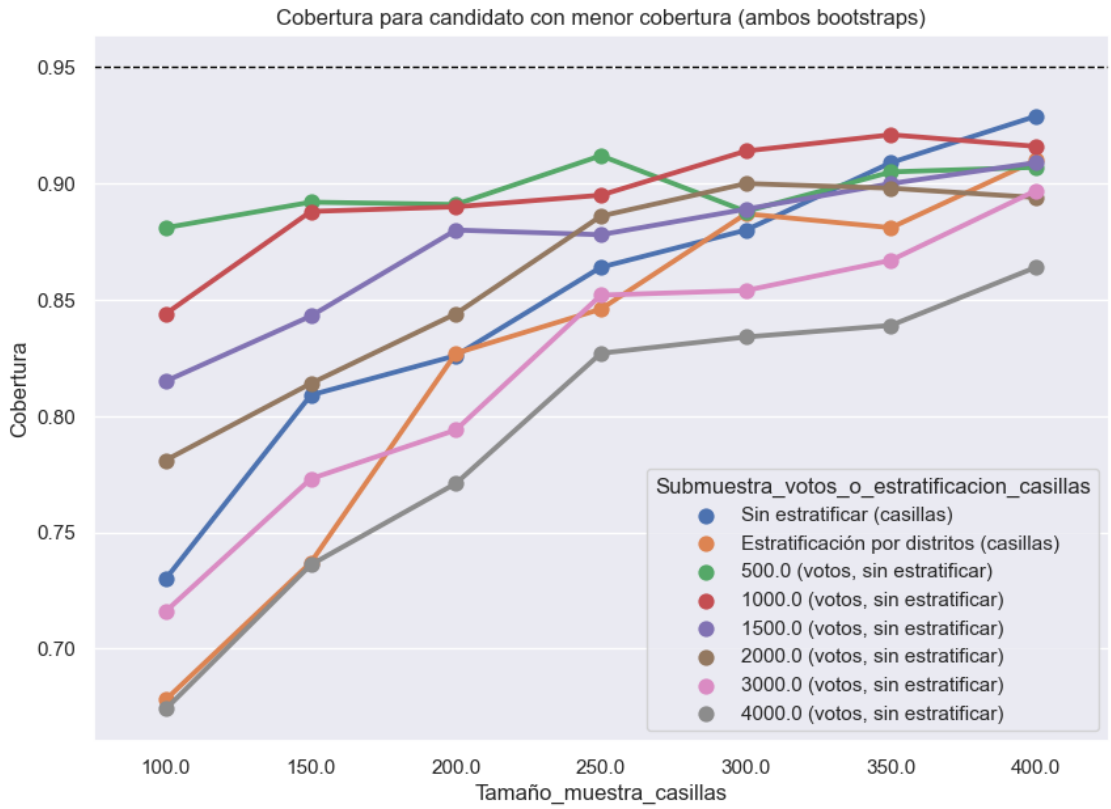


[231]:
```
# Para hacer la gráfica de longitud de intervalo (para el candidato con los␣
↪intervalos más grandes) con los distintos tamños de la submuestra de votos
df_grafica_int_mc_aux_comb_1=pd.
↪melt(df_bootraps_comb_1[["Tamaño_muestra_casillas",␣
↪"Submuestra_votos_o_estratificacion_casillas", "Cobertura_m_inter_q_025",␣
↪"Cobertura_m_inter_mediana",␣
↪"Cobertura_m_inter_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Submuestra_votos_o_estratificacion_casillas"],␣
↪value_vars=["Cobertura_m_inter_q_025", "Cobertura_m_inter_mediana",␣
↪"Cobertura_m_inter_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_mc_e=sns.catplot(data=df_grafica_int_mc_aux_comb_1,␣
↪x="Tamaño_muestra_casillas", y="value",␣
↪hue="Submuestra_votos_o_estratificacion_casillas", kind='strip', aspect=2/1,␣
↪palette="deep", jitter=False, dodge=True, s = 45);

per_mc_e.set(xlabel='Tamaño_muestra_casillas', ylabel='Longitud intervalo')
per_mc_e.figure.subplots_adjust(top=0.9)
```

```
per_mc_e.figure.suptitle('Candidatos con menor cobertura comparados (ambos␣
↪bootstraps)');
per_mc_e.refline(y=0.025, linewidth=1, color='black', ls='--');
```

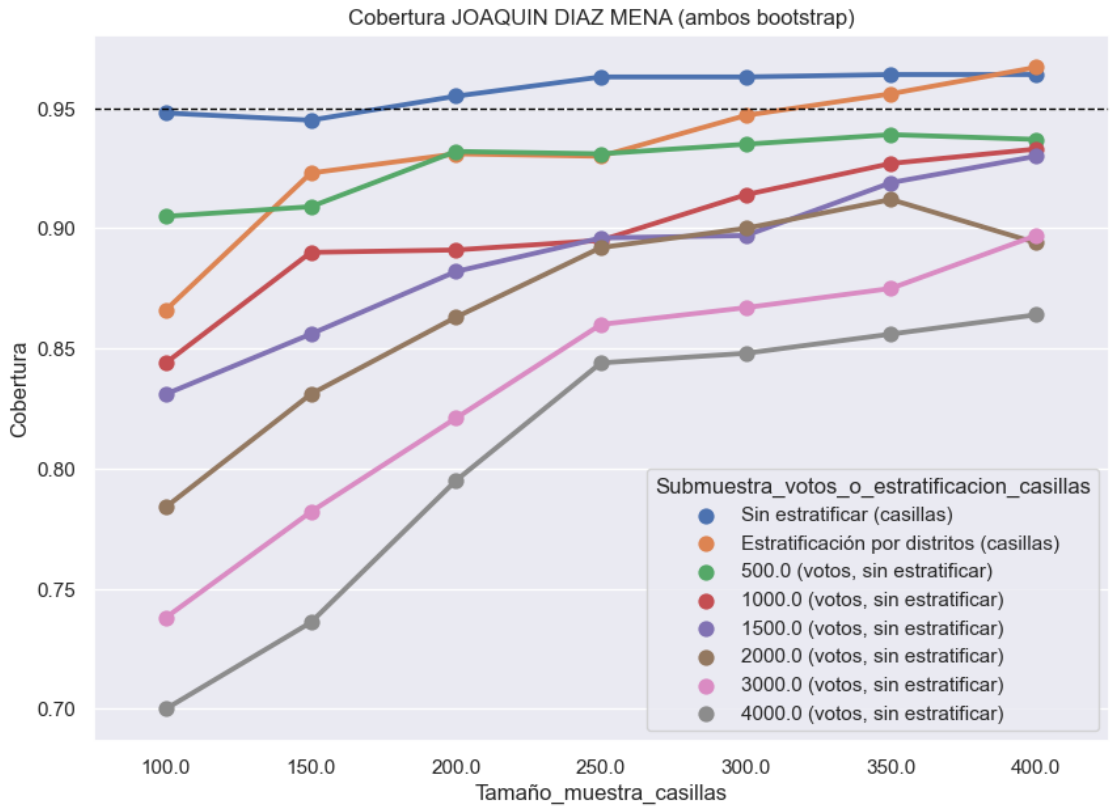


[284]:
```
fig, ax = plt.subplots(figsize=(10,7))
sns.pointplot(ax=ax, data=df_bootraps_comb_1, x="Tamaño_muestra_casillas",␣
↪y="Cobertura_m_candidato",
              hue="Submuestra_votos_o_estratificacion_casillas").
↪set_title("Cobertura para candidato con menor cobertura (ambos bootstraps)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
ax.set(ylabel='Cobertura');
```

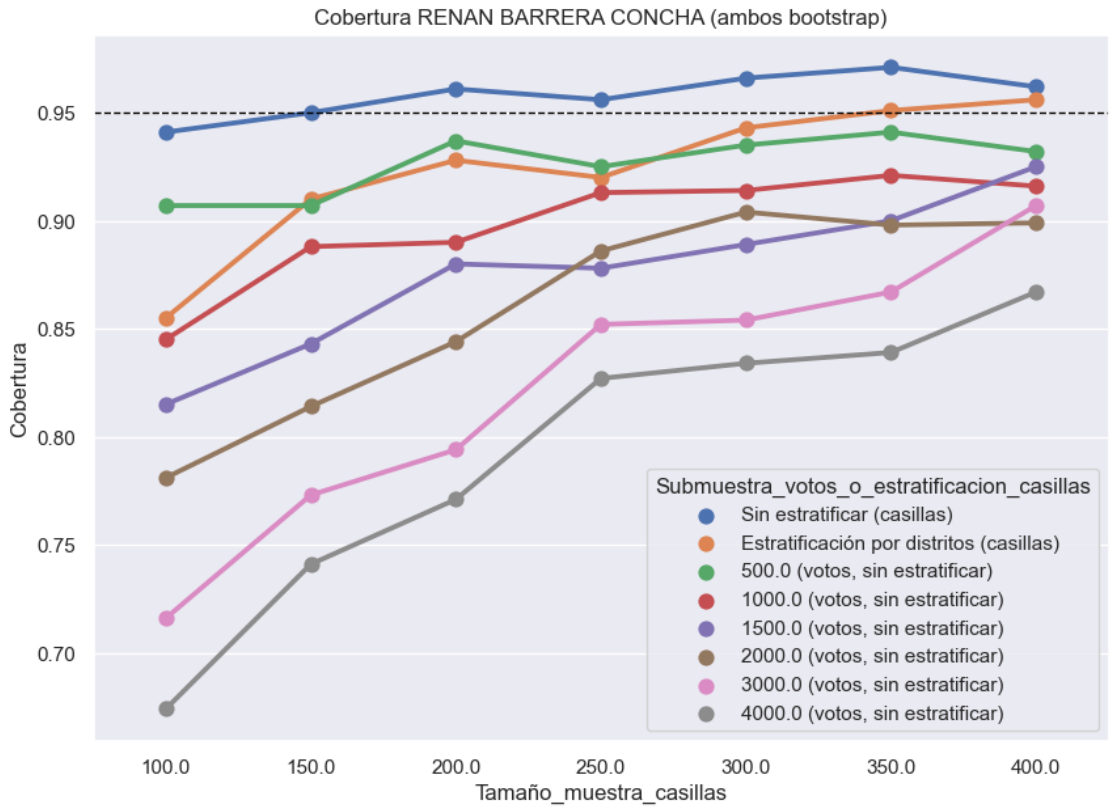


3.3.1 Gráficas de las coberturas por candidato

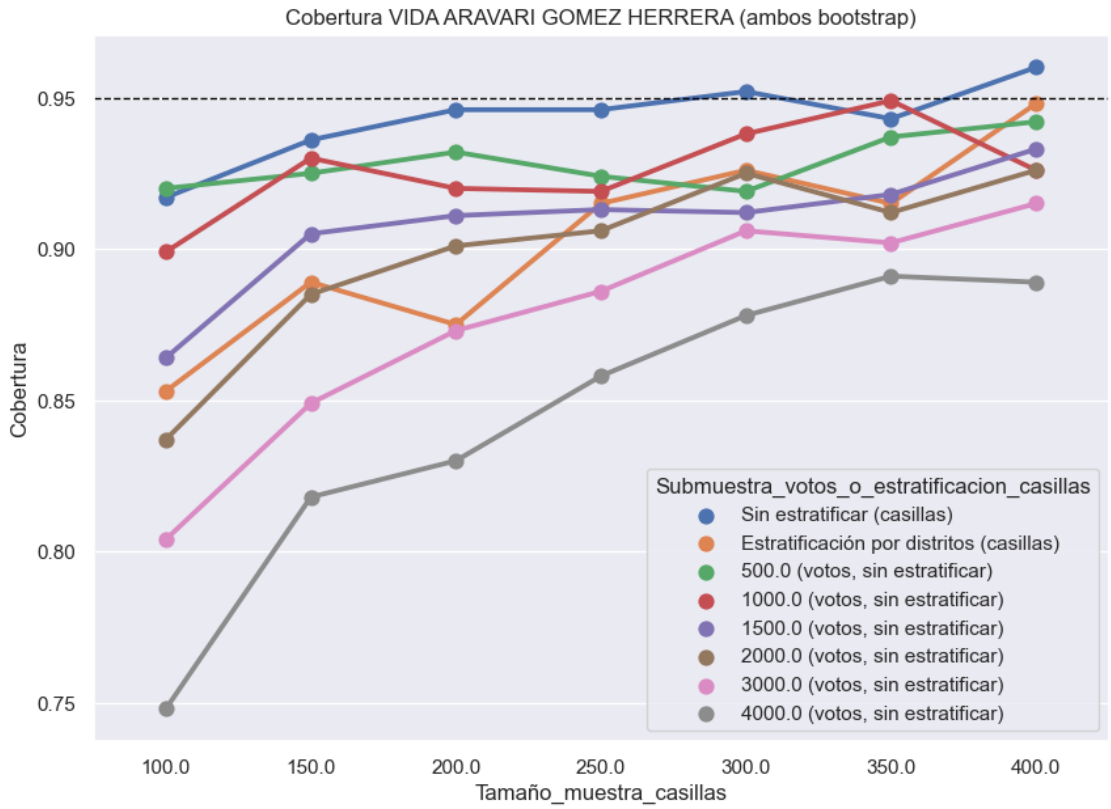
[228]:
```
fig, ax = plt.subplots(figsize=(10,7))

sns.pointplot(ax=ax,data=df_bootraps_comb_1, x="Tamaño_muestra_casillas",␣
↪y="Cob_JOAQUIN_DIAZ_MENA",
        hue="Submuestra_votos_o_estratificacion_casillas").set_title("Cobertura␣
↪JOAQUIN DIAZ MENA (ambos bootstrap)");
ax.set(ylabel='Cobertura');
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
```



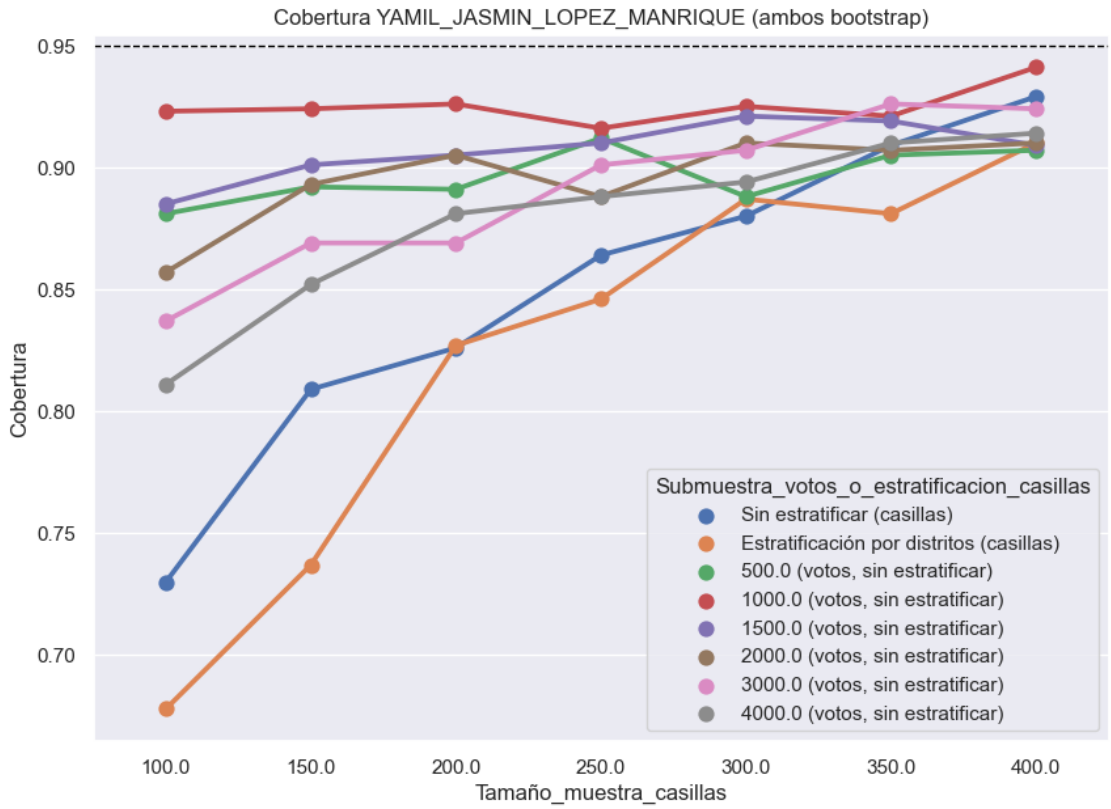
[227]:
```
fig, ax = plt.subplots(figsize=(10,7))
sns.pointplot(ax=ax,data=df_bootraps_comb_1, x="Tamaño_muestra_casillas",␣
↪y="Cob_RENAN_BARRERA_CONCHA",
        hue="Submuestra_votos_o_estratificacion_casillas").set_title("Cobertura␣
↪RENAN BARRERA CONCHA (ambos bootstrap)");
ax.set(ylabel='Cobertura');
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
```



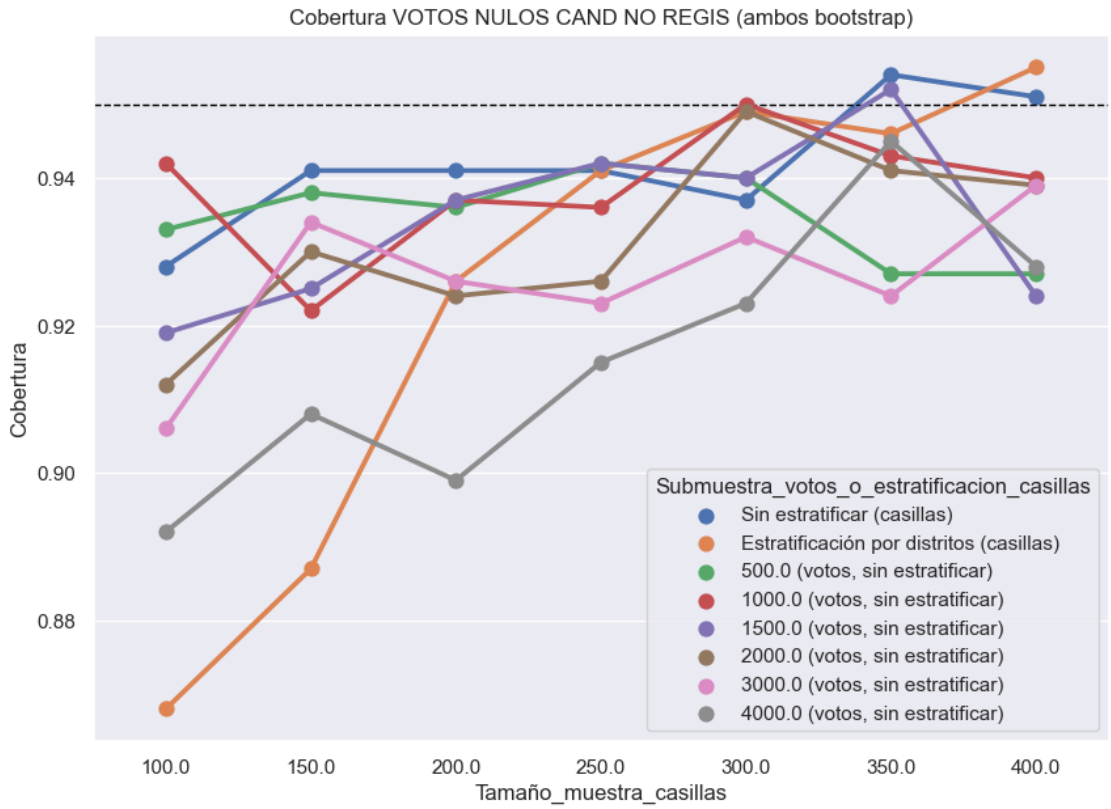
[226]:
```
fig, ax = plt.subplots(figsize=(10,7))

sns.pointplot(ax=ax,data=df_bootraps_comb_1, x="Tamaño_muestra_casillas",␣
↪y="Cob_VIDA_ARAVARI_GOMEZ_HERRERA",
        hue="Submuestra_votos_o_estratificacion_casillas").set_title("Cobertura␣
↪VIDA ARAVARI GOMEZ HERRERA (ambos bootstrap)");
ax.set(ylabel='Cobertura');
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
```



[225]:
```
fig, ax = plt.subplots(figsize=(10,7))

sns.pointplot(ax=ax,data=df_bootraps_comb_1, x="Tamaño_muestra_casillas",␣
↪y="Cob_YAMIL_JASMIN_LOPEZ_MANRIQUE",
        hue="Submuestra_votos_o_estratificacion_casillas").set_title("Cobertura␣
↪YAMIL_JASMIN_LOPEZ_MANRIQUE (ambos bootstrap)");
ax.set(ylabel='Cobertura');
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
```



[224]:
```
fig, ax = plt.subplots(figsize=(10,7))

sns.pointplot(ax=ax,data=df_bootraps_comb_1, x="Tamaño_muestra_casillas",␣
↪y="Cob_VOTOS_NULOS_CAND_NO_REGIS",
        hue="Submuestra_votos_o_estratificacion_casillas").set_title("Cobertura␣
↪VOTOS NULOS CAND NO REGIS (ambos bootstrap)");
ax.set(ylabel='Cobertura');
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
```

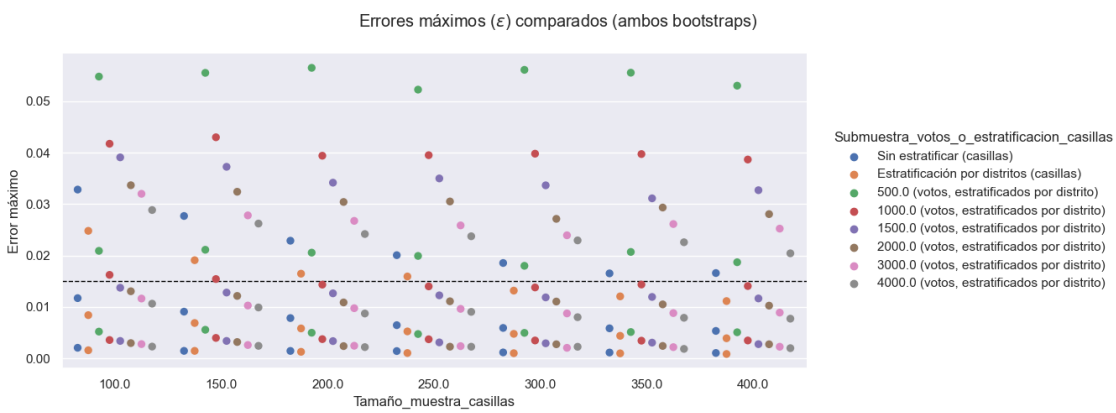


Posteriormente, juntamos el bootstrap de casillas (estratificado y no estratificado) con el bootstrap de votos estratificado por distritos

[186]:
```
# Guardamos todo en nuevas bases
df_boostraps_votos_ed_f=df_boostraps_votos_ec_f.copy().
 ↪rename(columns={'Tamaño_submuestra_votos':
 ↪'Submuestra_votos_o_estratificacion_casillas'})
df_boostraps_votos_ed_f["Submuestra_votos_o_estratificacion_casillas"]=df_boostraps_votos_ed_f
 ↪astype(str)

df_boostraps_votos_ed_f["Submuestra_votos_o_estratificacion_casillas"]=df_boostraps_votos_ed_f
 ↪(votos, estratificados por distrito)'
# Juntamos las bases
df_bootraps_comb_2=pd.concat([df_boostraps_c_f,df_boostraps_votos_ed_f]).
 ↪reset_index(drop=True)
```

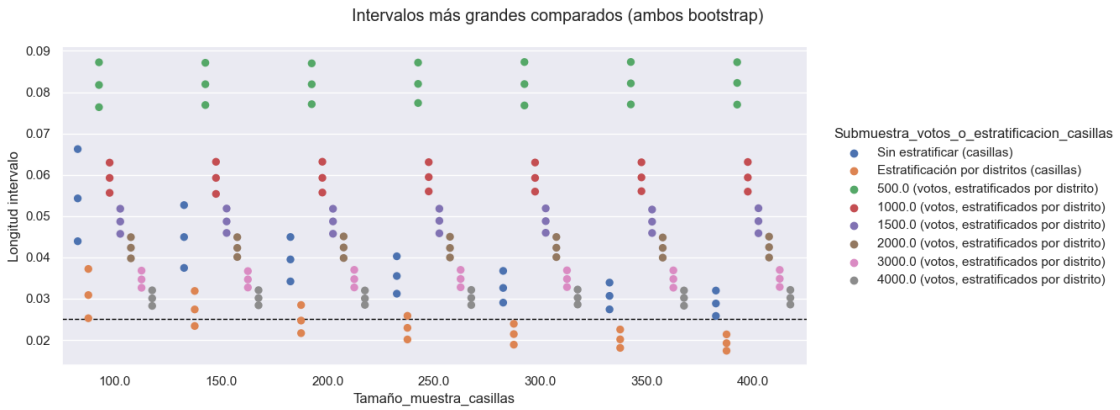
[223]:
```
# Para hacer la gráfica del error con los distintos tamaños de la submuestra de␣
 ↪votos
```

```
df_grafica_error_aux_comb_2=pd.
 ↪melt(df_bootraps_comb_2[["Tamaño_muestra_casillas",␣
 ↪"Submuestra_votos_o_estratificacion_casillas", "Error_maximo_q_025",␣
 ↪"Error_maximo_mediana",␣
 ↪"Error_maximo_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
 ↪"Submuestra_votos_o_estratificacion_casillas"],␣
 ↪value_vars=["Error_maximo_q_025", "Error_maximo_mediana",␣
 ↪"Error_maximo_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
 ↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_e=sns.catplot(data=df_grafica_error_aux_comb_2,␣
 ↪x="Tamaño_muestra_casillas", y="value",␣
 ↪hue="Submuestra_votos_o_estratificacion_casillas", kind='strip', aspect=2/1,␣
 ↪palette="deep", jitter=False, dodge=True, s = 45);
per_e.set(xlabel='Tamaño_muestra_casillas', ylabel='Error máximo')
per_e.figure.subplots_adjust(top=0.9)
per_e.figure.suptitle('Errores máximos ($\epsilon$) comparados (ambos␣
 ↪bootstraps)');
per_e.refline(y=0.015, linewidth=1, color='black', ls='--');
```

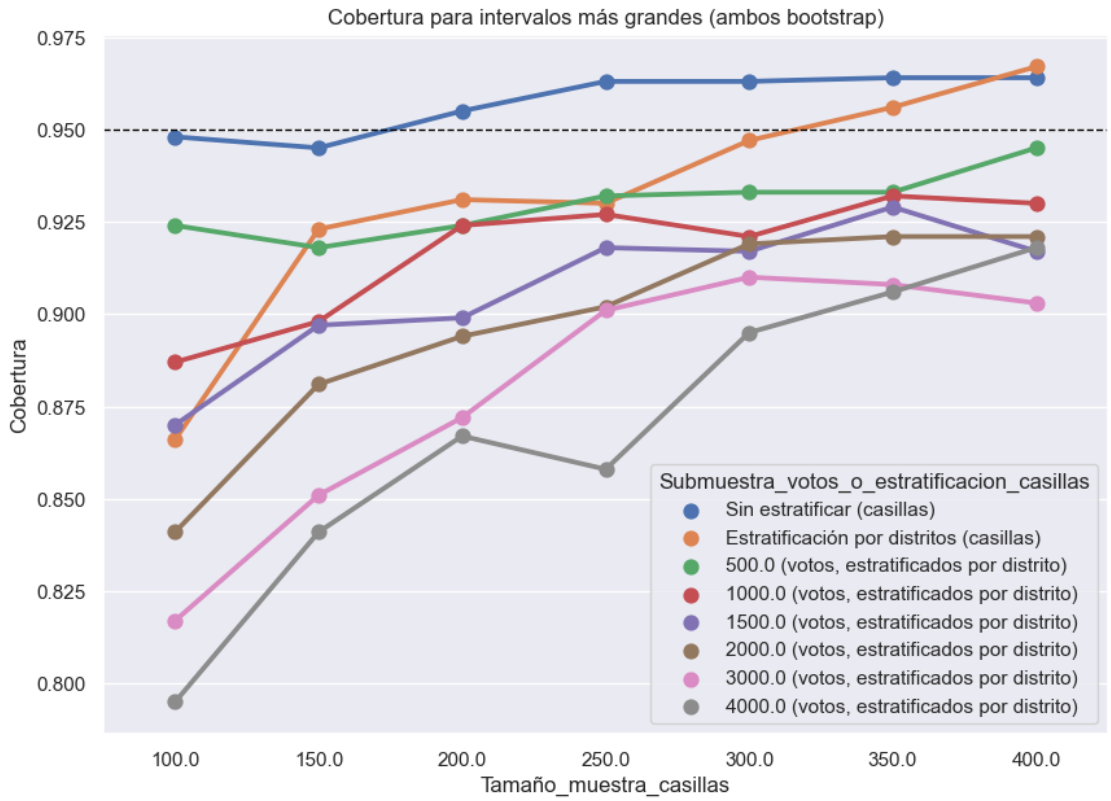


[221]:
```
# Para hacer la gráfica de longitud de intervalo (para el candidato con los␣
 ↪intervalos más grandes) con los distintos tamños de la submuestra de votos
df_grafica_int_im_aux_comb_2=pd.
 ↪melt(df_bootraps_comb_2[["Tamaño_muestra_casillas",␣
 ↪"Submuestra_votos_o_estratificacion_casillas",␣
 ↪"Intervalo_M_grande_inter_q_025", "Intervalo_M_grande_inter_mediana",␣
 ↪"Intervalo_M_grande_inter_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
 ↪"Submuestra_votos_o_estratificacion_casillas"],␣
 ↪value_vars=["Intervalo_M_grande_inter_q_025",␣
 ↪"Intervalo_M_grande_inter_mediana", "Intervalo_M_grande_inter_q_975"])
```

```
# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_i_e=sns.catplot(data=df_grafica_int_im_aux_comb_2,␣
↪x="Tamaño_muestra_casillas", y="value",␣
↪hue="Submuestra_votos_o_estratificacion_casillas", kind='strip', aspect=2/1,␣
↪palette="deep", jitter=False, dodge=True, s = 45);

per_i_e.set(xlabel='Tamaño_muestra_casillas', ylabel='Longitud intervalo')
per_i_e.figure.subplots_adjust(top=0.9)
per_i_e.figure.suptitle('Intervalos más grandes comparados (ambos bootstrap)');
per_i_e.refline(y=0.025, linewidth=1, color='black', ls='--');
```

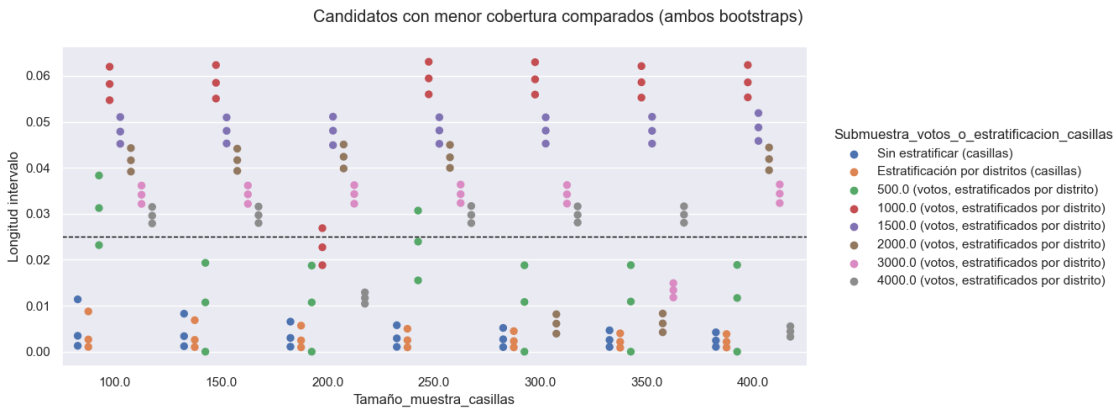


[285]:
```
fig, ax = plt.subplots(figsize=(10,7))

sns.pointplot(ax=ax,
    data=df_bootraps_comb_2, x="Tamaño_muestra_casillas",␣
↪y="Intervalo_M_grande_candidato_cob",␣
↪hue="Submuestra_votos_o_estratificacion_casillas"
).set_title("Cobertura para intervalos más grandes (ambos bootstrap)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
ax.set(ylabel='Cobertura');
```

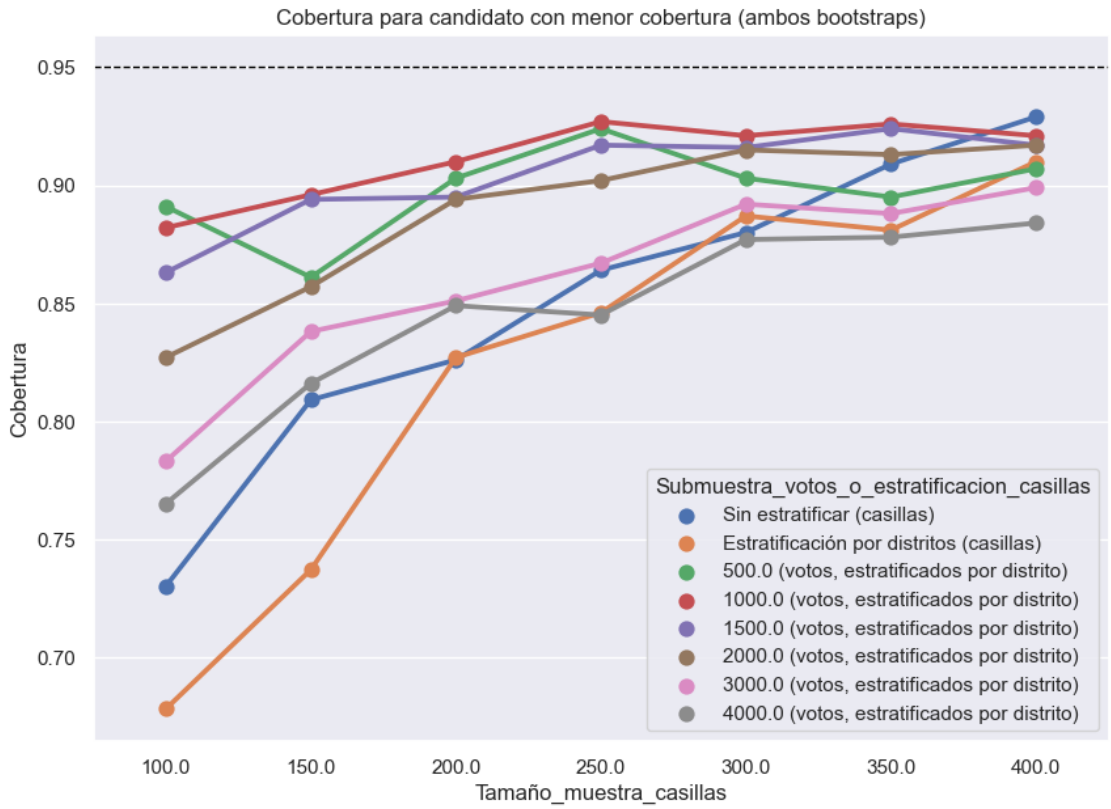


[219]:
```
# Para hacer la gráfica de longitud de intervalo (para el candidato con los␣
↪intervalos más grandes) con los distintos tamños de la submuestra de votos
df_grafica_int_mc_aux_comb_2=pd.
↪melt(df_bootraps_comb_2[["Tamaño_muestra_casillas",␣
↪"Submuestra_votos_o_estratificacion_casillas", "Cobertura_m_inter_q_025",␣
↪"Cobertura_m_inter_mediana",␣
↪"Cobertura_m_inter_q_975"]],id_vars=["Tamaño_muestra_casillas",␣
↪"Submuestra_votos_o_estratificacion_casillas"],␣
↪value_vars=["Cobertura_m_inter_q_025", "Cobertura_m_inter_mediana",␣
↪"Cobertura_m_inter_q_975"])

# Los tres puntos para cada categoría son los cuantiles 0.025, 0.5 (mediana) y␣
↪0.975 (esta fue la mejor forma de identificar hacer la gráfica)

per_mc_e=sns.catplot(data=df_grafica_int_mc_aux_comb_2,␣
↪x="Tamaño_muestra_casillas", y="value",␣
↪hue="Submuestra_votos_o_estratificacion_casillas", kind='strip', aspect=2/1,␣
↪palette="deep", jitter=False, dodge=True, s = 45);

per_mc_e.set(xlabel='Tamaño_muestra_casillas', ylabel='Longitud intervalo')
per_mc_e.figure.subplots_adjust(top=0.9)
```

```
per_mc_e.figure.suptitle('Candidatos con menor cobertura comparados (ambos␣
↪bootstraps)');
per_mc_e.refline(y=0.025, linewidth=1, color='black', ls='--');
```

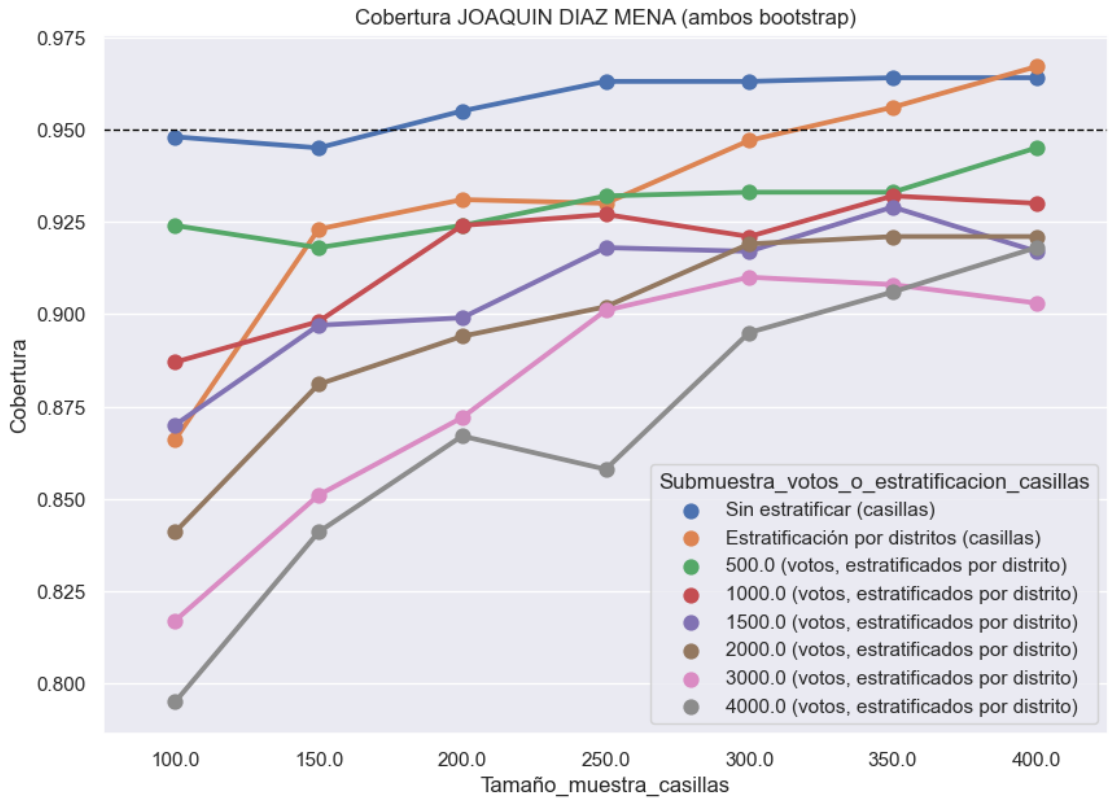


[286]:
```
fig, ax = plt.subplots(figsize=(10,7))
sns.pointplot(ax=ax, data=df_bootraps_comb_2, x="Tamaño_muestra_casillas",␣
↪y="Cobertura_m_candidato",
              hue="Submuestra_votos_o_estratificacion_casillas").
↪set_title("Cobertura para candidato con menor cobertura (ambos bootstraps)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--');
ax.set(ylabel='Cobertura');
```

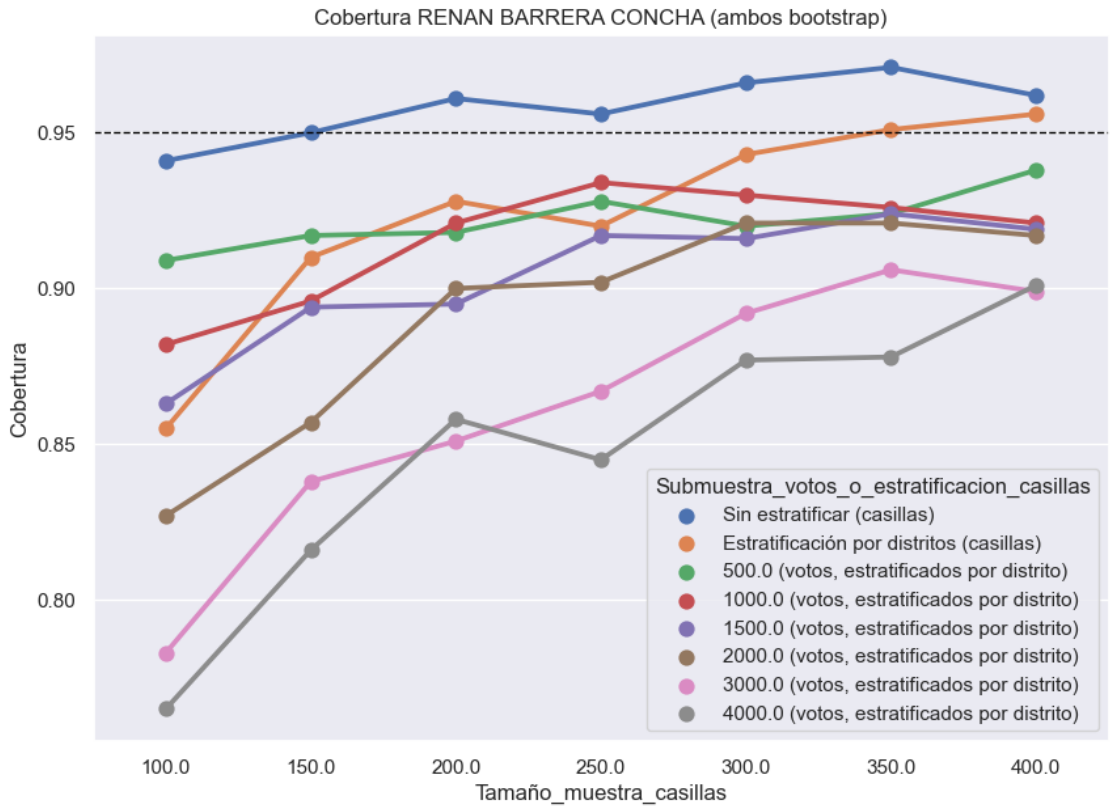


3.3.2 Gráficas de las coberturas por candidato

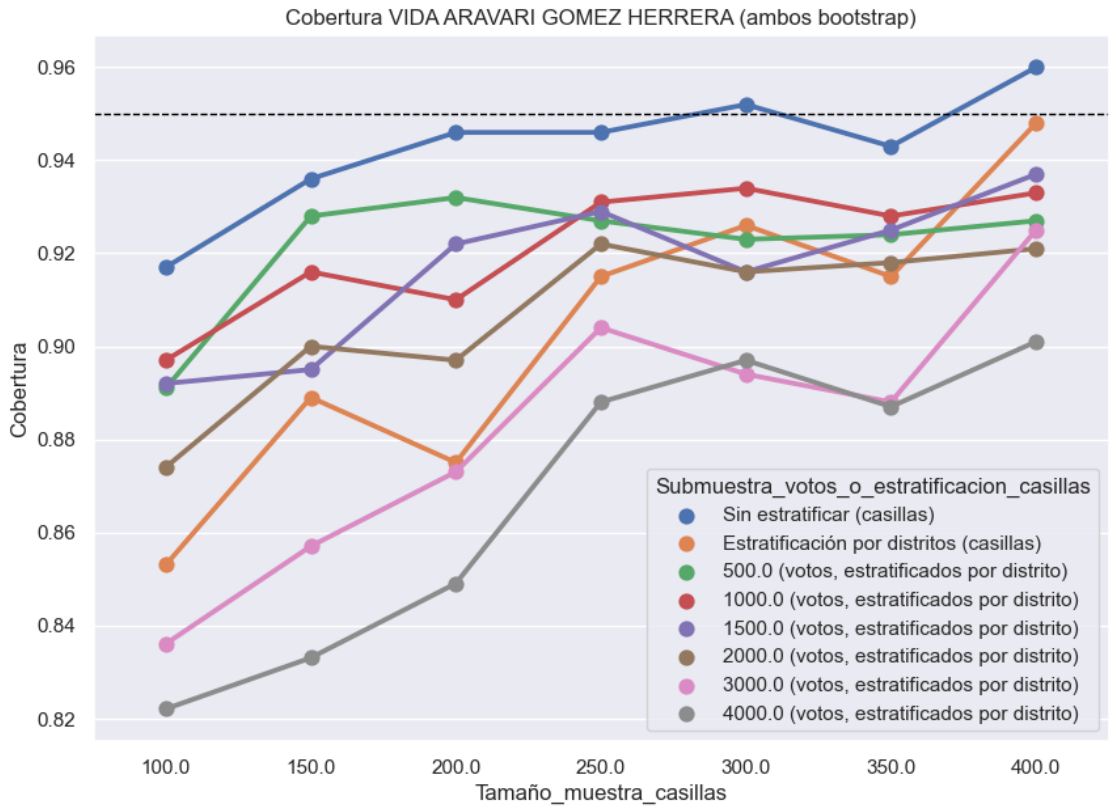
[212]:
```
fig, ax = plt.subplots(figsize=(10,7))

sns.pointplot(ax=ax,data=df_bootraps_comb_2, x="Tamaño_muestra_casillas",␣
 ↪y="Cob_JOAQUIN_DIAZ_MENA",
        hue="Submuestra_votos_o_estratificacion_casillas").set_title("Cobertura␣
 ↪JOAQUIN DIAZ MENA (ambos bootstrap)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--')
ax.set(ylabel='Cobertura');
```



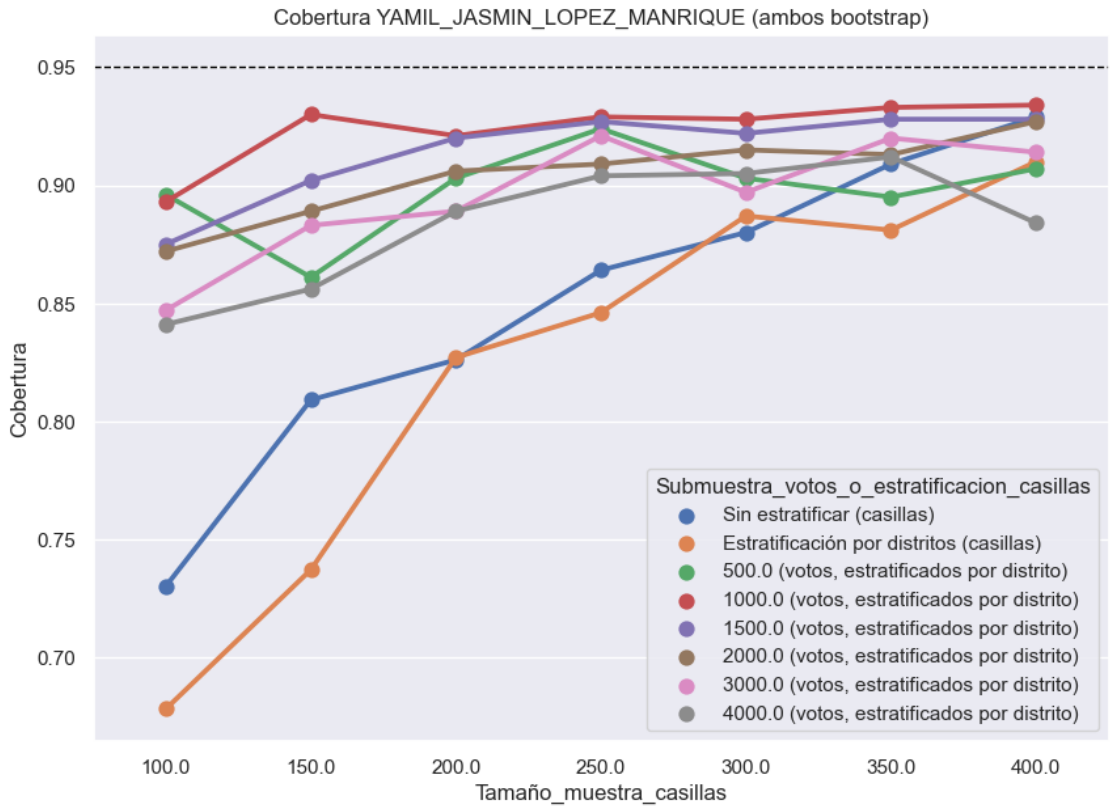
[213]:
```
fig, ax = plt.subplots(figsize=(10,7))
sns.pointplot(ax=ax,data=df_bootraps_comb_2, x="Tamaño_muestra_casillas",␣
↪y="Cob_RENAN_BARRERA_CONCHA",
        hue="Submuestra_votos_o_estratificacion_casillas").set_title("Cobertura␣
↪RENAN BARRERA CONCHA (ambos bootstrap)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--')
ax.set(ylabel='Cobertura');
```



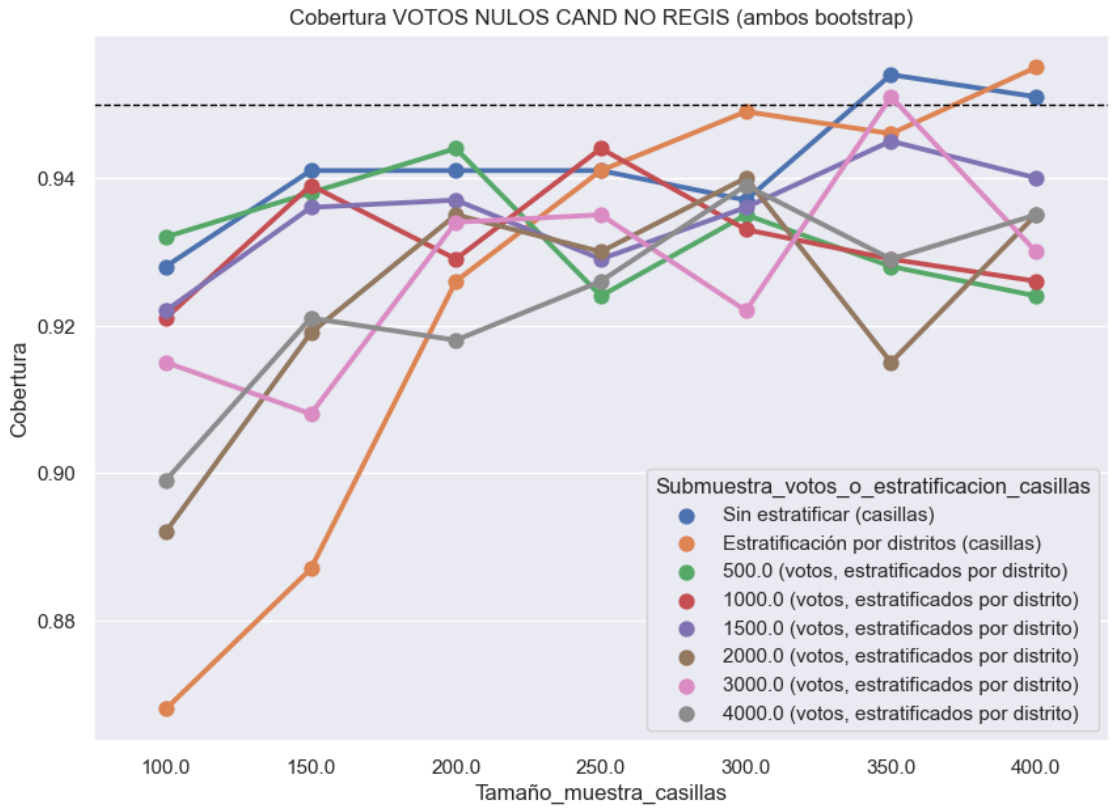
[211]:
```
fig, ax = plt.subplots(figsize=(10,7))

sns.pointplot(ax=ax,data=df_bootraps_comb_2, x="Tamaño_muestra_casillas",␣
↪y="Cob_VIDA_ARAVARI_GOMEZ_HERRERA",
        hue="Submuestra_votos_o_estratificacion_casillas").set_title("Cobertura␣
↪VIDA ARAVARI GOMEZ HERRERA (ambos bootstrap)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--')
ax.set(ylabel='Cobertura');
```



[210]:
```
fig, ax = plt.subplots(figsize=(10,7))

sns.pointplot(ax=ax,data=df_bootraps_comb_2, x="Tamaño_muestra_casillas",␣
↪y="Cob_YAMIL_JASMIN_LOPEZ_MANRIQUE",
        hue="Submuestra_votos_o_estratificacion_casillas").set_title("Cobertura␣
↪YAMIL_JASMIN_LOPEZ_MANRIQUE (ambos bootstrap)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--')
ax.set(ylabel='Cobertura');
```



[209]:
```
fig, ax = plt.subplots(figsize=(10,7))

sns.pointplot(ax=ax,data=df_bootraps_comb_2, x="Tamaño_muestra_casillas",␣
↪y="Cob_VOTOS_NULOS_CAND_NO_REGIS",
        hue="Submuestra_votos_o_estratificacion_casillas").set_title("Cobertura␣
↪VOTOS NULOS CAND NO REGIS (ambos bootstrap)");
ax.axhline(y=0.95, linewidth=1, color='black', ls='--')
ax.set(ylabel='Cobertura');
```



3.4 Conclusiones

Los siguientes puntos aplican para el muestreo estratificado y no estratificado de votos - En terminos generales el bootstrap utilizando submuestreo de votos (boostrap de votos) tiene peores resultados que utilizando el bootstrap de solo casillas. Más aún, los errores, en el caso del bootstrap por casillas van disminuyendo conforme aumenta el número de casillas mientras que en el bootstrap por votos los errores se dismuyen muy poco con respecto a las casillas y realmente se ve su reducción cuando se incrementa el número de votos de la submuestra. Además, el decrecimiento del error es más rápido en el bootstrap de casillas que en el bootstrap de votos. Por lo tanto, a partir de 300 casillas el bootstrap por casillas tiene menor error (en el caso del estratificado), o el error es igual o ligeramente menor (en el caso de no estratificado) que la submuestra de votos con error más pequeño. En las longitudes de los intervalos de los candidatos con intervalos más grandes, sucede algo analógo a los errores. La única diferencia es que la longitud de los intervalos del bootstrap de votos no disminuye cuando se aumenta el número de casillas. - Por otra parte, para los candidatos con mayores porcentajes de votos las coberturas del candidato con menores cobertura son mayores en el bootstrap de casillas. Mientras que, para las coberturas del candidato con el menor cobertura el bootstrap por votos es igual o ligeramente mejor al bootstrap por casillas esto se debe a que: - En el caso del bootstrap por casillas el candidato con menor cobertura siempre es el mismo, el candidato con menor porcentaje de votos, (alrededor del 0.52%). Mientras que en el bootstrap por votos el candidato con menor cobertura va cambiando y no siempre es el mismo. - Las coberturas de los candidatos en el bootstrap por votos tienden a no tener mucha diferencia entre candidatos y todas están alrededor de 0.88 a 0.93, y aunque normalmente aumentan conforme

se incrementa el número de casillas, este incremento no es uniforme ni tan rápido como el bootstrap de casillas. - Por otro lado, las coberturas de los candidatos tienden a ser más grandes para aquellos que tengan mayor porcentaje de votos mayor y en particular la cobertura para el candidato con menor porcentaje de votos es mucho menor comparada con el resto de candidatos (no llega al 95%). - Las diferencias entre las coberturas de los bootstraps se puede explicar porque en el caso del bootstrap por votos, se está submuestreando los votos de las casillas muestreadas por lo que es más probable que si sean muestreados todos los candidatos (sobre todo si se tiene una muestra de casillas suficientemente grande). En cambio, en el bootstrap por casillas como se muestrean las casillas, puede ocurrir que en simulaciones de la población se seleccionen casillas que no contengan votos del candidato con el menor porcentaje. - Específicamente, para el boostrap de votos, sin importar la submuestra de votos (hasta 4000) o número de casillas (hasta 400), los intervalos de probabilidad de los errores siempre son mayores a 0.15. Aunado a esto, entre más crece el tamaño de la submuestra de votos menor son los intervalos de probabilidad del error. Esto se debe a que si tomamos mayor población mejor se representa la muestra de las casillas y por ende el error (que es el promedio de las estimaciones de los bootstrap menos el valor) se acerca más al valor real (a la muestra de casillas). Por otra parte, al aumentar el número de casillas también decrece el tamaño de los intervalos del error, aunque el decrecimeinto es más pequeño. Esto dado que, al tomar más casillas de la población y subsecuentemente al tomar votos de la misma, te aproximas más a muestrar aleatoriamente de toda la población. Lo cual reduce el error (el cual se calcula restando el promedio de estimación menos el valor real). - En el muestreo de votos, con respecto a la longitud de los intervalo (de porcentaje de votos), estos no cambian (o el cambia muy poco) al aumentar las casillas puesto que los intervalos solo se ven afectados por el tamaño de la submuestra de votos. Entre más votos tenga la submuestra mayor será su aproximación a la muestra de casillas y por ende habrá menor variabilidad. Nota: Esto solo aplica hasta cierto número de votos de la muestra, no es bueno tomar toda la muestra, en especial si se muestrearon pocas casillas. Para el candidato con intervalos más grandes, en ninguna combinación de submuestra de votos y muestra de casillas estos intervalos son menores a 0.025. - En el muestreo de votos, las coberturas de los candidatos, aumentan entre más casillas se consideran, puesto que, al tomar más casillas de la población y subsecuentemente al tomar votos de la misma, te aproximas más a muestrar aleatoriamente de toda la población. En general tiene mejor cobertura tomar una menor cantidad de votos aunque tomar entre 500 a 2000 votos de submuestra tienen coberturas parecidas, sobre todo al tomar más de 200 casillas. Mientras que tomar más de 2000 votos de submuestra nos da las peores coberturas, menores o cercanas a 0.9 para el candidato con menor cobertura aún 400 casillas. Lo anterior se debe a que si se toma una submuestra mayor de votos (más de 2000) en una muestra de casillas pequeña, la variabilidad de los pesos Dirichlet es menor y por ende la población generada por el boostrap se va a asemeja más a la muestra de casillas, la cual no siempre es representativa de toda la población y por ende la cobertura es menor. Por lo que, al aumentar la muestra de casillas, la muestra es más representativa de la población y por lo tanto, la cobertura con submuestras más grandes de votos (más de 2000) se acerca a la cobertura de las submuestras más pequeñas (algo similar ocurre con el bootstrap frecuentista). Por último, en ninguna combinación de submuestra de votos y muestra de casillas, ningún candidato (o casi ninguno) alcanza la cobertura del 95%.

Este punto es para la comparación entre el muestreo estratificado y no estratificado de votos - Por último, al utilizar la estratificación (por distrito) en el bootstrap de votos, se reduce el error y los cuantiles de las longitudes de los intervalos comparado con el bootstrap de votos sin estratificar. Mientras que las coberturas de los candidatos son ligeramente mejores en el bootstrap de votos sin estratificar que en el estratificado, aunque la diferencia no es mucha.

3.5 Pendientes (dudas)

- Duda: ¿En el bootstrap de votos es mejor utilizar el tamaño de la población como el número de votos totales o cómo el número de votos en la muestra? Puesto que, en la realidad no se tiene el número de votos (se podría estimar pero tal vez no sería rápido o preciso) (al momento de la elección). A diferencia del número de casillas el cuál si se sabe al momento de la elección, dado que este se establece antes. Lo que si sabría al momento de la elección sería el número de votos dentro de la muestra de casillas, por lo que ese número de votos podría ser el más apropiado.

[ ]:

```

```

[ ]:

```
def vectorized_choice(p, n, items=None):
    s = p.cumsum(axis=1)
    r = np.random.rand(p.shape[0], n, 1)
    q = np.expand_dims(s, 1) >= r
    k = q.argmax(axis=-1)
    if items is not None:
        k = np.asarray(items)[k]
    return k
```

[ ]:

```
import timeit

start = timeit.default_timer()

vectorized_choice(pesos_bootstrap, n_votos-int(df_stra_sam.sum()), items=None)

stop = timeit.default_timer()

print('Time: ', stop - start)
```

```
Time:  44.074410042001546
```

[ ]:

```
t=vectorized_choice(pesos_bootstrap, 100000, items=None)
t
```

```
array([[1, 0, 0, …, 1, 1, 1],
       [1, 1, 2, …, 1, 0, 1],
       [0, 0, 1, …, 1, 1, 0],
       …,
       [1, 1, 1, …, 0, 1, 1],
       [2, 0, 2, …, 0, 1, 1],
       [0, 0, 0, …, 1, 0, 0]])
```